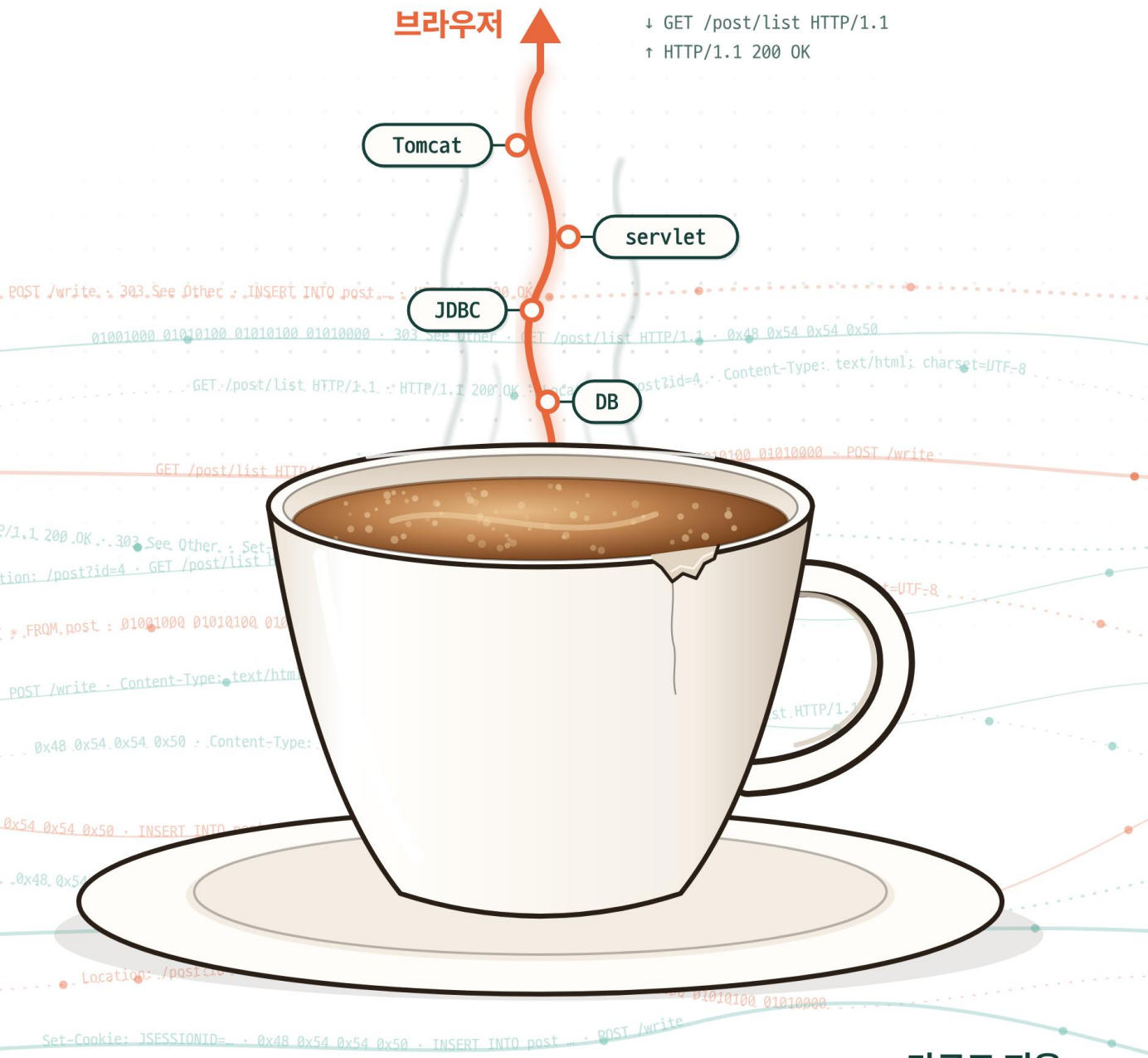
소켓에서 블로그까지, 직접 만들며 따라가는 자바 웹

# 웹 애플리케이션은 어떻게 동작하는가




마르코 지음

# 웹 애플리케이션은 어떻게 동작하는가

검토본

마르코 지음

**웹 애플리케이션은 어떻게 동작하는가**

검토본
[1판 1쇄 펴낸 날 자리]

지은이 마르코 [저자 이름 자리표시]
펴낸이 [자리]
펴낸곳 [출판사 자리]
출판 등록 [등록 번호와 날짜 자리]
주소 [자리]
전화 [자리] · 팩스 [자리]
홈페이지 [자리] · 전자우편 [자리]

ISBN [종이책 ISBN 자리]
ISBN [전자책 ISBN 자리]
값 [자리]

© 2026 마르코 [저작권자 표기 자리]

이 책은 저작권법에 따라 보호받는 저작물이므로 무단 전재와 무단 복제를 금합니다. 이 책 내용의 전부 또는 일부를 이용하려면 저작권자와 [출판사 자리]의 서면 동의를 받아야 합니다.

[잘못 만든 책의 교환 안내: 출판사 문안 자리]

이 책에 나오는 Java, Maven, Gradle, Tomcat, Spring, MySQL, Docker, GitHub, IntelliJ IDEA, Visual Studio Code, Windows, macOS, Eclipse Temurin 등 제품과 서비스의 이름은 각 소유자의 상표 또는 등록 상표입니다. 본문에서는 ™, ® 표시를 생략했습니다. 이 책은 언급한 회사와 제휴하거나 후원을 받지 않았고, 각 회사가 이 책을 검토하거나 보증하지 않았습니다.

화면을 캡처한 그림을 빼면, 이 책의 그림은 모두 저자가 직접 그렸습니다. 다른 책이나 인터넷의 그림, 아이콘을 가져오지 않았습니다. 다른 사이트나 프로그램의 화면을 캡처한 그림은 캡션 끝에 출처와 캡처 시기를 적었습니다.

종이책의 글꼴은 본문 Noto Serif KR, 제목과 상자와 표 Pretendard, 코드 D2Coding이며, 세 글꼴 모두 SIL Open Font License 1.1에 따라 썼습니다. D2Coding에 없는 일부 기호는 DejaVu Sans Mono(Bitstream Vera·DejaVu 라이선스)로 찍었습니다. 전자책은 읽는 기기의 글꼴을 따릅니다.

실습 저장소의 코드는 MIT 라이선스로 공개합니다. 누구나 자유롭게 쓰고, 고치고, 다시 배포할 수 있습니다.

# 머리말

프레임워크와 AI 도구의 도움을 받으면 블로그 하나를 만들어 실행하기까지 몇 분이면 충분합니다. 글을 쓰면 목록에 올라오고, 수정을 누르면 고치는 화면이 열립니다. 편한 만큼 그 아래에서 무슨 일이 일어나는지는 잘 보이지 않습니다. 그 화면 앞에서 저는 스스로 물었습니다. “이 블로그는 어떻게 동작하는 걸까?”

프레임워크와 라이브러리는 많은 일을 대신해 줍니다. 그래도 대신해 주는 일이 무엇인지 모르면, 잘못되었을 때 어디를 봐야 하는지도 모릅니다. 응답의 길이를 바이트가 아닌 글자 수로 적은 서버도, 요청마다 다른 값을 Servlet의 필드에 담은 코드도, 데이터베이스 연결을 닫지 않는 글 저장소도 처음에는 화면에 글을 잘 보여 줍니다. 동작하는 원리를 모르면 맞는 코드와 그럴듯하게 틀린 코드가 똑같아 보입니다.

3장에서 두원이 처음 만든 서버가 그랬습니다. 영어로 시험할 때는 잘 나오던 화면이, 한글이 들어가자 끝이 잘려서 나왔습니다. 코드에는 오류가 없었고 서버도 멈추지 않았습니다. 원인은 응답 헤더에 적은 숫자 하나였습니다. 브라우저는 그 숫자만큼만 읽고 나머지를 버렸고, 그 숫자는 바이트가 아니라 글자 수였습니다. 프레임워크를 쓰면 이런 숫자를 직접 적을 일이 없습니다. 편하기는 하지만, 비슷한 일이 다른 모양으로 일어났을 때 어디부터 봐야 할지 짐작하기도 어렵습니다.

그래서 이 책은 프레임워크에 맡기기 전에 웹 애플리케이션이 구동되는 바탕을 직접 만들어 봅니다. 자바 파일 하나를 JAR로 묶는 데서 시작해 로그를 남기고, 소켓으로 HTTP 서버를 세우고, Servlet과 JSP로 블로그를 올리고, JDBC로 글을 데이터베이스에 저장합니다. 그 과정에서 모든 요청을 먼저 받는 입구, 객체를 만들어 맡아 두는 장치, 메서드 앞뒤에 끼어드는 proxy를 차례로 만듭니다. 마지막 12장에서 블로그를 Spring MVC로 옮기면, 직접 만든 조각들이 프레임워크 안에서 어떤 이름을 달고 자동으로 돌아가는지 한눈에 보입니다.

목표는 특정 프레임워크를 익히는 것이 아닙니다. 요청 하나가 소켓과 스레드를 지나 Servlet container와 내 코드에 닿았다가 응답으로 돌아오기까지 무슨 일이 일어나는지 설명할 수 있게 되는 것입니다. 이 기본기는 어떤 언어와 프레임워크로 개발하든 그대로 통합니다. 도구의 이름은 바뀌어도 바탕에서 일어나는 일은 쉽게 바뀌지 않기 때문입니다.

AI를 활용해 개발하더라도 이 기본기는 꼭 갖춰야 합니다. 오히려 AI 도구가 코드를 써 주는 지금 더 필요해졌습니다. 도구는 순식간에 그럴듯한 코드를 내놓습니다. 하지만 그 코드가 요청을 끝까지 읽는지, 여러 스레드가 함께 써도 안전한지, 연결을 제때 돌려주는지는 받아 든 사람이 판단해야 합니다. 무엇을 물어야 할지 알아야 좋은 답을 얻고, 답이 틀렸을 때 알아차릴 수 있습니다. 이 책에서 직접 만들어 보는 부품들은 그 판단의 기준이 되어 줄 것입니다.

이 책은 주인공 두원이 공부한 것을 모아 둘 블로그 하나를 키워 가는 이야기로 진행됩니다. 두원은 블로그에 생기는 문제를 하나씩 만나며, 프레임워크가 해 주는 일을 먼저 손수 해 봅니다. 첫 코드를 직접 쓰고, 직접 틀리고, 로그와 테스트로 왜 틀렸는지 찾아냅니다. 도구가 많은 일을 대신해 줄수록, 무엇을 만들지 정하고 결과가 맞는지 확인하는 일은 더 분명하게 개발자의 몫으로 남습니다. 그 일을 할 수 있게 해 주는 것이 기본기라고 저는 믿습니다.

장 묶음마다 실습 저장소가 있고, 단계마다 시작 상태와 완성 상태를 담은 브랜치가 있습니다. 시작 브랜치에는 채워야 할 자리가 TODO로 비어 있고, 책에는 그 자리의 모습과 무엇을 채워야 하는지가 함께 실려 있습니다. 저장소를 열지 않고 책만 읽어도 흐름을 따라갈 수 있게 썼지만, 가능하면 직접 실행해 보시길 권합니다. 결과는 되도록 브라우저로 확인하게 했고, 명령은 macOS와 Linux용, Windows용 두 가지로 나눠 적었습니다. 막히면 완성 브랜치와 비교해 보면 됩니다. 틀린 코드가 왜 틀렸는지 찾아가는 과정도 이 책이 전하고 싶은 내용입니다. 일부러 틀리게 만든 장면에서는 서두르지 말고 로그부터 읽어 보세요.

자바 문법을 익히고 다음 걸음을 찾는 개발자를 생각하며 이 책을 썼습니다. 빨리 무언가를 만들어 보고 싶은 분께는 이 길이 멀게 느껴질 수 있습니다. 그래도 한 번은 지나가 보시면 좋겠습니다. 같은 화면을 보고도 그 아래에서 일어나는 일을 읽을 수 있게 될 것입니다.

2026년 10월 27일
마르코

# 마르코의 말

저는 대학교 4학년 때부터 회사에 다니며 개발을 시작했습니다. 지금 돌아보면 회사 생활이 무엇인지도 모른 채 밤낮없이 개발만 좋아하던 대학생이었습니다. 밤새 코드를 만지다 아침마다 지각하기 일쑤였고, 돌아가는 코드를 빨리 내놓는 데만 익숙했지 그 코드가 왜 돌아가는지는 잘 몰랐습니다. 그 뒤 인테리어 중개 서비스 스타트업에 뛰어들어서도 문제가 생기면 어디부터 봐야 할지 몰라 오래 헤맸습니다. 그 서툰 시간을 함께 버텨 준 스타트업 시절의 친구들에게 고맙습니다.

이후 10년 넘게 소프트웨어 개발자로 일했고, 이어서 5년쯤은 자바 백엔드 부트캠프에서 멘토로 학생들과 함께했습니다. 교실에서 저는 예전의 제 모습을 자주 만났습니다. 한글이 섞이자 잘려 나온 응답, 둘이 동시에 쓰면 섞이는 글, 며칠 잘 돌다가 멈춰 버린 서버. 바로 고쳐 주고 싶은 마음을 누르고 기다리는 일은 쉽지 않았지만, 스스로 찾은 답이 오래 남는다는 것을 그때 배웠습니다. 책 속의 마르코가 답 대신 질문을 건네는 것도 그래서입니다. 교육자로 보낸 그 5년을 정리하며 이 책을 썼습니다. 학생들이 자주 막히던 자리와, 그 자리를 함께 넘으며 찾은 설명을 블로그 하나를 만드는 순서대로 엮었습니다. 그동안 함께해 주신 분들께 감사의 말씀을 드립니다.

묵묵히 시간을 내어 준 가족이 없었다면 이 책은 끝나지 못했을 것입니다. 사랑하는 아내 아네스, 원고가 막혀 말수가 줄어든 날에도 곁에서 기다려 주고 끝까지 믿어 주어 고맙습니다. 책 속의 두원처럼 국수를 좋아하는 아들 빈첸시오, 고마워.

이 책을 집어 든 여러분, 두원과 함께 블로그 하나를 끝까지 만들어 보시길 바랍니다.

마르코

# 차례


**머리말** 3

**마르코의 말** 5

**이 책의 구성** 13

**이 책을 읽는 법** 17

**일러두기** 25

**1부 웹 애플리케이션은 어떻게 동작하는가** 29

**1장 빌드와 패키징: 소스 코드가 실행 가능한 파일이 되기까지** 31

1.1 개발 환경 준비 34

1.2 직접 컴파일하고 실행하기 40

1.3 라이브러리와 classpath 47

1.4 JAR로 묶기 56

1.5 Maven에 맡기기 62

1.6 Maven으로 묶기 74

1.7 또 하나의 빌드 도구, Gradle 84

**2장 로깅: 프로그램이 하는 일을 기록으로 남기기** 93

2.1 같은 번호의 글 두 개 96

2.2 System.out으로는 할 수 없는 것 98

2.3 API와 구현체: SLF4J와 Logback 100
2.4 로그 레벨을 고르고, 값을 넘기는 법 107
2.5 매번 쓰는 한 줄: Lombok의 @Slf4j 113
2.6 logback.xml: pattern, appender, logger 120
2.7 내 로그를 리뷰하기 129
2.8 스레드 두 개가 같은 파일에 쓸 때 130
2.9 자기 환경에 대입하기 137

**3장 "주소 좀 줘": 소켓 하나로 만드는 웹 서버 143**

3.1 연결 하나를 받는다 147
3.2 빈 줄에서 멈추는 이유 156
3.3 응답을 쓴다, 그리고 flush 158
3.4 Content-Length를 틀리면 163
3.5 Content-Type: 같은 바이트, 다른 해석 170
3.6 메서드, 경로, 상태 코드 173
3.7 로그 레벨을 바꿔 쓴다 178
3.8 완성 브랜치와 비교한다 179

**4장 "한참 돌다가 떴어": 느린 손님 하나와 일꾼들 193**

4.1 느린 손님 하나가 모두를 세운다 195
4.2 요청마다 스레드 하나 198
4.3 손님이 몰리면 203
4.4 미리 만든 스레드에 차례로 맡긴다 206
4.5 작업 큐 하나를 함께 쓴다 214
4.6 큐가 가득 차면 221

**5장 "글은 어디서 읽어?": 요청 객체와 서비스, context, 글쓰기 폼 231**

5.1 요청을 객체로, 응답도 객체로 233
5.2 abc 세 번이면 서버가 응답하지 않는다 242
5.3 경로가 늘어날수록 길어지는 if 244
5.4 메서드마다 나눠 받는 서비스: 템플릿 메서드 패턴 247
5.5 요청마다 새로 만들까, 싱글톤으로 둘까 257
5.6 객체를 맡아 두는 곳, context와 reflection 261

5.7 여러 스레드가 함께 쓰는 객체와 경쟁 상태 270
5.8 글쓰기 폼, 그리고 사라진 본문 273
5.9 저장한 뒤에는 303: PRG 패턴 280
5.10 요청 하나의 여행 286

**6장 Servlet 1: CGI에서 Servlet까지 293**

6.1 요청마다 프로그램 하나: CGI 295
6.2 임베디드 Tomcat: 프로그램 안에서 실행한다 299
6.3 Servlet의 기본 구조 305
6.4 Servlet container가 하는 일 310
6.5 글은 어디에 둘까: 메모리 저장소 318
6.6 로그인: 세션과 쿠키, 그리고 잘못 읽힌 한글 322
6.7 JSP: 화면은 HTML 쪽에서 328
6.8 Tomcat을 다시 실행하면 341

**7장 Servlet 2: 필터와 프런트 컨트롤러 349**

7.1 다섯 번 쓴 같은 코드 351
7.2 필터: Servlet보다 먼저 요청을 받는다 354
7.3 프런트 컨트롤러: 모든 요청을 먼저 받는 FrontServlet 358
7.4 사라진 글과 저장소의 약속 365
7.5 두 번째 구현, JsonPostRepository 372
7.6 같은 약속, 같은 테스트 381
7.7 아직 Servlet인 기능 Servlet 388

**8장 Servlet 3: 컨트롤러는 평범한 객체 397**

8.1 Servlet일 필요가 있을까 399
8.2 컨트롤러를 만들고, FrontServlet이 고른다 403
8.3 모델, 뷰, 컨트롤러 412
8.4 약속을 어긴 컨트롤러 415
8.5 글이 스스로 지키는 규칙 424
8.6 쓰는 쪽이 직접 고르는 두 줄 438
8.7 곁가지: 파일 업로드와 multipart 요청 439

**9장 제어를 넘기다: IoC, DI, AOP** **461**

9.1 직접 채운 맵을 치우자 463

9.2 Servlet container가 시작할 때 알려 주는 클래스 468

9.3 객체의 lifecycle은 누가 관리하나 474

9.4 컨트롤러마다 붙은 getInstance(): 생성자 주입 482

9.5 setter 주입 490

9.6 annotation과 필드 주입 492

9.7 어느 화면이 느린가 500

9.8 proxy 패턴 504

9.9 @StopWatch와 dynamic proxy 512

9.10 직접 만든 것과 붙인 이름 522

**10장 JDBC: 글을 데이터베이스에 저장하다** **529**

10.1 코드보다 테이블을 먼저 531

10.2 세 번째 구현, 직접 쓴 SQL 539

10.3 따옴표 하나로 일어나는 SQL 주입 548

10.4 2만 편의 목록과 페이징 554

10.5 반쯤 고쳐진 글과 transaction 560

**11장 connection pool과 transaction proxy: 몰려드는 요청을 버티다** **577**

11.1 connection pool: 연결을 미리 열어 두기 579

11.2 close()를 가로채는 proxy, 그리고 DBCP2 589

11.3 두 pool의 크기를 맞춘다 599

11.4 되풀이되는 JDBC 코드와 JdbcTemplate 608

11.5 transaction을 proxy로: 연결을 스레드에 묶기 612

11.6 앞뒤에서 transaction을 여닫는 proxy 620

**12장 Spring MVC로 옮기기: 직접 만든 것들은 어디로 갔나** **635**

12.1 입구는 여전히 하나다: DispatcherServlet 638

12.2 시작 과정을 따라가 보자: SCI에서 DispatcherServlet까지 644

12.3 스테레오타입 annotation과 컴포넌트 스캔 650

12.4 요청 처리 흐름: HandlerMapping에서 뷰까지 655

12.5 조용히 바뀐 숫자 하나 661
12.6 주입: 세 가지 방법과 생성자를 권하는 까닭 663
12.7 이름까지 같은 JdbcTemplate 667
12.8 @Transactional, 11장의 annotation 그대로 673
12.9 시간이 출력되지 않는다 679
12.10 WAR로 묶어 Tomcat에 배포하기 688
12.11 직접 만든 것들은 Spring에서 어디로 갔나 693

**부록 A 개발 환경 설치 705**

**부록 B 실습 저장소 안내 757**

**찾아보기 767**

# 이 책의 구성

이 책은 열두 장으로 이루어져 있다. 두원은 터미널에서만 실행되는 작은 블로그에서 시작해, 블로그가 부딪히는 문제를 장마다 하나씩 풀어 가며 웹 애플리케이션 하나를 완성한다. 앞 장에서 만든 것이 다음 장의 출발점이므로 차례대로 읽기를 권한다. 1장과 2장은 프로그램을 빌드해 실행하고 로그를 남기는 법을 다룬다. 3장부터 5장까지는 HTTP 서버를, 6장부터 8장까지는 Servlet 위의 블로그를 만든다. 9장은 객체를 관리하는 장치를, 10장과 11장은 데이터베이스를 다룬다. 12장은 앞에서 만든 것들을 하나로 묶어, 웹 애플리케이션이 시작되어 요청에 응답하기까지의 전체 그림을 그린다. 그림 0-1은 이 흐름을 보여 준다.




**그림 0-1** 이 책의 흐름. 괄호 안은 장 번호, 그 아래는 실습 저장소다. 마지막 12장에서는 앞에서 만든 것들을 Spring MVC에서 다시 찾아본다

**1장 빌드와 패키징: 소스 코드가 실행 가능한 파일이 되기까지.** JDK를 설치하고 환경 변수를 설정한 뒤, 글을 쓰고 읽고 고치고 지우는 터미널 블로그를 `javac`와 `java`만으로 컴파일하고 실행한다. 다른 사람이 만든 라이브러리(Jackson)로 글을 JSON 파일에 저장하려고 classpath를 지정하고, manifest를 작성해 `java -jar`로 실행할 수 있는 JAR로 묶은 뒤, JAR 안을 열어 무엇이 들어 있는지 읽어 본다. 그다음 같은 일을 Maven에 맡겨, 빌드 도구가 의존성 관리와 테스트, 패키징(JAR와 라이브러리 폴더, uber jar)을 어떻게 대신하는지 확인한다. Gradle은 Maven과 비교하며 간단히 살펴본다.

**2장 로깅: 프로그램이 하는 일을 기록으로 남기기.** 버그를 찾느라 곳곳에 넣은 `System.out.println`을 SLF4J와 Logback의 로그로 바꾼다. 로그 레벨을 고르는 기준, 예외를 원인까지 남기는 법, Lombok `@Slf4j`로 로거 선언을 줄이는 법, `logback.xml`로 로그의 형식, 내보낼 곳, 레벨을 코드 밖에서 정하는 법을 익힌다. 끝으로 두 스레드가 함께 남긴 로그에서 어느 스레드가 무엇을 했는지 읽어 낸다. 3장부터 만들 서버의 로그를 읽기 위한 연습이다.

**3장 "주소 좀 줘": 소켓 하나로 만드는 웹 서버.** 대학 친구 준서가 블로그 주소를 달라고 하자, 두원은 소켓 하나로 요청을 받는 단일 스레드 웹 서버를 만든다. 브라우저가 보내는 HTTP 메시지를 글자 하나하나까지 읽어 보고, 서버가 빈 줄에서 읽기를 멈추는 이유를 알아본다. `flush`, `Content-Length`, `Content-Type`, 메서드, 경로, 상태 코드가 각각 무슨 일을 하는지는 실험으로 확인한다.

**4장 "한참 돌다가 떴어": 느린 손님 하나와 일꾼들.** 아무 말도 하지 않는 연결 하나가 서버 전체를 멈춰 세우는 장면을 일부러 만든다. 요청마다 스레드를 만드는 서버, 미리 만들어 둔 일꾼에게 차례로 맡기는 서버를 거쳐, 일꾼들이 크기가 정해진 작업 큐 하나를 함께 쓰는 thread pool 서버로 간다. 스레드를 늘리는 데 드는 비용과, 스레드 수와 작업 큐에 상한을 두어야 하는 이유를 측정으로 확인한다.

**5장 "글은 어디서 읽어?": 요청 객체와 서비스, context, 글쓰기 폼.** 바이트를 읽는 일과 블로그의 일을 나눠, 요청과 응답을 객체로 다루고 페이지마다 서비스를 둔다. 이 과정에서 단일 책임, 개방-폐쇄, 리스코프 치환 원칙을 처음 배우고, 인터페이스의 명세를 사전·사후 조건으로 적어 모든 구현에 같은 질문을 던지는 계약 테스트를 작성한다. annotation과 reflection으로 서비스를 찾아 등록하는 애플리케이션 context를 만들고, 여러 스레드가 함께 쓰는 객체에서 생기는 문제를 찾는다. 마지막으로 POST로 글을 쓰고 303으로 redirect하는 폼을 만들어, 요청 하나가 서버 안에서 처리되는 흐름을 처음부터 끝까지 따라간다.

**6장 Servlet 1: CGI에서 Servlet까지.** 서버 일을 Tomcat에 맡긴다. 요청마다 프로그램을 새로 실행하던 CGI를 보고 Servlet이 왜 나왔는지 알아본 뒤, Servlet container가 맡는 일을 3~5장에서 직접 만든 부품과 짝지어 본다. 기능마다 Servlet을 하나씩 두고 글은 메모리에 저장하는 블로그를 만든다. 그 과정에서 세션과 쿠키로 하는 로그인, JSP로 그리는 화면을 익히고, Servlet 객체 하나를 여러 스레드가 함께 쓴다는 것이 무슨 뜻인지 확인한다.

**7장 Servlet 2: 필터와 프런트 컨트롤러.** Servlet마다 반복되던 인코딩과 로그인 확인을 필터로 옮기고, `.do`로 들어오는 요청을 먼저 받는 FrontServlet을 만든다. Servlet이 글 저장소에

바라는 일을 `PostRepository`라는 인터페이스, 곧 약속으로 적는다. 그리고 5장에서 처음 쓴 계약 테스트를 저장소에도 써서, 메모리와 JSON 파일 두 구현이 같은 약속을 지키는지 한 번에 확인한다. 장 끝에서는 필터 체인에 든 책임 연쇄 패턴과, 약속은 그것을 쓰는 쪽이 정한다는 의존성 역전 원칙에 이름을 붙인다.

**8장 Servlet 3: 컨트롤러는 평범한 객체.** FrontServlet이 생긴 뒤로 브라우저가 직접 요청하지 않게 된 기능 Servlet을 평범한 자바 객체인 컨트롤러로 바꾸고, 모델, 뷰, 컨트롤러가 각각 맡는 일을 블로그 코드에서 짚는다. 할 일 하나를 `execute` 메서드 하나짜리 객체로 감싼 이 모양에는 커맨드 패턴이라는 이름이 있다. 타입만 맞춘 컨트롤러가 약속을 어기는 장면에서 리스코프 치환 원칙을 다시 보고, 글 객체가 만들어지는 순간 스스로 규칙을 확인하게 만든다. 신고받은 버그를 실패하는 테스트로 먼저 재현하고 고치는 과정에서 회귀 테스트와 단위·통합 테스트를 정리한다. 마지막으로 글에 파일을 붙이며 multipart 요청이 바이트로 어떻게 생겼는지 본다.

**9장 제어를 넘기다: IoC, DI, AOP.** 컨트롤러를 직접 만들어 맵에 넣던 일을 다른 객체에 맡긴다. 서버가 시작될 때 Servlet container가 클래스 목록을 알려 주는 장치(`ServletContainerInitializer`)로 컨트롤러를 찾아 만들고 맡아 둔다. 그리고 생성자, setter, annotation으로 의존성을 넣는 세 방법을 비교하고, 생성자로 넣은 덕분에 Tomcat 없이 컨트롤러를 시험하며 진짜 객체를 대신하는 테스트 대역(Mockito의 mock, spy)을 써 본다. 느린 화면을 찾으려고 시간을 재는 proxy를 직접 만들고, `@StopWatch`가 붙은 객체만 dynamic proxy로 감싼다. 장 끝에서 직접 만든 것들에 제어의 역전, 의존성 주입, AOP라는 이름을 붙인다.

**10장 JDBC: 글을 데이터베이스에 저장하다.** 글을 MySQL에 두는 세 번째 저장소를 만든다. 테이블부터 설계하고 구현을 붙이자 약속에 섞여 있던 파일 이름이 드러나는데, 그 이름을 걷어 내며 인터페이스 분리 원칙을 배운다. 따옴표 하나로 일어나는 SQL 주입을 `PreparedStatement`로 막고, 글 2만 편의 목록을 페이지 단위로 나눠 읽는다. 글을 고칠 때 이전 버전을 남기는 SQL 두 문장을 transaction 하나로 묶고, 격리 수준에 따라 다른 연결이 고친 내용이 언제 보이는지 확인한다.

**11장 connection pool과 transaction proxy: 몰려드는 요청을 버티다.** 사람이 한꺼번에 몰리자 블로그가 열리지 않았다는 준서의 메시지로 시작한다. 요청마다 새로 열던 데이터베이스 연결을 미리 열어 두는 connection pool을 직접 만들어 본 뒤 DBCP2로 바꾸고, 그 크기를

Tomcat thread pool에 맞춰 측정하며 다듬는다. 되풀이되는 코드를 걷어 내는 `JdbcTemplate`과 transaction proxy를 직접 만들며 장을 마친다.

**12장 Spring MVC로 옮기기: 직접 만든 것들은 어디로 갔나.** 책을 마무리하는 장이다. 두원은 블로그를 Spring MVC로 직접 옮기며, 앞에서 만든 조각마다 Spring의 어느 부분이 그 일을 대신하는지 하나씩 짝지어 본다. Tomcat이 시작될 때부터 요청 하나가 응답이 될 때까지의 흐름을 따라가고, 옮기는 사이 조용히 바뀐 상태 코드 하나와 proxy가 끼어들지 못하는 경우를 찾아낸다. 다 옮긴 블로그는 WAR로 묶어 따로 내려받은 Tomcat에도 올려 본다. 장 끝에서는 FrontServlet, 직접 만든 context, `JdbcTemplate`, transaction proxy가 Spring에서 어떤 이름을 달고 자동으로 돌아가는지 정리표로 모은다. 그러면 앞에서 만든 조각들이 웹 애플리케이션 하나가 구동되는 흐름 안에 그대로 들어 있음을 알게 된다.

책 끝의 부록 A에는 실습 도구의 설치 방법(JDK와 Maven은 1장에서 설치한다)과 자바, Tomcat, Servlet, JSP, Spring의 버전 대응 표를, 부록 B에는 장마다 쓰는 실습 저장소와 단계별 브랜치를 모았다.

# 이 책을 읽는 법

## 누가 읽으면 좋은가

자바로 클래스와 인터페이스를 만들고, 컬렉션(`List`, `Map`)과 예외를 다룰 줄 알지만 아직 숙련되지 않은 개발자를 위해 썼다. 예제를 따라 하면 프로그램은 실행되는데 그 안에서 무슨 일이 일어나는지 설명하기 어려운 사람에게 특히 권한다. 다른 사람이 만든 코드가 왜 동작하는지 모르거나, 제대로 만든 것인지 판단하기 어려운 사람에게도 도움이 된다.

개념은 처음 나오는 자리에서, 그 장면 안에서 설명한다. 앞 장에서 설명한 것은 한 줄로 다시 떠올리게 한다. 이미 Servlet과 JDBC를 써 본 독자라면 실습 단계는 훑고 지나가도 좋다. 다만 그럴듯하게 틀린 코드를 잡아내는 장면과 12장의 정리표(12.11절)는 꼭 읽어 보자.

## 미리 알아 두면 좋은 자바 문법

예제 코드는 자바 21로 썼고, 앞에 적은 기본 문법을 넘어서는 문법도 쓴다. 본문이 따로 설명하지 않거나 한두 마디로만 짚는 문법을 표로 모았다. 처음 나오는 절에 이르기 전에 공식 자료로 한 번 읽어 두면 코드가 막히지 않는다. 자료 이름은 자바 공식 학습 사이트 Dev.java(https://dev.java/learn/)의 튜토리얼 제목이고, 그곳에 따로 없는 두 가지는 자바 언어 명세(Java SE 21)의 절이다.

| 문법 | 처음 나오는 절 | 미리 읽을 공식 자료 |
|---|---|---|
| 람다(`() -> …`) | 2.1 | “Writing Your First Lambda Expression” |

| 문법 | 처음 나오는 절 | 미리 읽을 공식 자료 |
|---|---|---|
| 화살표 `switch`(`case "write" -> …`) | 2.1 | "Branching with Switch Expressions" |
| `catch` 하나에 예외 타입 여럿 쓰기 | 2.4 | "Catching and Handling Exceptions" |
| 메서드 참조(`all::addAll`) | 2.8 | "Writing Lambda Expressions as Method References" |
| `Optional` | 3.4 | "Using Optionals" |
| 정규식(`split("\\?")`, `matches("\\d+")`) | 3.6 | "Regular Expressions" |
| 스트림(`stream().filter(…)`) | 5.1 | "The Stream API" |
| 인터페이스의 `default` 메서드 | 5.4 | "Interfaces" |
| 와일드카드 제네릭(`Class<?>`) | 5.6 | "Wildcards" |
| 가변 인자(`Class<?>... classes`) | 5.6 | 자바 언어 명세 8.4.1 Formal Parameters |
| `instanceof`의 패턴 변수 | 9.3 | "Using Pattern Matching" |
| 제네릭 메서드(`<T> T findByType(…)`) | 9.6 | 자바 언어 명세 8.4.4 Generic Methods |

자바 21에서 컬렉션에 생긴 `addFirst`(2.1절), `reversed`(9.3절)는 JDK 21 API 문서의 `SequencedCollection`에 있다. 본문은 7.4절에서 `Optional`을, 11.4절에서 람다의 이름과 쓰임, 그리고 가변 인자가 메서드 안에서 배열이 된다는 것을 다시 짚는다.

다음은 몰라도 된다. 처음 나오는 자리에서 이 책이 풀어 준다.

- try-with-resources(3.1절), 익명 클래스와 제네릭의 타입 지우기(1.3절), record(1.2절, 2.5절)
- 스레드(2.8절, 4장)와 `synchronized`, `wait`(4장)
- annotation 선언과 reflection(5.6절)
- HTML 기초(3장), JSP와 EL, JSTL(6.7절)
- SQL 기초(10장)

## 장의 차례

모든 장은 같은 차례로 구성되어 있다.

1. 장 첫머리의 **이 장에서 배울 것**. 배울 것과 이 장의 실습(저장소, 브랜치, 만들 것)을 적었다.
2. 두원과 마르코가 나오는 여는 장면. 이 장의 문제가 어디서 왔는지 보여 준다.
3. 번호가 붙은 절. 실습은 **직접 해 보기**로 진행한다. 직접 해 보기는 브랜치, 목표, 왜 필요한가, 단계 순서로 적고, 끝의 **생각해 보기**에 남긴 질문의 답은 뒤의 본문이나 연습 문제에 있다. 절 사이에는 **개발 상식**, 참고, 주의 같은 덧붙임 글이 들어간다(이어지는 "덧붙임 글과 번호").
4. 장 끝의 **이 장에서 배운 것**(표), **연습 문제**, **마르코의 질문**. 연습 문제 번호 뒤의 [기본]은 장에서 한 일을 조금 바꿔 되풀이하는 문제, [응용]은 본문을 넘어 여러 내용을 엮거나 추론해야 하는 문제, [도전]은 답이 열려 있거나 장 밖까지 살펴보는 긴 문제다.
5. 다음 장으로 넘어가는 짧은 장면.

3장부터 각 장 첫머리에는 작은 지도가 있다. 12장 끝의 그림 12-5를 줄인 것으로, 요청 하나가 지나는 칸을 차례로 늘어놓았다. 노란 칸은 그 장에서 만들거나 바꾸는 곳이고, 민트 칸은 앞 장에서 만든 그대로 쓰는 곳이다. "Tomcat"이나 "Spring"이라고 적힌 흰 칸은 그 일을 남이 맡는 곳이고, 회색 점선 칸은 아직 만들지 않은 곳이다.

## 실습 저장소와 브랜치

실습 저장소는 장 묶음마다 하나씩 두었다. 이어지는 표의 저장소는 모두 GitHub의 backend-blog 조직(https://github.com/backend-blog)에 있다. 각 저장소의 주소와 단계별 브랜치 목록은 부록 B에 모았다.

| 저장소 | 장 | 단계 |
|---|---|---|
| `build-and-packaging` | 1장 | `step1` ~ `step4` |
| `logging` | 2장 | `step1` ~ `step4` |
| `simple-http-server` | 3~5장 | `step1` ~ `step8` |

| 저장소 | 장 | 단계 |
|---|---|---|
| `servlet-blog` | 6~11장 | `step1` ~ `step14`, 곁가지 `step5-file`(8.7절) |
| `spring-blog` | 12장 | `step1` ~ `step3` |

단계마다 브랜치가 두 개 있다. 이름이 `-exercise`로 끝나는 실습 브랜치에는 채울 곳에 `TODO`가 적힌 코드가 있고, `-solution`으로 끝나는 완성 브랜치에는 그 단계를 마친 코드가 있다. 실습 브랜치에서 직접 해 보고, 다 하면 완성 브랜치와 비교한다. 직접 해 보기의 번호는 장 안의 차례가 아니라 실습 저장소의 단계(브랜치) 번호를 따르므로, 10장의 직접 해 보기 12는 `servlet-blog`의 `step12-exercise`에서 시작한다. 한 단계의 완성 브랜치는 다음 단계 실습 브랜치의 출발점이라, 단계를 차례로 밟으면 장 하나를 끝까지 따라가게 된다. 이어지는 명령은 저장소를 내려받은 폴더 안에서 실행한다(내려받는 법은 1.1절과 부록 B.2절).

터미널

```
$ git switch step1-exercise          # 단계 시작
$ git diff origin/step1-solution     # 다 하면 완성 브랜치와 비교
```

완성 브랜치와 내 코드가 달라도 괜찮다. 다른 곳마다 왜 다른지, 어느 쪽이 무엇을 더 지키는지 따져 보는 것이 비교의 목적이다.

## 덧붙임 글과 번호

참고, 팁, 주의, 개발 상식과 직접 해 보기는 본문과 구별되는 상자에 담기고, 상자 제목 앞의 이름이 종류를 알린다. 참고에는 펼친 책, 팁에는 전구 아이콘을 붙였다.

| 이름 | 무엇을 담나 |
|---|---|
| 이 장에서 배울 것 | 그 장의 학습 목표와 실습(차례는 앞의 "장의 차례") |
| 직접 해 보기 | 실습 한 단계(차례는 앞의 "장의 차례") |
| 참고 | 처음 나온 용어나 도구의 짧은 설명 |
| 팁 | 알아 두면 편한 것 |
| 주의 | 실수하기 쉬운 것 |

| 이름 | 무엇을 담나 |
|---|---|
| 개발 상식 | 그 장의 주제와 이어지는 깊은 설명 |
| 마르코의 질문 | 그 장의 주제로 마르코가 던지는 질문 하나 |

**마르코의 질문**에서는 코드 리뷰, 운영 중에 들어온 신고, 서버 증설처럼 개발하며 실제로 마주칠 상황 하나를 마르코가 질문으로 던진다. 답할 때 짚어야 할 것과 빠지기 쉬운 답도 함께 적었다.

직접 쓰거나 열어 볼 코드에는 **예제 3-1**처럼 장 번호와 차례를 붙이고, 번호 옆에 파일 이름을 적었다. 그림도 **그림 3-1**처럼 장 안에서 차례대로 번호를 매긴다. 터미널 명령, 출력, 로그에는 번호를 붙이지 않는다. 종이책과 전자책이 같은 원고에서 나오므로 본문에서는 쪽의 배치에 기대는 말 대신 예제, 그림, 절의 번호로 가리킨다.

## 운영체제별 명령

운영체제마다 다른 명령은 macOS와 Linux의 터미널 명령과 Windows의 PowerShell 명령으로 나눠 적었다. Windows 명령은 Windows에 기본으로 들어 있는 PowerShell을 기준으로 한다. 두 운영체제에서 똑같은 명령(`javac`, `java`, `jar`, `git`, `mvn`)은 한 번만 적는다. macOS와 Linux가 서로 다른 곳(JDK 폴더의 모양 등)은 그 자리에 따로 적었다. 이야기 속 장면은 두원이 쓰는 macOS를 기준으로 한다.

종이책 본문에는 macOS/Linux 명령만 실었다. Windows 명령은 부록 A.13절에 장과 절 순서로 모았으니, Windows를 쓴다면 그 절을 펴 두고 함께 따라 하면 된다. 부록 A의 설치 안내에는 두 운영체제의 명령을 나란히 실었다. 전자책에는 본문 자리마다 두 운영체제의 명령이 모두 있다.

Windows PowerShell에서 `curl`은 다른 명령(`Invoke-WebRequest`)의 별칭이므로 `curl.exe`로 입력한다. 자세한 것은 부록 A.6절에 있다.

## 등장인물

| 이름 | 설정 |
|---|---|
| 두원 | 몇 달째 혼자 자바를 공부하고 있다. 공부한 것을 모아 둘 블로그를 직접 만들고 싶어 한다 |
| 마르코 | 두원의 오랜 친구로, 개발을 몇 해 먼저 시작했다. 답을 바로 주기보다 질문을 던지고, 두원이 먼저 해 보게 둔다 |
| 준서 | 두원의 대학 친구. 블로그의 첫 독자이자 첫 필자로, 메시지 한 줄로 다음 문제를 가져온다 |

두원과 마르코는 반말을 쓰는 친구 사이다. 두원은 첫 코드를 직접 쓰고, 그 코드가 그럴듯하게 틀리는 장면을 직접 겪는다. 두원이 처음 쓴 코드의 실수는 경험이 적은 개발자가 흔히 저지르는 것으로 골랐다. 눈여겨볼 것은 고친 코드보다, 처음 코드가 틀렸다는 것을 알아챈 단서다.

## 실습 환경

| 도구 | 쓰는 곳 | 비고 |
|---|---|---|
| JDK 21 (Eclipse Temurin) | 모든 장 | LTS. 내려받고 설정하는 순서는 1.1절 |
| Maven | 1.5절부터 모든 장 | 1장에서 직접 설치한다 |
| Git | 모든 장 | 실습 저장소를 내려받고 브랜치를 전환한다 |
| curl | 1.3절, 3장부터 | 1.3절에서는 라이브러리를 내려받는 스크립트가 쓴다. Windows는 `curl.exe` |
| `nc` | 3장, 4장 | Windows는 3장에서 Nmap에 든 `ncat`을, 4장에서 PowerShell 명령을 쓴다 |
| 편집기 | 모든 장 | IntelliJ IDEA나 VS Code |

| 도구 | 쓰는 곳 | 비고 |
|---|---|---|
| Tomcat 11.0.27 | 6장부터 | Servlet 6.1, JSP 4.0, EL 6.0(Jakarta EE 11)을 구현한다. 프로그램 안에서 실행하고, 12장에서는 따로 내려받은 배포판에도 올려 본다 |
| Spring Framework 7.0.9 | 12장 | Jakarta EE 11(Servlet 6.1)을 기준으로 하므로 Tomcat 11과 짝이 맞는다 |
| Docker | 10~12장 | MySQL 8.4를 실행한다 |

1장의 1~3단계는 Maven 없이 JDK만으로 진행한다. 4단계부터는 Maven이 실습 저장소의 설정대로 라이브러리를 내려받고, 6장부터는 Tomcat도 내려받는다. 도구의 설치는 부록 A에 모았다. 다만 JDK와 Maven은 1장(1.1절, 1.5절)에서 내려받아 환경 변수를 설정하고, 부록 A에는 버전 갈아타기와 패키지 설치 같은 나머지 내용만 있다. 자바, Tomcat, Servlet, JSP, Spring의 버전이 서로 어떻게 맞물리는지와 라이브러리 버전 전체는 부록 A.10절의 표에 모았다(Maven 플러그인의 버전은 실습 저장소의 `pom.xml`에서 본다).

# 일러두기

## 원어 표기

제품, 라이브러리, 프레임워크, 운영체제의 이름은 한글로 읽어 적지 않고 원어로 쓴다. Maven, Gradle, Tomcat, Spring, Lombok, GitHub, Docker, Linux, Windows, macOS가 그렇다. 기술과 형식의 이름도 업계에서 흔히 쓰는 대소문자를 따른다(Servlet, JSP, JDBC, JSON, JAR, WAR, Javadoc). 조사는 원어를 우리말로 읽은 소리에 맞춘다(Maven을, macOS와).

흔히 한글로 옮겨 적는 기술 용어 가운데 일부도 원어로 쓴다. Servlet container, classpath, manifest, uber jar, reflection, annotation, context, proxy, dynamic proxy, transaction, thread pool, connection pool(줄여 pool), bean, advice, pointcut, aspect, handler, interceptor, listener, wrapper, scope, redirect, escape, lifecycle, fat jar, goal, appender, flush, keep-alive, forward, multipart, setter, getter, join point, coordinates(Maven의 groupId·artifactId·version 묶음)가 그렇다. 이런 용어와, round robin처럼 읽는 법을 짐작하기 어려운 몇몇 용어는 본문에 처음 나오는 곳에 reflection(리플렉션)처럼 읽는 소리를 함께 적었다. 우리말로 굳은 말은 한글로 둔다. 클래스, 메서드, 스레드, 서버, 테스트, 라이브러리, 프레임워크, 인터페이스, 의존성, 세션, 쿠키, 헤더, 소켓, 컨트롤러, 필터가 그렇다. 데이터베이스와 맺은 연결(JDBC의 `Connection` 객체)도 서술에서는 "연결"로 쓴다. 여러 스레드가 함께 쓰며 처리할 일을 담아 두는 큐(4장의 `RequestChannel`)는 "작업 큐"라고 쓴다. 우리말 "줄"로 옮기면 텍스트의 한 줄과 헷갈리기 때문이다. 외래어는 외래어 표기법을 따른다(메서드, 리팩터링).

## 글자 모양

| 모양 | 쓰는 곳 | 예 |
|---|---|---|
| 코드 글꼴 | 클래스, 메서드, 명령, 파일과 폴더 이름, 설정 값. 번역하지 않는다. 다만 FrontServlet과 Factory(`ControllerFactory`)는 역할 이름이라 본문에서 보통 글꼴로 적는다 | `PostRepository`, `javac`, `pom.xml` |
| 굵은 글씨 | 처음 정의하는 낱말, 예제 번호 | **예제 3-1** |
| 고딕 글씨 | 제목과 소제목, 표, 그림과 예제의 캡션, 대화에서 말하는 사람의 이름 | |

코드와 출력은 바탕 없이 위아래 가는 실선으로 시작과 끝을 알린다. 터미널에 입력하는 명령과 그 출력은 터미널 창 모양 하나에 담는다. 종이책에서는 명령 줄 앞에 프롬프트(`$`, Windows는 `PS>`)를 붙이고, 출력은 한 단계 옅은 글자로 적었다. 프롬프트는 입력하지 않는다. 한 줄은 80칸을 넘지 않게 나누었다. 이야기와 상관없는 부분은 `…`나 `// … (생략: 무엇을 생략했는지)`로 줄였다. 줄인 곳을 빼면 코드는 실습 저장소의 것과 같고, 출력은 실제로 실행한 결과를 옮겼다. 시각, 해시값, 걸린 시간처럼 실행할 때마다 달라지는 값은 독자의 화면과 다를 수 있다.

## 문장

본문은 "~다" 체로 쓰고, 두원과 마르코의 대화는 친구 사이의 반말로 쓴다. 각주는 쓰지 않는다. 보충 설명은 본문이나 참고, 개발 상식 같은 덧붙임 글에 둔다. 다른 곳을 가리킬 때는 쪽 번호 대신 장과 절의 번호(5.3절), 예제와 그림의 번호(예제 3-1, 그림 3-1)를 쓴다. 다만 부록 A.13절은 종이책에서 블록마다 본문의 쪽을 함께 적는다. 웹 주소는 꼭 필요한 공식 문서만 적고, 실습 저장소의 주소는 부록 B에 모았다.

## 그림의 색과 테두리

그림의 선은 먹색 한 가지로 그렸다. 강조에는 형광펜 세 색만 쓰고, 그림마다 한두 곳에만 칠했다. 흑백으로 인쇄해도 뜻이 남도록 색마다 테두리 모양을 함께 정했다.

| 색 | 뜻 | 테두리 |
|---|---|---|
| 노랑 | 지금 보고 있는 곳 | 보통 굵기의 실선 |
| 민트 | 우리가 직접 쓴 코드 | 굵은 실선 |
| 분홍 | 문제나 실패 | 점선 |

그림 안에는 제목을 넣지 않고, 번호와 설명은 그림에 붙인 캡션에 적었다. 다른 사이트나 프로그램의 화면을 캡처한 그림은 캡션 끝에 출처와 캡처 시기를 적었다. 이 책의 실습 블로그를 캡처한 그림에는 출처를 따로 적지 않았다.

## 버전

버전은 이름 뒤에 숫자로 적는다(Tomcat 11.0.27, MySQL 8.4). 자바 언어의 버전은 "자바 21"로, 설치하는 개발 도구는 "JDK 21"로 적는다. 도구의 버전, 설치 화면, 명령의 출력은 글을 쓰는 시점(2026년 10월)을 기준으로 한다. 책과 다른 버전을 쓰면 출력의 숫자나 문구가 조금 다를 수 있다.

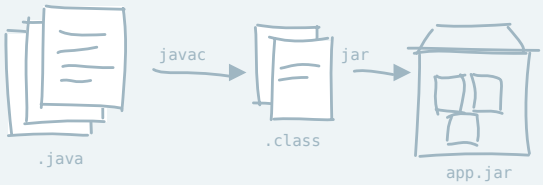
# 1부

# 웹 애플리케이션은 어떻게 동작하는가

자바 파일 하나에서 시작해 빌드, 로그, HTTP 서버, Servlet, 객체 관리, 데이터베이스를 차례로 직접 다루고, 마지막에 그 모두가 함께 움직이는 그림 하나로 묶는다.

1장 빌드와 패키징 · 2장 로깅 · 3~5장 HTTP 서버 · 6~8장 Servlet · 9장 제어의 역전 · 10장 JDBC · 11장 connection pool과 transaction proxy · 12장 Spring MVC로 옮기기

1
장




# 빌드와 패키징

소스 코드가 실행 가능한 파일이 되기까지

1.1 개발 환경 준비
1.2 직접 컴파일하고 실행하기
1.3 라이브러리와 classpath
1.4 JAR로 묶기
1.5 Maven에 맡기기
1.6 Maven으로 묶기
1.7 또 하나의 빌드 도구, Gradle

### 이 장에서 배울 것

- JDK 명령(`javac`, `java`)만으로 글을 쓰고 읽고 고치고 지우는(CRUD) 터미널 블로그를 컴파일하고 실행하며, 소스 파일이 실행되기까지 어떤 과정을 거치는지 설명할 수 있다.
- 메모리에 담던 글을 JSON 파일에 저장하도록 바꾸며, 프로그램이 끝나도 데이터가 남으려면 무엇이 필요한지 설명할 수 있다.
- 이 블로그를 빌드 도구 없이 실행 가능한 JAR로 직접 묶으며, classpath(클래스패스)와 manifest(매니페스트)가 각각 왜 필요한지 설명할 수 있다.
- 직접 빌드한 블로그를 Maven으로 다시 빌드하며, 의존성 관리와 테스트, 패키징을 빌드 도구가 어떻게 대신하는지 설명할 수 있다. 패키징은 JAR와 라이브러리 폴더, 그리고 의존성까지 담은 uber jar(우버 자)로 해 본다.

### 이 장의 실습

- **저장소:** `build-and-packaging`
- **브랜치:** `step1-exercise`에서 시작해 `step4-solution`까지. 단계마다 `stepN-exercise`에서 시작, `stepN-solution`과 비교
- **만들 것:** 글을 메모리에 담았다가 JSON 파일에 저장하도록 바꾸는 터미널 블로그, 직접 묶은 JAR, 같은 블로그의 Maven 프로젝트(JAR와 라이브러리 폴더, uber jar)

금요일 저녁, 두원은 마르코와 국숫집에 마주 앉았다. 마르코는 오랜 친구이고, 개발을 두원보다 몇 해 먼저 시작했다. 두원은 몇 달째 혼자 자바를 공부하고 있었다. 공부한 내용은 노트와 메모 앱 여기저기에 흩어져 있었고, 며칠 전에는 정리해 둔 줄 알았던 내용을 찾지 못해 같은 예제를 처음부터 다시 풀었다. 국수가 나오기도 전에 두원이 노트북을 열었다.

> **두원** 이것 좀 봐. 공부한 걸 모아 두려고 블로그를 만들었어. AI한테 "블로그 만들어 줘"라고 한 줄 썼더니 10분 만에 이게 나오더라.

화면에는 글 목록이 나와 있었다. 글을 쓰면 목록에 올라오고, 제목을 누르면 본문이 보였다.

> **마르코** 오, 잘 돌아가네. 코드도 깔끔하고. 프레임워크 위에 AI가 코드를 써 준 거구나.

**두원** 프레임워크가 뭘 해 준 건데? 난 AI가 다 한 줄 알았어.

**마르코** 요청을 받고, 데이터를 저장하고, 화면에 돌려주는 일 같은 건 대부분 프레임워크가 해. 프레임워크는 프로그램의 뼈대와 흐름을 미리 짜 둔 코드야. AI는 그 프레임워크를 쓰는 코드를 흔한 방식대로 빠르게 만들어 준 거고. 그럼 하나 물어볼게. 이 블로그는 목록을 어떤 순서로 보여 줘?

**두원** 최신 글이 위에 오겠지?

**마르코** 그렇게 정했어?

**두원** 아니, 정한 적은 없는데.

**마르코** 그럼 글은 어디에 저장돼? 서버를 다시 실행해도 남아 있어?

두원은 대답하려다 입을 닫았다. 자바 문법은 안다. `main` 메서드도, 클래스도, 리스트도 써 봤다. 그런데 자기 블로그가 어떻게 동작하는지 설명하려니 말문이 막혔다.

**마르코** AI를 쓰지 말라는 게 아니야. 나도 매일 써. 다만 AI한테 시키는 것도 결국 네가 알아야 시킬 수 있어. 뭘 정해 달라고 해야 하는지, 나온 코드에서 뭘 확인해야 하는지, 문제가 생기면 어디를 봐야 하는지. 프레임워크도 마찬가지야. 안에서 무슨 일을 대신해 주는지 모르면, 고장 났을 때 손을 못 대.

**두원** 그럼 뭐부터 알아야 하는데?

**마르코** 이번 주말에 프레임워크도 AI도 빌드 도구도 없이 한번 직접 만들어 볼래? 아주 작게, 자바 파일 하나로. 하나씩 해 보면, 나중에 프레임워크가 무슨 일을 해 주는지, AI가 써 준 코드가 어떤 코드인지 보이기 시작할 거야.

**두원** 자바 파일 하나로 블로그가 된다고?

**마르코** 처음엔 화면도 없어. 터미널에서 글 쓰고 목록 보는 게 전부야. 시시해 보여도 거기서 시작하는 게 제일 빨라.

국수가 나왔다. 두원은 노트북을 덮고 젓가락을 들며 고개를 끄덕였다.

## 1.1 개발 환경 준비

토요일 아침, 두원은 코드를 열기 전에 개발 환경부터 갖추기로 했다. 자바 코드를 컴파일하려면 먼저 JDK(Java Development Kit)가 있어야 한다. 컴파일러인 `javac`가 JDK에 들어 있기 때문이다. JDK는 여러 곳에서 배포하는데, 이 책은 Eclipse Adoptium이 내놓는 Eclipse Temurin을 쓴다.

**참고 JDK 배포판과 Temurin**

자바의 소스 코드는 OpenJDK라는 이름으로 공개되어 있다. 이 소스를 빌드해 설치 파일로 내놓는 곳이 여럿이라 JDK도 여러 종류가 있다. Oracle JDK, Amazon Corretto, Microsoft Build of OpenJDK가 그런 예다. 소스가 같으므로 어느 것을 써도 자바 코드는 똑같이 실행된다. 차이는 사용 조건과 보안 업데이트를 제공하는 기간에 있다. Temurin은 Eclipse 재단의 Adoptium 프로젝트가 만드는 배포판이다. 회사에서든 개인 프로젝트에서든 비용 없이 쓸 수 있고, 자바 표준과 호환되는지 확인하는 시험(TCK)을 통과한 빌드만 내놓는다. 그래서 특정 회사에 종속되지 않는 JDK를 원할 때 많이 고른다. 이름에는 작은 말장난이 숨어 있다. Temurin(테무린)의 철자 순서를 바꾸면 runtime이 된다. 자바 프로그램을 실행하는 환경, 곧 runtime을 배포한다는 뜻을 그렇게 이름에 담았다.

두원은 설치 프로그램 대신 압축 파일을 내려받아 직접 풀고, 환경 변수를 설정하기로 했다. 그래야 어느 `java`가 어디에 설치되어 있는지 스스로 알 수 있다.

### JDK 설치와 환경 변수 설정

환경 변수는 운영체제가 프로그램에 알려 주는 설정 값이다. JDK를 쓰려면 두 가지가 필요하다.

- `PATH`는 명령을 찾는 폴더의 목록이다. `java`라고 입력하면 터미널은 `PATH`에 적힌 폴더를 앞에서부터 차례로 찾아보고, 처음 찾은 `java`를 실행한다. 같은 이름의 명령이 두 곳에 있으면 앞에 적힌 쪽이 실행된다.
- `JAVA_HOME`은 JDK가 설치된 폴더다. Maven 같은 빌드 도구와 IDE(IntelliJ IDEA 같은 통합 개발 환경)가 이 값을 읽어 어느 JDK를 쓸지 정한다.

`JAVA_HOME`에 JDK 폴더를 적고, `PATH` 맨 앞에 `JAVA_HOME` 아래의 `bin` 폴더를 넣는다. 그러면 터미널에서 실행하는 `java`와 빌드 도구가 쓰는 `java`가 같은 JDK를 가리킨다.

**버전 고르기.** 이 책은 LTS(장기 지원 버전)인 자바 21을 쓴다. 자바 25가 더 새 LTS지만, 오래 다듬어져 안정되었고 널리 쓰이는 버전이라 21을 골랐다.

### 참고 LTS

자바는 6개월마다 새 버전을 내놓고, 그중 하나를 LTS(Long-Term Support, 장기 지원 버전)로 정한다. LTS가 아닌 버전은 다음 버전이 나오면 곧 보안 업데이트가 중단되지만, LTS는 여러 해 동안 보안 업데이트를 받는다. 프레임워크와 라이브러리도 LTS를 기준으로 지원 범위를 정하므로, 실제로 운영되는 서비스는 대개 LTS 버전을 쓴다. 2021년부터 LTS는 2년마다 나오고, 지금까지의 LTS는 8, 11, 17, 21, 25다.

**내려받기.** Eclipse Temurin 내려받기 페이지에서 버전 21, 종류 JDK를 고르고, 운영체제와 CPU에 맞는 압축 파일을 내려받는다(그림 1-1). CPU 종류를 확인하는 법은 그림 뒤에 있다.

```
https://adoptium.net/temurin/releases/
```




**그림 1-1** Eclipse Temurin 내려받기 페이지의 JDK 21 macOS 줄

1. 버전
2. CPU(x64 또는 aarch64)
3. 종류(JDK)
4. 내려받을 압축 파일(TAR.GZ)

CPU 종류는 macOS라면 터미널에서 다음 명령으로 확인한다.

터미널 · macOS/Linux

```
$ uname -m
```

결과가 `arm64`이면 aarch64 파일(Apple 실리콘)을, `x86_64`이면 x64 파일(Intel)을 고른다. Linux도 같은 명령으로 확인하는데, 결과가 `arm64` 대신 `aarch64`로 나온다. Windows는 설정 > 시스템 > 정보의 "시스템 종류"에서 확인한다. 글을 쓰는 시점의 파일 이름은 다음과 같다. 숫자는 업데이트마다 바뀐다.

```
macOS(Apple 실리콘) OpenJDK21U-jdk_aarch64_mac_hotspot_21.0.12.1_1.tar.gz
macOS(Intel)       OpenJDK21U-jdk_x64_mac_hotspot_21.0.12.1_1.tar.gz
Windows(x64)       OpenJDK21U-jdk_x64_windows_hotspot_21.0.12.1_1.zip
```

**macOS.** 순서는 네 단계다.

1. 개발 도구를 모아 둘 `~/dev` 폴더(`~`는 내 사용자 폴더)에 압축을 푼다. 명령의 파일 이름은 내려받은 파일의 이름으로 바꾼다.

   터미널 · macOS/Linux

   ```
   $ mkdir -p ~/dev
   $ cd ~/dev
   $ tar xzf ~/Downloads/OpenJDK21U-jdk_aarch64_mac_hotspot_21.0.12.1_1.tar.gz
   $ ls ~/dev
   ```

   `mkdir -p`는 폴더가 이미 있으면 그대로 두고, `tar xzf`는 gzip으로 압축한(`z`) 파일을 푼다(`x`, `f` 뒤가 파일 이름). `jdk-21.0.12.1+1` 같은 폴더가 생긴다. macOS용 JDK는 `bin`이 폴더 바로 아래가 아니라 `Contents/Home` 아래에 있으므로, `JAVA_HOME`은 `Contents/Home`까지다.

2. 셸 설정 파일을 연다. macOS의 기본 셸(터미널에서 입력한 명령을 읽어 실행하는 프로그램)은 zsh이고, zsh는 시작할 때마다 설정 파일 `~/.zshrc`를 읽는다. 이 파일은 처음에는 없을 수 있고, 이름이 점으로 시작해 Finder에 보이지 않는다. 먼저 만든 뒤 텍스트 편집기에서 연다.

   터미널 · macOS/Linux

   ```
   $ touch ~/.zshrc
   $ open -e ~/.zshrc
   ```

   터미널 안에서 편집하려면 nano로 연다. nano는 Ctrl+O, Enter로 저장하고 Ctrl+X로 나온다.

   터미널 · macOS/Linux

   ```
   $ nano ~/.zshrc
   ```

3. 파일 끝에 다음 두 줄을 넣고 저장한다. 폴더 이름은 1번에서 `ls`로 본 그대로 쓴다.

터미널 · macOS/Linux

```
$ export JAVA_HOME="$HOME/dev/jdk-21.0.12.1+1/Contents/Home"
$ export PATH="$JAVA_HOME/bin:$PATH"
```

첫 줄은 `JAVA_HOME`을 만들고, 둘째 줄은 원래의 `PATH`(`$PATH`) 맨 앞에 JDK의 `bin`을 넣는다. `$HOME`은 내 사용자 폴더이고, 폴더와 폴더 사이는 `:`로 나눈다.

4. 지금 열린 터미널에 반영한다.

터미널 · macOS/Linux

```
$ source ~/.zshrc
```

**Linux.** CPU에 맞춰 `OpenJDK21U-jdk_x64_linux_hotspot_…tar.gz`나 `…_aarch64_linux_hotspot_…tar.gz`를 내려받아 macOS와 같은 명령으로 푼다. Linux용 JDK는 압축을 풀어 생긴 폴더(`jdk-21.0.12.1+1`) 바로 아래에 `bin`이 있으므로, `Contents/Home`을 붙이지 않고 그 폴더를 `JAVA_HOME`으로 쓴다. 내 셸이 무엇인지는 다음 명령으로 본다.

터미널 · macOS/Linux

```
$ echo $SHELL
```

결과가 `/bin/bash`처럼 bash로 끝나면 `~/.zshrc` 대신 `~/.bashrc`에 같은 두 줄을 넣는다.

두원은 마르코와 영상 통화를 연결해 둔 채 macOS의 순서를 따라갔다. 그런데 폴더 이름을 잘못 적어 `~/.zshrc`를 다시 고치다가 투덜거렸다.

> **두원** 아, 이런 건 그냥 인터넷에 있는 설치 명령 복사해서 붙여 넣으면 안 돼?
>
> **마르코** 네 노트북이면 그래도 괜찮아. 근데 나중에 블로그를 배포한 서버에서도 뭘 하는지 모르는 명령을 붙여 넣을 거야?
>
> **두원** 안 될 건 없잖아.

마르코 나도 그런 적 있어. 서버에서 급하게 오류 메시지로 검색해서 나온 명령을 그대로 붙여 넣었거든. 오류는 사라졌는데, 다음 날 다른 프로그램이 실행이 안 되는 거야. 자바 버전을 가리키는 설정 한 줄이 바뀌어 있었어. 어느 글에서 본 명령인지도, 뭘 바꿨는지도 기록이 없으니 되돌릴 수도 없더라. 그거 찾느라 몇 시간을 썼지.

두원 음, 그건 좀 무섭네.

마르코 남이 써 둔 명령을 쓰지 말라는 얘기가 아니야. 그 명령이 뭘 바꾸는지 알고 써야 결과가 맞는지도 보고, 문제가 생기면 되돌릴 수도 있거든.

두원은 어제 국숫집에서 마르코가 한 말이 생각났다. 시키는 것도 알아야 시킬 수 있다. 설치 명령 하나도 마찬가지였다. 두원은 남은 설정을 직접 마무리하고, 터미널을 새로 열어 설치가 제대로 되었는지 확인했다.

터미널 · macOS/Linux Windows 730쪽

```
$ echo $JAVA_HOME
$ which java
$ java -version
$ javac -version
/Users/user/dev/jdk-21.0.12.1+1/Contents/Home
/Users/user/dev/jdk-21.0.12.1+1/Contents/Home/bin/java
openjdk version "21.0.x" …
… (생략: 런타임과 VM 이름 두 줄)
javac 21.0.x
```

두 번째 줄(`which java`의 결과, Windows는 `where.exe java`)이 방금 설정한 폴더의 `java`를 가리키고, `java`와 `javac`의 버전이 모두 21로 시작하면 된다. 다른 경로가 먼저 나오면 예전에 설치한 자바가 `PATH` 앞쪽에 남아 있다는 뜻이다. macOS에서 `/usr/bin/java`가 나오면 `$JAVA_HOME/bin`을 `PATH`의 맨 뒤에 붙였는지 본다. `export PATH="$JAVA_HOME/bin:$PATH"`처럼 맨 앞에 넣어야 한다. 생략한 줄의 VM은 1.2절에서 볼 JVM(자바 가상 머신)이다.

### TIP javac를 찾을 수 없을 때

`javac`를 찾을 수 없다는 메시지가 나오면 JDK의 `bin` 폴더가 `PATH`에 빠져 있다. JRE(Java Runtime Environment, 실행에 필요한 것만 담고 `javac`는 없는 묶음)만 설치되어 있으면

`java`는 실행되는데 `javac`만 찾지 못하는 일도 있다. 여러 버전의 JDK를 두고 바꿔 쓰는 법과 패키지 관리자로 설치하는 법은 부록 A의 A.2절에 정리해 두었다.

### `git clone` 실습 저장소 내려받기

실습 코드는 Git 저장소로 내려받는다. Git이 설치되어 있는지는 다음 명령으로 확인한다.

터미널

```
$ git --version
```

버전이 나오지 않으면 부록 A.5절의 순서대로 먼저 설치한다.

### 참고 Git과 브랜치

Git은 파일이 바뀌어 온 기록을 남기는 버전 관리 도구다. 저장소(repository)는 그 기록을 담은 폴더이고, 기록을 남기는 한 번 한 번을 **커밋**(commit)이라고 한다. 브랜치(branch)는 커밋이 이어지는 흐름 하나에 붙인 이름이다. 한 저장소 안에 여러 흐름의 코드를 나란히 두고, 필요한 브랜치로 전환해 가며 작업할 수 있다. GitHub(https://github.com)은 Git 저장소를 인터넷에 올려 두고 여러 사람이 함께 쓰는 서비스다. 이 장의 실습 저장소 주소는 https://github.com/backend-blog/build-and-packaging이다. `git clone`은 저장소를 기록까지 통째로 내려받고, `git switch`는 브랜치를 전환해 폴더의 파일을 그 브랜치의 내용으로 바꾼다. 그 밖의 Git 명령은 책을 따라가며 필요할 때마다 소개한다.

커밋에 남을 이름과 이메일(`git config --global user.name`, `user.email`)도 부록 A.5절에서 한 번 정해 둔다. 정하지 않으면 1.3절에서 처음 커밋할 때 Git이 "Please tell me who you are"라는 메시지를 출력하고 커밋하지 않는다.

JDK 설치를 마쳤을 즈음 마르코가 메시지를 보냈다. 실습 저장소 주소에 이어 이렇게 적혀 있었다. "단계마다 실습 브랜치(exercise)와 완성 브랜치(solution)를 나눠 뒀어. 실습 브랜치에서 직접 해 보고, 다 하면 완성 브랜치랑 비교해 봐." 두원은 통화를 이어 둔 채 저장소를 처음으로 자기 컴퓨터에 내려받기로 했다. 앞으로 내려받을 실습 저장소는 모두 JDK를 푼 `~/dev`(Windows는 `C:\dev`) 폴더에 모아 둔다. 먼저 그 폴더로 이동한다.

터미널 · macOS/Linux Windows 731쪽

```
$ cd ~/dev
```

그리고 저장소를 내려받아 1단계 실습 브랜치로 전환한다.

터미널

```
$ git clone https://github.com/backend-blog/build-and-packaging.git
$ cd build-and-packaging
$ git switch step1-exercise
```

터미널에 `Switched to a new branch 'step1-exercise'`가 나오면 브랜치가 바뀌었다. 지금 어느 브랜치에 있는지는 언제든 다음 명령으로 확인할 수 있다. 이름 앞에 `*`가 붙은 줄이 지금 있는 브랜치다.

터미널

```
$ git branch
```

실습 저장소에는 단계마다 브랜치가 두 개씩 있다. 이름이 exercise로 끝나는 실습 브랜치에는 채울 곳에 `TODO`가 적힌 코드가 있고, solution으로 끝나는 완성 브랜치에는 그 단계를 마친 코드가 있다. 다음 단계로 넘어갈 때는 브랜치 이름의 숫자만 바꿔 다시 `git switch`를 실행한다. 이 장의 명령은 모두 `build-and-packaging` 폴더 안에서 실행한다.

## 1.2 직접 컴파일하고 실행하기

### 직접 해 보기 1 컴파일하고 실행하기

**브랜치:** `step1-exercise`에서 시작, `step1-solution`과 비교

**목표:** 첫 번째 인자로 명령을 고르는 터미널 블로그를 `javac`와 `java`만으로 컴파일하고 실행한다.

**왜 필요한가:** 소스가 어떻게 실행되는지 알아야 다음 단계에서 볼 classpath(클래스를 찾는 경로) 오류를 이해할 수 있다.

1. `src/net/java21/blog/BlogMain.java`를 컴파일하고, 인자 없이 실행해 사용법이 나오는지 확인한다.
2. `write`로 글을 하나 쓰고, 이어서 `list`를 실행한다.
3. `list`가 왜 아무것도 출력하지 않는지 생각해 본다.

**생각해 보기:** 글이 프로그램을 다시 실행해도 남아 있으려면 메모리 말고 어디에 두어야 할까? 본문에 쉼표가 들어 있어도 나중에 제대로 읽어 내려면 어떤 모양으로 적어야 할까?

실습 브랜치에는 `src/net/java21/blog/BlogMain.java` 하나가 들어 있다. 이 장이 끝날 때까지 블로그의 코드는 모두 이 파일 하나에 작성한다. 글 하나는 번호, 제목, 본문을 담은 record `Post`이고, 글 목록은 `List<Post>`에 담는다. record는 값을 담는 클래스를 짧게 선언하는 자바 문법이다(2.5절에서 다시 본다). 무엇을 할지는 실행할 때 넘기는 첫 번째 인자로 정한다(예제 1-1).

명령은 다섯 가지다. `{ }` 자리에는 실제 값을 넣는다.

- **글쓰기:** `write {제목} {본문}` (예: `write "첫 글" "안녕, 블로그"`)
- **글 목록:** `list`
- **글 읽기:** `view {글 번호}` (예: `view 1`)
- **글 수정:** `edit {글 번호} {제목} {본문}` (예: `edit 1 "첫 글" "수정한 본문"`)
- **글 삭제:** `delete {글 번호}` (예: `delete 1`)

글쓰기(Create), 글 목록과 글 읽기(Read), 글 수정(Update), 글 삭제(Delete)의 네 가지 일을 영어 머리글자를 따서 **CRUD**라고 한다. 블로그든 쇼핑몰이든 데이터를 다루는 프로그램은 대부분 이 네 가지 기능을 기본으로 갖춘다.

예제 1-1은 길어 보여도 명령 하나에 `if` 블록이 하나씩이다. 이 다섯 명령이 각각 어느 블록인지 짝지어 보며 읽으면 된다.

**예제 1-1** `src/net/java21/blog/BlogMain.java`: 글을 메모리의 `List`에 담는 터미널 블로그

```
package net.java21.blog;

import java.util.ArrayList;
import java.util.List;

// 터미널 블로그 첫 버전: 글을 메모리의 List에 담고, 첫 번째 인자로 할 일을 고른다.
public class BlogMain {

    // 글 하나. record는 생성자, 읽는 메서드, equals를 컴파일러가 만들어 준다
    record Post(long id, String title, String body) {
    }
```

```
public static void main(String[] args) {
    // 메모리에만 담는다. 프로그램이 끝나면 목록도 함께 사라진다(1.2절)
    List<Post> posts = new ArrayList<>();

    if (args.length == 0) {
        System.out.println("사용법: write {제목} {본문} ¦ list ¦ view {글 번호}");
        System.out.println("        edit {글 번호} {제목} {본문} ¦ delete {글 번호}");
        return;
    }

    String command = args[0];
    if (command.equals("write")) {
        // 새 번호는 마지막 글의 번호 + 1. 글이 없으면 1번
        long id = posts.isEmpty() ? 1 : posts.getLast().id() + 1;
        posts.add(new Post(id, args[1], args[2]));
        System.out.println(id + "번 글을 저장했습니다.");
    } else if (command.equals("list")) {
        for (Post post : posts) {
            System.out.println(post.id() + ". " + post.title());
        }
    } else if (command.equals("view")) {
        long id = Long.parseLong(args[1]);
        for (Post post : posts) {
            if (post.id() == id) {
                System.out.println("[" + post.title() + "] " + post.body());
            }
        }
    } else if (command.equals("edit")) {
        long id = Long.parseLong(args[1]);
        for (int i = 0; i < posts.size(); i++) {
            if (posts.get(i).id() == id) {
                // 새 Post로 바꿔 넣는다. record는 값을 바꿀 수 없다
                posts.set(i, new Post(id, args[2], args[3]));
                System.out.println(id + "번 글을 수정했습니다.");
            }
        }
    } else if (command.equals("delete")) {
        long id = Long.parseLong(args[1]);
        for (int i = 0; i < posts.size(); i++) {
            if (posts.get(i).id() == id) {
                posts.remove(i);
                System.out.println(id + "번 글을 삭제했습니다.");
            }
        }
    } else {
```

```
            System.out.println("알 수 없는 명령입니다: " + command);
        }
    }
}
```

예를 들어 터미널에서 다음 명령으로 실행하면(컴파일은 곧이어 한다)

터미널

```
$ java -cp out net.java21.blog.BlogMain write "첫 글" "안녕, 블로그"
```

`main`의 `args`에는 `write`, `첫 글`, `안녕, 블로그` 세 문자열이 차례로 담긴다. 큰따옴표는 공백이 든 값을 인자 하나로 묶으라는 표시이고, `args`에는 들어가지 않는다. `main`은 `args[0]`을 보고 `if` 문으로 할 일을 고른다. 새 글의 번호는 목록의 마지막 글 번호에 1을 더해 정한다. `getLast()`는 자바 21부터 `List`에 추가된 메서드다. 마지막 요소를 돌려주고, 목록에서 빼지는 않는다.

두원은 코드를 컴파일하고, 인자 없이 한 번 실행해 보았다.

터미널 · macOS/Linux Windows 731쪽

```
$ javac -d out src/net/java21/blog/BlogMain.java
$ ls out/net/java21/blog
$ java -cp out net.java21.blog.BlogMain
```

실행 결과

```
BlogMain$Post.class
BlogMain.class
사용법: write {제목} {본문} ¦ list ¦ view {글 번호}
        edit {글 번호} {제목} {본문} ¦ delete {글 번호}
```

두 명령이 한 일은 그림 1-3과 같다. 소스가 `.class`로 바뀌고 JVM(자바 가상 머신, `.class`를 읽어 실행하는 프로그램)에서 실행되기까지, 그림을 따라가며 살펴보자.




**그림 1-3** `javac`는 소스를 `.class`로 바꾸고, `java`는 classpath에서 그 `.class`를 찾아 실행한다

`javac`는 소스(`.java`)를 읽어 `out/net/java21/blog/BlogMain.class`를 만든다. record `Post`는 `BlogMain` 안에 선언했으므로 `BlogMain$Post.class`라는 파일로 따로 생긴다. `java`가 실행하는

것은 이 `.class`다. 소스 파일을 지워도 `.class`만 있으면 프로그램은 실행된다.

`javac`의 `-d out`은 만든 `.class`를 `out` 폴더 아래에 패키지 구조대로 두라는 옵션이고, `java`의 `-cp out`은 "클래스는 `out` 폴더에서 찾아라"라는 뜻이다. 클래스를 찾는 이 경로를 **classpath**라고 한다. `net.java21.blog.BlogMain`이라는 이름은 `out` 아래의 `net/java21/blog/BlogMain.class` 파일에 대응한다. 패키지 이름의 점(`.`)이 폴더 구분 문자 `/`로 바뀐다(그림 1-3).

두원은 글을 써 보려다 본문을 빠뜨렸다.

터미널

```
$ java -cp out net.java21.blog.BlogMain write "첫 글"
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
        Index 2 out of bounds for length 2
    at net.java21.blog.BlogMain.main(BlogMain.java:27)
```

> 두원 첫 실행부터 오류네.
>
> 마르코 몇 번째 줄에서 났대?

이처럼 어떤 예외가 발생했고 그 예외가 어떤 메서드 호출을 거쳐 왔는지 차례로 보여 주는 출력을 **스택 트레이스**(stack trace)라고 한다. 첫 줄(지면이 좁아 두 줄로 나눠 실었다)은 무슨 일이 일어났는지를 나타낸다. 길이가 2인 배열에서 2번 칸을 꺼내려 했다는 뜻이다. 그 아래의 `at` 줄은 예외가 `BlogMain.java`의 27번째 줄에서 발생했다고 알려 준다. 호출이 여러 단계를 거쳤다면 `at` 줄이 여러 개 나오고, 맨 위의 `at` 줄이 예외가 발생한 곳이다.

27번째 줄은 `posts.add(new Post(id, args[1], args[2]));`다. 인자를 두 개만 넘겼으니 `args`에는 0번과 1번 칸뿐이다. 두원은 본문까지 넣어 다시 실행하고, 이어서 목록을 보았다.

터미널

```
$ java -cp out net.java21.blog.BlogMain write "첫 글" "안녕, 블로그"
$ java -cp out net.java21.blog.BlogMain list
1번 글을 저장했습니다.
```

저장했다는 메시지는 나왔는데 `list`는 아무것도 출력하지 않았다. `view 1`도 마찬가지였다.

두원 분명히 저장했다고 나왔는데? 어디로 간 거지?

마르코 글을 어디에 담았는지 코드를 다시 볼래?

글은 `main` 안에서 만든 `posts`, 곧 메모리에 있는 `List`에 담겼다. `java` 명령을 실행하면 JVM이 하나 시작되고, `main`이 끝나면 JVM도 종료된다. 이때 JVM이 쓰던 메모리는 운영체제로 반환되고, 그 안의 `List`도 함께 사라진다. `write`와 `list`는 서로 다른 JVM에서 실행되었고, `list` 쪽은 빈 `List`에서 시작했다.

프로그램이 끝난 뒤에도 데이터가 남는 성질을 **영속성**(persistence)이라고 한다. 메모리에 둔 데이터에는 영속성이 없다.

두원 그럼 고치기랑 지우기도 소용이 없네. 고칠 글이 남아 있질 않으니까.

마르코 그렇지. 프로그램이 끝나도 남아 있어야 하는 데이터는 메모리 말고 다른 데 둬야 해. 제일 쉬운 건?

두원 파일. 글을 파일에 적어 두고, 시작할 때 다시 읽으면 되겠네. 한 줄에 글 하나씩, `1, 첫 글,안녕, 블로그` 이렇게.

마르코 쉼표로 나눌 거야? 방금 쓴 본문엔?

본문 "안녕, 블로그"에 이미 쉼표가 있다. 쉼표로 나누면 이 줄은 `1`, `첫 글`, `안녕`, `블로그` 네 칸이 된다. 구분 문자를 탭으로 바꿔도, 그 문자가 제목이나 본문에 들어오면 같은 문제가 다시 생긴다.

마르코 지금은 번호, 제목, 본문뿐이잖아. 나중에 작성 시각이나 태그가 붙으면 이 파일은 어떻게 될까? 브라우저나 다른 프로그램이 이 글을 읽어 가야 할 때는?

두원이 생각한 답은 JSON이었다. 웹에서 데이터를 주고받을 때 가장 흔히 쓰는 형식이다.

JSON(JavaScript Object Notation, 제이슨)은 이름 그대로 JavaScript의 객체 표기법에서 왔다. JavaScript 코드에서 `{ title: "첫 글" }`처럼 객체를 적는 그 표기법을 빌려, 프로그램 사이에 데이터를 주고받는 텍스트 형식으로 정리한 것이다. 다만 JSON에서는 `title` 같은 이름

(키)에도 반드시 큰따옴표를 붙여 `{ "title": "첫 글" }`처럼 적는다. 지금은 JavaScript와 상관없이 인터넷 표준 문서(RFC 8259)로 정해져 있고, 자바를 비롯한 거의 모든 언어가 읽고 쓸 수 있다.

JSON 값의 타입은 표 1-1의 여섯 가지뿐이다.

**표 1-1** JSON 값의 여섯 가지 타입

| 타입 | 적는 법 | 예 |
|---|---|---|
| 문자열 | 큰따옴표로 감싼다 | `"첫 글"` |
| 숫자 | 정수와 소수를 구분하지 않는다 | `3`, `1.5` |
| 불리언 | 참과 거짓 | `true`, `false` |
| 값 없음 | 비어 있음을 나타낸다 | `null` |
| 객체 | 중괄호 안에 `"이름": 값` 쌍을 쉼표로 나열한다 | `{"id": 1, "title": "첫 글"}` |
| 배열 | 대괄호 안에 값을 순서대로 나열한다 | `[1, 2, 3]` |

객체는 보통 여러 줄로 나눠, 쌍 하나를 한 줄에 적는다. 쌍과 쌍 사이에는 쉼표를 넣고, 마지막 쌍 뒤에는 넣지 않는다.

```
{
  "id": 1,
  "title": "첫 글",
  "body": "안녕"
}
```

복잡한 구조는 객체와 배열을 서로 중첩해 만든다. 객체 안에 배열을, 배열 안에 객체를 넣을 수 있고, 표준은 그 깊이를 제한하지 않는다. 다만 읽는 프로그램은 깊이에 한도를 두어도 된다. JSON을 읽는 라이브러리(1.3절)도 대개 지나치게 깊은 JSON을 막는 기본 한도를 둔다. 블로그의 글 목록이라면 이렇게 적힌다.

```
[
  {
    "id": 1,
    "title": "첫 글",
```

```
    "body": "안녕, 블로그"
  },
  {
    "id": 2,
    "title": "둘째 글",
    "body": "\"따옴표\"도 쓸 수 있다",
    "tags": ["일기", "자바"]
  }
]
```

값의 경계는 큰따옴표가 정하므로, 본문에 쉼표가 들어와도 구조가 어긋나지 않는다. 본문에 든 큰따옴표는 `\"`처럼 앞에 역슬래시를 붙여 escape(이스케이프)한다. 둘째 글처럼 `tags`를 추가해도 첫 글을 읽는 데는 지장이 없다. 쉼표로 구분한 파일에서 생기던 두 문제가 모두 해결된다.

JSON이 널리 쓰이게 된 데에는 이유가 있다. 사람이 읽고 고칠 수 있는 텍스트 형식이면서, 같은 용도로 쓰이던 XML(태그로 값을 감싸는 형식, 1.5절에서 본다)보다 훨씬 짧다. 브라우저의 JavaScript가 별도 도구 없이 바로 읽을 수 있어서, 웹 화면과 서버가 데이터를 주고받는 형식으로 자리 잡았다. 지금은 웹 API(다른 프로그램이 요청해 데이터를 받아 가도록 만든 주소)의 응답, 설정 파일, 로그까지 JSON으로 적는 경우가 많다. 이 책의 블로그는 화면을 HTML로 응답하지만, 글은 JSON 파일에 저장한다(1.3절에서 시작해 7장의 JSON 파일 저장소까지 이어진다).

문제는 JSON을 읽고 쓰는 코드를 직접 작성하는 일이었다. 따옴표와 역슬래시를 하나하나 처리하다 보면 주말이 다 갈 것 같았다. 두원은 이미 만들어진 코드를 찾아보기로 했다.

## 1.3 라이브러리와 classpath

### 직접 해 보기 2 다른 사람이 만든 라이브러리 쓰기

**브랜치:** `step2-exercise`에서 시작, `step2-solution`과 비교

**목표:** JSON 라이브러리 Jackson으로 글 목록을 `posts.json`에 저장하고, 컴파일과 실행 모두에서 classpath를 지정한다.

**왜 필요한가:** 프로그램은 대부분 다른 사람이 만든 라이브러리를 가져다 쓴다. JVM이 라이브러리를 어디서 찾는지 모르면 실행 오류의 원인을 알 수 없다.

1. 2단계 실습 브랜치로 전환하고, `fetch-libs.sh`로 `lib/`에 JAR 세 개를 내려받는다(Windows는 Git Bash에서 실행한다).

터미널

```
$ git switch step2-exercise
$ ./fetch-libs.sh
```

2. `BlogMain.java`에서 글을 고치는 `TODO#1`(`edit`)과 글을 지우는 `TODO#2`(`delete`)를 채운다. 첫 버전에 있던 두 명령을 연습용으로 비워 두었다.
3. 본문의 명령을 차례로 실행하며 오류를 직접 확인한다.
4. 글을 쓰고 프로그램을 다시 실행해도 `list`에 글이 남아 있는지, `posts.json`에 JSON으로 저장되는지 확인한다.

**생각해 보기:** 컴파일할 때 이미 `-cp`로 Jackson의 위치를 알려 주었는데, 실행할 때 왜 또 알려 주어야 할까?

JSON을 다루는 코드는 이미 많은 사람이 만들어 두었다. 마르코는 그중 자바에서 가장 널리 쓰는 Jackson을 권했다. 이렇게 다른 사람이 만들어 JAR 파일(클래스 파일을 하나로 묶은 압축 파일)로 배포하는 코드를 **라이브러리**라고 한다. JAR는 Java ARchive를 줄인 이름이다. 영어 낱말 jar(병, 단지)와 철자가 같아서 그 낱말처럼 "자"라고 읽는다. 철자대로 "자르"라고 읽는 사람도 많아서 둘 다 통한다.

실습 브랜치의 `fetch-libs.sh`는 Jackson의 JAR 세 개를 `lib/` 폴더에 내려받는 스크립트다. 브랜치를 전환하고 스크립트를 실행한 뒤, `lib/`에 무엇이 생겼는지 본다. 스크립트는 JAR마다 내려받는 주소(`받는 중: …`)와 확인 결과(`확인: lib/…`)를 한 줄씩 출력한다. 이어지는 결과는 그 뒤에 실행한 `ls lib`의 출력이다.

터미널 Windows 731쪽

```
$ git switch step2-exercise
$ ./fetch-libs.sh
$ ls lib
jackson-annotations-2.22.jar
jackson-core-3.2.3.jar
jackson-databind-3.2.3.jar
```

**두원** 라이브러리 하나 쓰는데 JAR가 세 개야?

**마르코** 우리가 고른 건 `jackson-databind` 하나야. 나머지 둘은 `jackson-databind`가 쓰는 거고. 왜 셋 다 있어야 하는지는 이따 실행해 보면 알게 돼. 버전이 서로 다른 이유는 나중에 보자.

**두원** 이 JAR들은 어디서 구한 거야?

**마르코** 그건 내일 알려 줄게. 오늘은 JAR가 있다 치고, 이걸 어떻게 써야 프로그램이 돌아가는지부터 보자.

`BlogMain.java`는 시작할 때 `posts.json`을 읽어 `posts`에 담고, 명령을 처리한 뒤 `posts`를 통째로 `posts.json`에 다시 쓰도록 바뀌었다. 첫 버전에서 바뀐 부분만 보면 예제 1-2와 같다.

**예제 1-2** `src/net/java21/blog/BlogMain.java`: Jackson으로 `posts.json`에 저장하도록 바뀐 부분

```
import java.nio.file.Files;
import java.nio.file.Path;
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.ObjectMapper;
import tools.jackson.databind.SerializationFeature;
import tools.jackson.databind.json.JsonMapper;

// 터미널 블로그 둘째 버전: 글 목록을 posts.json에 JSON으로 저장한다(Jackson).
public class BlogMain {

    public static void main(String[] args) throws Exception {
        Path file = Path.of("posts.json");
        // 자바 객체와 JSON 문자열을 서로 바꿔 준다. INDENT_OUTPUT은 들여 쓰기
        ObjectMapper mapper = JsonMapper.builder()
                .enable(SerializationFeature.INDENT_OUTPUT)
                .build();

        List<Post> posts = new ArrayList<>();
        if (Files.exists(file)) {
            // List<Post>의 Post는 컴파일할 때 지워지니 TypeReference로 알린다
            posts.addAll(mapper.readValue(Files.readString(file),
                    new TypeReference<List<Post>>() { }));
        }
        // … (생략: 첫 버전과 같은 명령 처리. edit와 delete는 TODO)
```

```
        // 명령을 마치면 목록 전체를 파일에 다시 쓴다. 다음 실행이 이어서 읽는다
        Files.writeString(file, mapper.writeValueAsString(posts));
    }
}
```

---

`ObjectMapper`는 자바 객체와 JSON 문자열을 서로 바꿔 준다. `JsonMapper`는 JSON용 `ObjectMapper`이고, `builder()`로 설정을 이어 붙인 뒤 `build()`로 만든다. `writeValueAsString`은 `List<Post>`를 JSON 문자열로, `readValue`는 그 반대로 바꾼다. record의 필드 이름 `id`, `title`, `body`가 그대로 JSON의 이름이 된다. `INDENT_OUTPUT`은 사람이 읽기 좋게 줄을 바꾸고 들여쓰라는 설정이다.

`TypeReference`는 "`Post`를 담은 `List`로 읽어라"처럼 결과의 타입을 알려 주는 클래스다. `jackson-core`에 들어 있다. 제네릭의 타입 정보(`List<Post>`의 `Post`)는 컴파일할 때 지워져 실행 중에는 알 수 없으므로 이렇게 따로 알려 준다. 지금은 목록을 읽을 때 이렇게 쓴다고만 알아두어도 된다. 끝의 `{ }` 때문에 익명 클래스(이름 없이 그 자리에서 만드는 클래스)가 하나 생겨, 컴파일하면 `BlogMain$1.class`가 하나 더 나온다. 인터넷의 Jackson 예제는 대부분 패키지 이름이 `com.fasterxml.jackson`으로 시작하는데, 이름이 다른 까닭은 1.5절에서 본다.

`main`의 명령 처리 가운데 `edit`와 `delete` 두 자리는 직접 채워 보도록 비워 두었다. 실습 브랜치에서는 그 자리에 TODO 주석만 남아 있다.

**실습 브랜치의 모습** `src/net/java21/blog/BlogMain.java`: 비워 둔 `edit`와 `delete`

---

```
} else if (command.equals("view")) {
    // … (생략: 첫 버전과 같다. 번호가 같은 글을 찾아 출력한다)
} else if (command.equals("edit")) {
    // TODO#1 args[1] 번 글을 args[2](제목), args[3](본문)으로 바꾸고
    //        "N번 글을 수정했습니다."를 출력한다.
    //        (1.3절) view의 반복문을 본뜨되, 찾은 자리를 알도록
    //        인덱스 i로 반복한다. record는 값을 바꿀 수 없으니
    //        같은 번호로 새 Post를 만들어 그 자리에 넣는다.
    //        힌트: List.get(i), List.set(i, 새 값)
} else if (command.equals("delete")) {
    // TODO#2 args[1] 번 글을 목록에서 지우고
    //        "N번 글을 삭제했습니다."를 출력한다.
    //        (1.3절) TODO#1과 같이 인덱스 i로 돌며 번호가
    //        같은 글을 찾는다.
```

```
        //        힌트: List.remove(int index)
    } else {
        System.out.println("알 수 없는 명령입니다: " + command);
    }
```

글을 고치는 `TODO#1`은 `view`처럼 번호가 같은 글을 찾되, 찾은 자리를 알아야 하므로 `for (Post post : posts)` 대신 인덱스 `i`를 쓰는 `for` 문으로 반복한다. record는 값을 바꿀 수 없으니 같은 번호로 새 `Post`를 만들어 `posts.set(i, …)`로 그 자리에 넣는다. 글을 지우는 `TODO#2`는 같은 방법으로 찾은 자리를 `posts.remove(i)`로 지운다. 두 명령 모두 일을 마친 뒤에 "N번 글을 수정했습니다.", "N번 글을 삭제했습니다."를 출력한다. 첫 버전에 있던 두 블록을 되살리는 일이다.

`TODO#1`과 `TODO#2`를 직접 채운 뒤 예제 1-1과 비교해 보자.

두원은 TODO 자리에 `edit`와 `delete`를 채웠다. `view`에서 번호로 글을 찾던 반복문을 본떠, 찾은 자리의 글을 바꾸거나 지웠다. 그리고 완성 브랜치와 비교해 보았다. 정답을 맞혀 보려는 비교가 아니다. 어느 줄이 왜 다른지 따져 보려는 비교다.

터미널

```
$ git diff origin/step2-solution -- src
… (생략: 비교한 파일 이름과 줄 번호)
             for (int i = 0; i < posts.size(); i++) {
                 if (posts.get(i).id() == id) {
                     posts.remove(i);
-                    System.out.println(id + "번 글을 삭제했습니다.");
                 }
             }
+            System.out.println(id + "번 글을 삭제했습니다.");
```

`origin/`은 내려받아 온 원래 저장소의 브랜치라는 뜻이고, `-- src`는 `src` 폴더만 비교하라는 뜻이다. `-`로 시작하는 줄은 완성 브랜치에만 있고, `+`로 시작하는 줄은 두원의 코드에만 있다. 두원은 "삭제했습니다" 출력을 반복문 밖에 두었다. 이렇게 하면 `delete 7`처럼 없는 번호를 넘겨도 "7번 글을 삭제했습니다."가 출력된다. 두원은 출력을 `if` 안으로 옮기고, 이 단계에서 처음으로 컴파일했다. 그러자 옮긴 줄과는 상관없는 오류가 여러 개 발생했다.

터미널

```
$ javac -d out src/net/java21/blog/BlogMain.java
src/net/java21/blog/BlogMain.java:7: error: package tools.jackson.core.type
does not exist
import tools.jackson.core.type.TypeReference;
                            ^
… (생략: 비슷한 오류 일곱 개. package does not exist, cannot find symbol)
8 errors
```

**두원** 이상하네. JAR가 `lib`에 분명히 있는데 못 찾는대.

**마르코** 있는 건 맞아. 그런데 `lib`에 있다는 걸 `javac`는 어떻게 알까?

컴파일러도 클래스를 classpath에서 찾는다. `lib/`라는 폴더 이름은 두원에게만 의미가 있을 뿐, 컴파일러는 그 폴더를 따로 살펴보지 않는다. 컴파일할 때도 `-cp`로 JAR를 알려 줘야 한다.

터미널

```
$ javac -d out -cp "lib/*" src/net/java21/blog/BlogMain.java
$ java -cp out net.java21.blog.BlogMain list
Error: Unable to initialize main class net.java21.blog.BlogMain
Caused by: java.lang.NoClassDefFoundError: tools/jackson/databind/ObjectMapper
```

`lib/*`는 "`lib` 폴더의 JAR 전부"라는 뜻이다. JAR를 하나하나 적는 대신 `*`로 줄여 적었다. 이 `*`를 JAR 목록으로 바꾸는 일은 셸이 아니라 `javac`와 `java`가 맡아야 한다. 그래서 셸이 `*`를 먼저 파일 이름으로 바꾸지 않도록 큰따옴표로 감싸 `javac`와 `java`에 그대로 넘긴다. 컴파일은 됐다. 그런데 실행하자마자 오류로 끝났다.

**두원** 아니, 컴파일은 됐잖아. 왜 이제 와서 없다는 거야?

**마르코** 컴파일할 때 찾은 `ObjectMapper`를, 실행할 때는 어디서 찾을 것 같아?

무엇이 달랐는지는 두 명령의 classpath를 위아래로 놓고 비교한 그림 1-4를 보면 분명해진다.

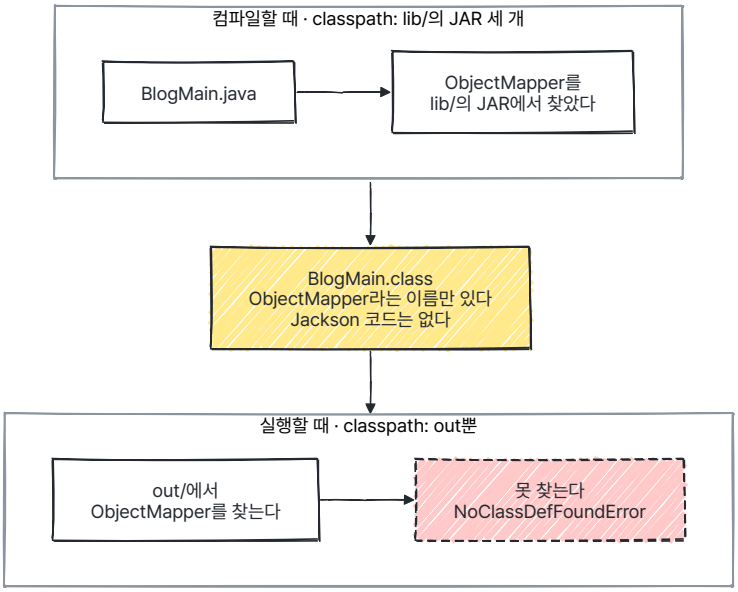



**그림 1-4** `BlogMain.class`에는 `ObjectMapper`의 이름만 있다. 컴파일할 때 찾은 JAR가 실행할 때의 classpath에 없으면 실행이 실패한다

컴파일러는 `BlogMain.class`에 "`tools.jackson.databind.ObjectMapper`를 쓴다"는 이름만 적는다. Jackson의 코드를 복사해 넣지 않는다. 실행할 때 JVM이 그 이름의 클래스를 다시 찾아야 하고, 찾을 곳은 실행 명령의 classpath뿐이다. 이번 실행의 classpath는 `out` 하나였다. 첫 줄의 "Unable to initialize main class"는 JVM이 `main`을 호출하기도 전에, `BlogMain` 클래스를 실행할 준비를 하다가 실패했다는 뜻이다. 컴파일에 성공했다고 실행까지 성공하는 것은 아니다.

이번에는 실행할 때도 `lib/*`를 classpath에 넣는다. classpath에 여러 곳을 적을 때 구분자는 Linux와 macOS에서 `:`, Windows에서 `;`다.

터미널 · macOS/Linux Windows 732쪽

```
$ java -cp "out:lib/*" net.java21.blog.BlogMain write "첫 글" "안녕, 블로그"
$ java -cp "out:lib/*" net.java21.blog.BlogMain \
    write "둘째 글" "오늘은 자바를 컴파일했다"
$ java -cp "out:lib/*" net.java21.blog.BlogMain list
$ java -cp "out:lib/*" net.java21.blog.BlogMain \
    edit 2 "둘째 글" "인자로 CRUD를 만들었다"
$ java -cp "out:lib/*" net.java21.blog.BlogMain delete 1
$ java -cp "out:lib/*" net.java21.blog.BlogMain list
$ cat posts.json
```

실행 결과

```
1번 글을 저장했습니다.
2번 글을 저장했습니다.
1. 첫 글
2. 둘째 글
2번 글을 수정했습니다.
1번 글을 삭제했습니다.
2. 둘째 글
[ {
  "id" : 2,
  "title" : "둘째 글",
  "body" : "인자로 CRUD를 만들었다"
} ]
```

드디어 실행되었다. 이번에는 앞의 실행에서 쓴 글이 다음 실행의 `list`에 그대로 나온다. 고치고 지운 결과도 마찬가지다. 프로그램은 끝나도 `posts.json`은 디스크에 남기 때문이다. 줄바꿈과 콜론 앞의 공백은 Jackson이 들여 쓴 모양일 뿐, 앞에서 본 것과 같은 JSON이다.

줄 끝의 `\`는 명령이 다음 줄로 이어진다는 표시다.

> **두원** 근데 JAR 세 개가 다 필요한 거 맞아? 우리가 고른 건 `jackson-databind` 하나라며.
>
> **마르코** 한번 해 볼래? `jackson-databind`만 classpath에 넣고.

터미널 · macOS/Linux Windows 732쪽

```
$ java -cp "out:lib/jackson-databind-3.2.3.jar" net.java21.blog.BlogMain list
```

실행 결과

```
Error: Unable to initialize main class net.java21.blog.BlogMain
Caused by: java.lang.NoClassDefFoundError: tools/jackson/core/TreeCodec
```

이번에는 못 찾은 클래스가 `tools.jackson.core` 패키지에 있다(한국어 Windows에서는 앞에서 본 것처럼 `오류:`, `원인:`으로 시작하는 한국어로 나온다). `TreeCodec`은 두원의 코드에 없는 이름이다. `jackson-databind`의 `ObjectMapper`가 `jackson-core`의 클래스를 쓰고 있었다. 프로그램이 쓰는 라이브러리가 또 다른 라이브러리를 쓰면, 그 라이브러리도 classpath에 있어야 한다.

> **마르코** 그 명령을 매번 입력할 거야? 라이브러리가 열 개면, 그 라이브러리들이 또 뭘 쓰는지는 어떻게 다 알아낼 거야?

두원은 대답하지 못했다. 오늘은 마르코가 세 개를 챙겨 줬지만, 혼자였다면 오류를 하나씩 겪으며 찾아야 했을 것이다.

### 참고 저장 방법이 또 바뀐다면

메모리에서 파일로 바꾸면서 달라진 곳은 `main`의 처음(파일을 읽어 `posts`에 담기)과 끝(`posts`를 파일에 쓰기) 두 곳이었다. 명령을 처리하는 `if`문은 그대로였다. 나중에 파일 대신 데이터베이스에 저장하게 되면 어디를 고쳐야 할까? 명령이 늘어나고 저장할 것이 많아져도 지금처럼 두 곳만 고치면 될까? 이 장에서는 `BlogMain.java` 한 파일로 충분하다. 저장하는 코드와 명령을 처리하는 코드를 나누는 방법은 블로그가 커지면서 뒤의 장에서 하나씩 다룬다.

다음 단계로 넘어가기 전에 두원은 지금까지 고친 것을 커밋해 두었다.

터미널

```
$ git add src
$ git commit -m "edit와 delete 채우기"
```

**주의 고친 채로 브랜치를 전환할 때**

고친 파일을 남겨 둔 채 `git switch`를 실행하면 Git은 "Your local changes … would be overwritten"이라며 전환을 거부한다. 남길 것은 `git commit`으로 남기고, 버릴 것은 `git restore .`로 되돌린 뒤 브랜치를 전환한다.

## 1.4 JAR로 묶기

### 직접 해 보기 3 JAR로 묶기

**브랜치:** `step3-exercise`에서 시작, `step3-solution`과 비교

**목표:** 클래스를 JAR 하나로 묶고, manifest를 작성해 `java -jar` 한 줄로 실행되게 한다.

**왜 필요한가:** 매번 긴 classpath를 입력하지 않고, 다른 사람에게 건넬 수 있는 파일 하나를 만들기 위해서다.

1. 3단계 브랜치로 전환하고, 1.3절과 같은 명령으로 코드를 다시 컴파일해 `out/`을 만들어 둔다. `lib/`가 없으면 그 전에 `./fetch-libs.sh`부터 실행한다(Windows는 Git Bash에서 실행한다).

터미널

```
$ git switch step3-exercise
$ javac -d out -cp "lib/*" src/net/java21/blog/BlogMain.java
```

2. `manifest.txt`(`Main-Class`, `Class-Path`)를 작성한다.
3. JAR 하나로 실행되는지 확인한다.

터미널

```
$ java -jar app.jar list
```

4. JAR 안의 파일 목록에 Jackson 클래스가 없고, JAR만 다른 폴더로 옮기면 실패하는지 확인한다.

터미널

```
$ jar tf app.jar
```

**생각해 보기:** Jackson의 클래스까지 JAR 하나에 모두 담으면 어떤 점이 편해질까? 대신 어떤 점을 따져 봐야 할까?

명령을 줄일 방법을 찾던 두원의 눈에 JAR가 들어왔다. Jackson도 JAR 파일로 배포되고 있었다. JAR는 여러 `.class` 파일을 zip 형식으로 묶은 파일이다. 그 안에 manifest(`META-INF/MANIFEST.MF`)라는 설명 파일을 넣을 수 있다. 시작할 클래스(`Main-Class`)와 함께 쓸 라이브러리의 경로(`Class-Path`)를 적어 두면 `java -jar` 한 줄로 실행된다.

실습 브랜치에는 `manifest.txt`가 없으므로 편집기로 새로 만든다. 자리는 `src`, `lib`, `out`이 있는 저장소 폴더 맨 위다. 이어서 실행할 `jar` 명령이 이 폴더에서 `manifest.txt`를 찾기 때문이다. 두원은 `manifest.txt`에 `Main-Class`와 `Class-Path`를 적었다. `Class-Path`에는 JAR 세 개를 공백으로 나눠 적는다. manifest에서는 `lib/*` 같은 `*`를 쓸 수 없으므로 JAR 이름을 하나하나 적어야 한다.

```
Main-Class: net.java21.blog.BlogMain
Class-Path: lib/jackson-databind-3.2.3.jar
  lib/jackson-core-3.2.3.jar
  lib/jackson-annotations-2.22.jar
```

`Class-Path`가 길어서 세 줄로 나눴다. manifest에서는 공백으로 시작하는 줄을 앞줄에 이어 붙인다. 이때 맨 앞의 공백 하나는 지워지므로, JAR 이름 사이에 공백이 하나 남도록 이어지는 줄 앞에 공백을 두 칸 넣었다. 공백을 ␣로 표시하면, 쓴 것과 JAR가 읽은 것은 이렇다.

```
# 쓴 것
Class-Path: lib/jackson-databind-3.2.3.jar
␣␣lib/jackson-core-3.2.3.jar
# JAR가 읽은 것
Class-Path: lib/jackson-databind-3.2.3.jar␣lib/jackson-core-3.2.3.jar
```

**! 주의 manifest를 쓸 때 빠지기 쉬운 두 함정**

이어지는 줄을 공백 한 칸으로 시작하면 그 공백이 지워져 JAR 이름끼리 붙어 버린다(`…3.2.3.jarlib/…`). 또 파일의 마지막 줄도 줄바꿈(Enter)으로 끝내야 한다. 그렇지 않으면 마지막 줄이 무시되어, 이 예에서는 이어 붙인 `Class-Path` 전체가 빠진다. 두 경우 모두 JAR는 오류 없이 만들어지고, 실행할 때가 되어서야 `NoClassDefFoundError`가 발생한다. 그래서 원인을 찾기 어렵다. 이 오류를 보면 manifest부터 의심한다. 잠시 뒤에 보는 `unzip -p app.jar`

`META-INF/MANIFEST.MF`(Windows는 `tar -xOf`)로 JAR가 읽은 `Class-Path`를 직접 확인할 수 있다.

터미널 · macOS/Linux — Windows 733쪽

```
$ rm -f posts.json        # 1.3절에서 쓴 글 파일을 지우고 새로 시작한다
$ jar --create --file app.jar --manifest manifest.txt -C out .
$ java -jar app.jar write "첫 글" "안녕, 블로그"
$ java -jar app.jar list
```

실행 결과

```
1번 글을 저장했습니다.
1. 첫 글
```

`-C out .`은 "`out` 폴더 안의 것을 JAR의 최상위 경로에 담아라"라는 뜻이다. 이제 classpath도 메인 클래스 이름도 입력하지 않아도 된다.

**두원** 됐다. 이제 이 JAR 하나만 보내면 누구 컴퓨터에서든 돌아가겠지? 너한테 보내 볼까?

**마르코** 좋지. 근데 보내기 전에 그 JAR 안에 뭐가 들었는지는 봤어?

터미널 · macOS/Linux Windows 733쪽

```
$ jar tf app.jar
$ unzip -p app.jar META-INF/MANIFEST.MF
```

실행 결과

```
META-INF/
META-INF/MANIFEST.MF
net/
net/java21/
net/java21/blog/
net/java21/blog/BlogMain$1.class
net/java21/blog/BlogMain$Post.class
net/java21/blog/BlogMain.class
Manifest-Version: 1.0
Main-Class: net.java21.blog.BlogMain
Class-Path: lib/jackson-databind-3.2.3.jar lib/jackson-core-3.2.3.jar li
 b/jackson-annotations-2.22.jar
Created-By: 21.0.x (…)
```

`unzip -p`와 `tar -xOf`는 JAR 안의 파일 하나를 풀지 않고 화면에 출력한다. 두원이 나눈 세 줄은 한 줄로 합쳐졌다가, 이번에는 `li`에서 끊겨 다음 줄로 넘어갔다. manifest는 한 줄을 72바이트까지만 쓰는 규칙이 있어서, `jar` 명령이 긴 줄을 나누고 이어지는 줄의 맨 앞에 공백 하나를 붙인다. 앞에서 본 이어 붙이기 규칙과 같으므로, 읽을 때는 다시 한 줄이 된다.

Linux에 `unzip`이 없으면 파일을 풀어서 본다.

터미널 · macOS/Linux

```
$ jar xf app.jar META-INF/MANIFEST.MF
$ cat META-INF/MANIFEST.MF
```

JAR 안과 밖에 무엇이 있는지 그려 보면 그림 1-5와 같다.

JAR 안의 클래스는 두원의 것 세 개뿐이다(`BlogMain$Post`는 record, `BlogMain$1`은 `TypeReference`의 익명 클래스다). Jackson 클래스는 하나도 없다. 파일 크기도 4KB 남짓이다. 크기는 다음 명령으로 확인할 수 있다.

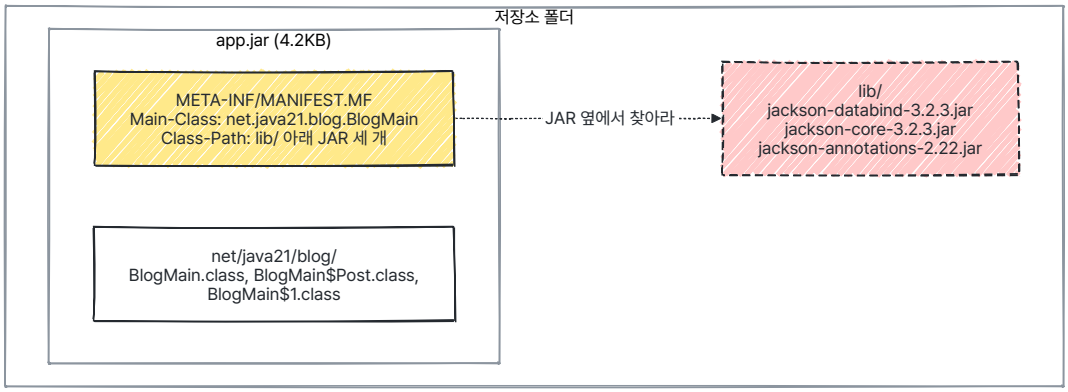



**그림 1-5** app.jar에는 내 클래스와 manifest만 들어 있다. Jackson의 JAR 세 개는 app.jar 바깥에 있고, manifest의 `Class-Path`는 그 위치를 가리킬 뿐이다

터미널

```
$ ls -l app.jar
```

그림 1-5처럼 `Class-Path`는 "JAR 옆의 `lib/`에 있는 JAR 세 개에서 찾아라"라는 경로일 뿐, 라이브러리를 담은 것이 아니다. JAR 파일만 다른 폴더에 복사해 마르코의 컴퓨터와 같은 상황을 만들어 보면 이 점이 바로 드러난다. 이어지는 명령 첫 줄의 `&&`는 앞 명령이 성공하면 뒤 명령을 실행하라는 뜻이다.

터미널 · macOS/Linux Windows 733쪽

```
$ mkdir friend && cp app.jar friend/
$ cd friend
$ java -jar app.jar list
```

실행 결과

```
Error: Unable to initialize main class net.java21.blog.BlogMain
Caused by: java.lang.NoClassDefFoundError: tools/jackson/databind/ObjectMapper
```

두원은 1.3절에서 배운 대로 classpath를 지정해 보았다.

터미널

```
$ java -cp "../lib/*" -jar app.jar list
Error: Unable to initialize main class net.java21.blog.BlogMain
Caused by: java.lang.NoClassDefFoundError: tools/jackson/databind/ObjectMapper
```

**두원** 어? `-cp`로 알려 줬는데도?

**마르코** `-jar`로 실행하면 classpath는 그 JAR랑 manifest가 정해. 명령줄에 적은 `-cp`는 아예 안 봐. 그럼 나한테 보낼 때는 뭘 같이 보내야 할까?

**두원** `lib` 폴더까지… 폴더 구조도 똑같이 맞춰서. 생각보다 번거롭네.

두원은 저장소 폴더로 돌아왔다.

터미널

```
$ cd ..
```

받는 사람이 폴더 구조까지 맞춰야 실행되는 결과물은 보낼 때마다 설명을 덧붙여야 한다. 라이브러리 버전이 바뀌면 manifest도 고쳐야 한다.

**두원** 라이브러리까지 JAR 하나에 다 넣어 버리면 안 돼?

**마르코** 돼. 그런 걸 uber jar라고 해. 라이브러리 JAR를 풀어서 네 클래스랑 같이 다시 묶는 건데, 손으로 하면 라이브러리마다 풀고 합치느라 꽤 지저분해. 그 전에, 오늘 한 일부터 적어 볼래?

두원은 하루 동안 한 일을 적어 보았다.

1. 마르코가 챙겨 준 라이브러리 JAR 세 개를 `lib/`에 두고 썼다. 어디서 구하는지는 아직 모른다.
2. 컴파일할 때와 실행할 때 매번 classpath를 지정했다.
3. manifest 파일을 작성하고, JAR를 만들었다.
4. JAR를 건넬 때는 `lib` 폴더를 같은 구조로 함께 보내야 한다.

3번과 4번처럼 실행에 필요한 것을 모아 결과물로 묶는 일을 **패키징**이라고 한다. 묶은 결과물이 있어야 다른 사람의 컴퓨터에서도, 나중에는 서버에서도 같은 것이 실행된다.

그리고 하지 못한 일도 있었다. `jackson-databind`가 어떤 라이브러리를 쓰는지는 실행해서 오류를 본 뒤에야 알았다. 코드가 맞게 동작하는지는 매번 눈으로 목록을 보고 판단했다. 라이브러리까지 JAR 하나로 묶지도 못했다.

> 마르코 방금 적은 일을 대신해 주는 프로그램이 있어. **빌드 도구**라고 해. 오늘은 푹 쉬고, 내일 그걸로 다시 해 보자.

## 1.5 Maven에 맡기기

### 직접 해 보기 4 Maven에 맡기기

**브랜치:** `step4-exercise`에서 시작, `step4-solution`과 비교

**목표:** 같은 블로그를 Maven 프로젝트로 옮겨, 명령 하나로 테스트와 패키징까지 한다. 1차로 JAR와 라이브러리 폴더를, 2차로 uber jar를 만든다.

**왜 필요한가:** 기능을 추가할 때마다 앞의 세 단계를 직접 반복할 수는 없다.

1. 4단계 실습 브랜치로 전환한다.

   터미널
   ```
   $ git switch step4-exercise
   ```

2. `pom.xml`의 `TODO#1`, `#2`(Jackson 의존성, `test` scope(스코프)의 JUnit)를 채운다.
3. 1차: `TODO#3`(maven-jar-plugin, maven-dependency-plugin)을 채우고 빌드한다. `target/blog-1.0.jar`와 `target/lib/`를 함께 옮기면 실행되고, JAR만 옮기면 실패하는지 확인한다.

   터미널
   ```
   $ mvn package
   ```

4. 2차: `TODO#4`대로 두 플러그인 설정을 maven-shade-plugin으로 바꾸고 3번과 같은 명령으로 다시 빌드한다. `target/blog-1.0.jar` 하나만으로 실행되는지 확인한다.

**생각해 보기:** `mvn package`는 JAR를 묶기 전에 테스트부터 실행한다. 결과물을 만드는 명령이 왜 테스트까지 실행할까?

일요일, 두원은 다시 마르코에게 영상 통화를 걸었다. 마르코가 권해 준 빌드 도구는 Maven이었다. 두원은 먼저 Maven을 설치하고, 같은 블로그를 Maven 프로젝트로 옮겼다.

## Maven 설치

Maven도 JDK와 같은 방식으로 설치한다. Maven 내려받기 페이지에서 **Binary** 압축 파일을 내려받는다(그림 1-6).

```
https://maven.apache.org/download.cgi
```

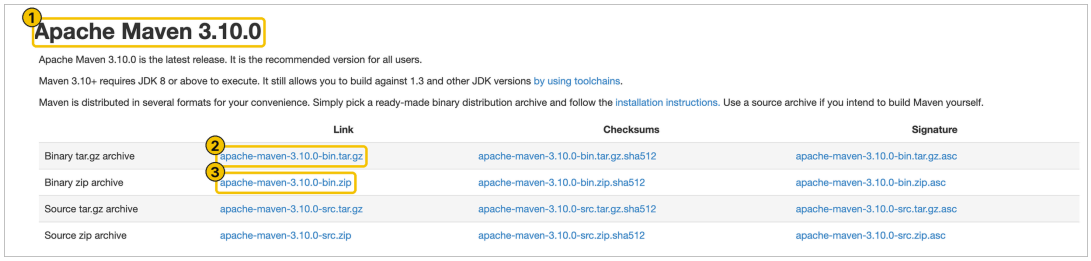



**그림 1-6** Maven 3.10.0의 파일 표. Source 줄은 소스 코드다

1. 버전
2. macOS용 `bin.tar.gz`
3. Windows용 `bin.zip`

글을 쓰는 시점의 안정 버전은 3.10.0이고, 파일은 `apache-maven-3.10.0-bin.tar.gz`(macOS, Linux)와 `apache-maven-3.10.0-bin.zip`(Windows)이다. 같은 페이지의 Maven 4는 아직 미리 보기 버전이라 고르지 않는다. Maven은 `JAVA_HOME`을 읽어 JDK를 찾으므로, 1.1절에서 설정한 값이 그대로 쓰인다.

**macOS.** 순서는 JDK 때와 같다. Linux도 같다.

1. JDK와 같은 `~/dev` 폴더에 압축을 푼다.

터미널 · macOS/Linux

```
$ cd ~/dev
$ tar xzf ~/Downloads/apache-maven-3.10.0-bin.tar.gz
```

2. `~/.zshrc`의 JDK 두 줄 다음에 다음 두 줄을 더 넣고 저장한다. `MAVEN_HOME`은 꼭 필요한 변수는 아니지만, 경로를 한곳에 적어 두면 버전을 바꿀 때 한 줄만 고치면 된다.

터미널 · macOS/Linux

```
$ export MAVEN_HOME="$HOME/dev/apache-maven-3.10.0"
$ export PATH="$MAVEN_HOME/bin:$PATH"
```

3. 지금 열린 터미널에 반영한다.

터미널 · macOS/Linux

```
$ source ~/.zshrc
```

새 터미널을 열고 확인한다.

터미널

```
$ mvn -v
Apache Maven 3.10.0 (…)
Maven home: /Users/user/dev/apache-maven-3.10.0
Java version: 21.0.x, vendor: Eclipse Adoptium, runtime: …
```

`mvn -v`는 Maven의 버전과 함께 Maven이 어느 JDK로 실행되는지도 보여 준다. `Java version` 줄이 21이 아니면 `JAVA_HOME`이 다른 JDK를 가리키고 있다. 명령을 찾을 수 없다는 메시지가 나오면 `PATH`를 확인한다.

### Maven 프로젝트의 폴더 구조

3단계에서 저장소에 들어 있던 파일을 고쳤다면 1.3절처럼 먼저 커밋해 둔다. 그래야 브랜치가 전환된다. 직접 만든 `manifest.txt`와 `out/`, `lib/`, `app.jar` 같은 파일은 Git이 추적하지 않아 전환한 뒤에도 폴더에 남는다. 이어지는 트리에는 이런 파일을 뺐다.

새로 연 터미널은 사용자 폴더에서 시작하므로, 먼저 저장소 폴더로 돌아가 4단계 브랜치로 전환한다.

터미널 · macOS/Linux Windows 733쪽

```
$ cd ~/dev/build-and-packaging
$ git switch step4-exercise
```

브랜치를 전환하자 폴더 구조부터 달라져 있었다. `src` 폴더 하나에 모여 있던 소스가 Maven이 정해 둔 자리로 옮겨져 있었다.

```
build-and-packaging/
├── pom.xml          프로젝트 설명서: 이름, 버전, 의존성
└── src/
    ├── main/
    │   ├── java/        프로그램 코드
    │   │   └── net/java21/blog/BlogMain.java
    │   └── resources/   코드가 아닌 파일(설정 파일 등)
    └── test/
        └── java/        테스트 코드
            └── net/java21/blog/BlogMainTest.java
```

**두원** `src` 아래에 `main`이랑 `test`가 따로 있고, 그 아래에 또 `java`가 있네. 왜 이렇게 깊게 들어가?

**마르코** Maven이 정해 둔 자리라서 그래. 정해진 자리에 두면 소스가 어디 있는지 안 적어도 Maven이 찾아.

**두원** 그럼 `java`는 왜 한 번 더 있어?

**마르코** `main` 아래엔 코드 말고 설정 파일도 들어가거든. 그건 `resources`에 둬.

`src/main`에 있는 것만 결과물 JAR에 들어간다. `src/test`의 코드는 빌드할 때 테스트를 실행하는 데만 쓰이므로, 테스트 코드나 테스트용 라이브러리가 운영 결과물(실제 사용자가 쓰는 환경에 내보내는 것)에 섞이지 않는다. `java` 아래의 폴더는 패키지 이름을 따른다. `net.java21.blog.BlogMain` 클래스가 `src/main/java/net/java21/blog/BlogMain.java`에 있는 이유다.

빌드 결과는 모두 `target/` 폴더에 모인다. 컴파일된 클래스는 `target/classes`에, JAR는 `target/` 바로 아래에 생긴다. 앞에서 직접 만든 `out` 폴더와 JAR를 Maven이 정해진 이름으로 대신 관리한다. `target/`은 언제든 다시 만들 수 있으므로 Git에 커밋하지 않는다(Git이 무시할 파일을 적어 두는 `.gitignore`에 적혀 있다). 프로젝트마다 자리가 같으니 처음 보는 Maven 프로젝트도 어디에 무엇이 있는지 바로 알 수 있다. Gradle(1.7절)도 같은 구조를 쓴다.

### 📖 참고 Maven 프로젝트를 새로 만드는 방법

새 프로젝트를 처음부터 만들 때도 폴더를 하나하나 만들 필요는 없다. archetype(프로젝트 뼈대를 만들어 내는 템플릿)으로 뼈대를 만드는 명령이 있다. 실습 저장소 밖의 폴더에서 실행하면 그 아래에 `my-blog` 폴더가 생긴다.

터미널 · macOS/Linux Windows 734쪽

```
$ mvn archetype:generate \
    -DgroupId=net.java21 \
    -DartifactId=my-blog \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DarchetypeVersion=1.5 \
    -DjavaCompilerVersion=21 \
    -DinteractiveMode=false
```

줄 끝의 `\`는 명령이 다음 줄로 이어진다는 표시다.

`-D`는 명령줄에서 설정 값을 넘기는 방법이다. `groupId`와 `artifactId`는 만들어질 `pom.xml`의 같은 이름 값이 되고(예제 1-3의 ①), quickstart는 클래스 하나와 테스트 하나뿐인 가장 작은 템플릿이다. 버전은 아직 개발 중이라는 표시가 붙은 `1.0-SNAPSHOT`이 된다. `src/main/resources` 폴더는 필요할 때 직접 만든다.

개발 도구로 만들 수도 있다. IntelliJ IDEA는 New Project에서 Java를 고르고 Build system을 Maven으로 정한 뒤, Advanced Settings에서 GroupId와 ArtifactId를 입력한다. Visual Studio Code는 Extension Pack for Java를 설치해 둔다. 그다음 명령 팔레트(macOS `Cmd+Shift+P`, Windows `Ctrl+Shift+P`)에서 "Java: Create Java Project"를 실행하고 Maven을 고른다.

어느 방법이든 템플릿이 적은 자바 버전이나 JUnit 버전은 이 책과 다를 수 있다. 인터넷 글에서 복사해 온 `pom.xml`도 마찬가지다. 이 절의 폴더 구조와 `pom.xml`의 네 부분(예제 1-3)을 기준으로 읽고 고친다.

## pom.xml 읽기

Maven 프로젝트의 중심은 `pom.xml`이다. POM은 Project Object Model의 줄임말로, 이 프로젝트가 무엇이고 어떻게 빌드하는지를 적은 설명서다. 어떤 라이브러리를 쓰는지, 자바 몇

버전으로 컴파일하는지처럼 두원의 머릿속과 명령에 흩어져 있던 정보를 파일 하나에 모아 적는다.

`pom.xml`은 XML로 적는다. XML은 `<이름>값</이름>`처럼 여는 태그와 닫는 태그로 값을 감싸고, 태그 안에 다시 태그를 넣어 적는 형식이다. 실습 브랜치의 `pom.xml`은 크게 네 부분으로 되어 있다(예제 1-3).

**예제 1-3** `pom.xml`: 네 부분

```
<project …>
  <modelVersion>4.0.0</modelVersion>

  <!-- ① 이 프로젝트의 coordinates -->
  <groupId>net.java21</groupId>
  <artifactId>blog</artifactId>
  <version>1.0</version>

  <!-- ② 빌드에 쓰는 설정 값 -->
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <!-- ③ 이 프로젝트가 쓰는 라이브러리 -->
  <dependencies> … </dependencies>

  <!-- ④ 빌드 과정에 일을 더하는 플러그인 -->
  <build><plugins> … </plugins></build>
</project>
```

- ① `<project>` 바로 다음의 `modelVersion`은 POM 형식의 버전으로 늘 4.0.0이다. 이어지는 `groupId`는 만든 곳(보통 사이트 주소의 도메인을 거꾸로 쓴 이름, 이 책은 `net.java21`), `artifactId`는 프로젝트 이름, `version`은 버전이다. 결과물 이름 `blog-1.0.jar`도 여기서 나온다.
- ② `maven.compiler.release`는 `javac --release 21`(자바 21 기준으로 컴파일하라는 옵션)과 같은 뜻이다. 소스 파일의 인코딩(글자를 바이트로 바꾸는 규칙, 3장에서 자세히 본다)을 UTF-8로 정해 두면 운영체제가 달라도 한글이 잘못 읽히지 않는다.
- ③ 이 프로젝트가 쓰는 라이브러리 목록이다. Maven에서는 이런 라이브러리를 **의존성**(dependency)이라고 부른다.

- ④ `compile`, `test`, `package` 같은 기본 과정에 일을 추가하고 싶을 때 플러그인(Maven에 기능을 더하는 부품)을 적는다. 이 과정 이름은 뒤의 lifecycle(라이프사이클)에서 설명한다.

예제 1-3 어디에도 소스가 `src/main/java`에 있다는 말은 없다. 정해진 자리에 두었으니 적을 필요가 없다. 설정으로 일일이 알려 주는 대신 정해진 관례를 따르게 하는 이런 방식을 **convention over configuration**(설정보다 관례)이라고 한다.

`groupId`, `artifactId`, `version` 세 값의 묶음을 **coordinates**(코디네이츠)라고 한다. 지도의 좌표가 위치 하나를 가리키듯, 세 값이 모이면 공개된 수많은 라이브러리 가운데 한 버전을 정확히 가리킨다. Jackson 3.2.3의 coordinates는 `tools.jackson.core`, `jackson-databind`, `3.2.3`이다. ①의 세 줄은 이 프로젝트의 coordinates이고, 다른 사람이 이 프로젝트를 쓸 때도 이 값으로 찾는다.

coordinates는 의존성, 경로와 헷갈리기 쉽다. coordinates는 라이브러리의 한 버전을 가리키는 식별자이고, 의존성은 "내 프로젝트가 이 라이브러리를 쓴다"는 관계다. ③의 `<dependency>`가 그 관계를 적는 자리이고, 그 안에 coordinates를 적어 어떤 라이브러리인지 밝힌다. 경로는 coordinates에서 만들어진다. Maven은 coordinates를 Maven 저장소(라이브러리를 모아 둔 곳, "의존성 적기"에서 설명한다) 안의 경로로 바꿔 파일을 찾는다. `groupId`의 점은 폴더 구분이 되고, 그 아래로 `artifactId`와 `version` 폴더가 이어진다.

```
tools.jackson.core : jackson-databind : 3.2.3
  → tools/jackson/core/jackson-databind/3.2.3/jackson-databind-3.2.3.jar
```

TODO는 ③과 ④에 있다. 실습 브랜치의 `pom.xml`에서 ③은 이렇게 비어 있다.

**실습 브랜치의 모습** `pom.xml`: 비어 있는 ③ 의존성 목록

```
<dependencies>
  <!-- TODO#1 Jackson 3.2.3을 의존성으로 적는다
            (groupId tools.jackson.core, artifactId jackson-databind).
            (1.5절 "의존성 적기", 예제 1-4)
            <dependency> 안에 coordinates 세 개를 적는다. -->
  <!-- TODO#2 테스트에만 쓰는 JUnit(org.junit.jupiter:junit-jupiter:6.1.3)을
            <scope>test</scope>로 적는다. (1.5절, 예제 1-4)
            채우면 src/test의 BlogMainTest(예제 1-5)가 컴파일되고
            mvn package가 테스트를 돌린다. -->
</dependencies>
```

`TODO#1`에는 블로그가 쓰는 Jackson을, `TODO#2`에는 테스트에만 쓰는 JUnit을 `<dependency>`로 하나씩 적는다. `<dependency>` 하나에는 coordinates 세 줄이 들어가고, JUnit에는 언제 쓰는지를 정하는 `<scope>`가 한 줄 더 붙는다. 두 개를 채우면 실습 브랜치에 이미 들어 있는 테스트(예제 1-5)가 컴파일되고, `mvn package`가 그 테스트를 실행한다.

### 의존성 적기

두원이 처음 막힌 곳은 Jackson의 coordinates였다. 우선 `groupId`가 `jackson`인지 `tools.jackson`인지부터 알 수 없었다.

> **마르코** 어제 물었지, 그 JAR를 어디서 구했냐고. 자바 라이브러리는 대부분 Maven Central이라는 공개 저장소에 올라와 있어. 그 저장소를 쉽게 검색할 수 있는 사이트가 mvnrepository.com이야. 거기서 jackson-databind를 한번 찾아볼래?

검색 결과에는 jackson-databind가 두 줄 나온다. groupId가 `com.fasterxml.jackson.core`인 쪽은 2.x 버전이고, `tools.jackson.core`인 쪽은 3.x 버전이다. Jackson은 3.0부터 groupId와 패키지 이름을 `tools.jackson`으로 바꿨다. 1.3절의 `import tools.jackson…`도 그래서 이 이름이다.

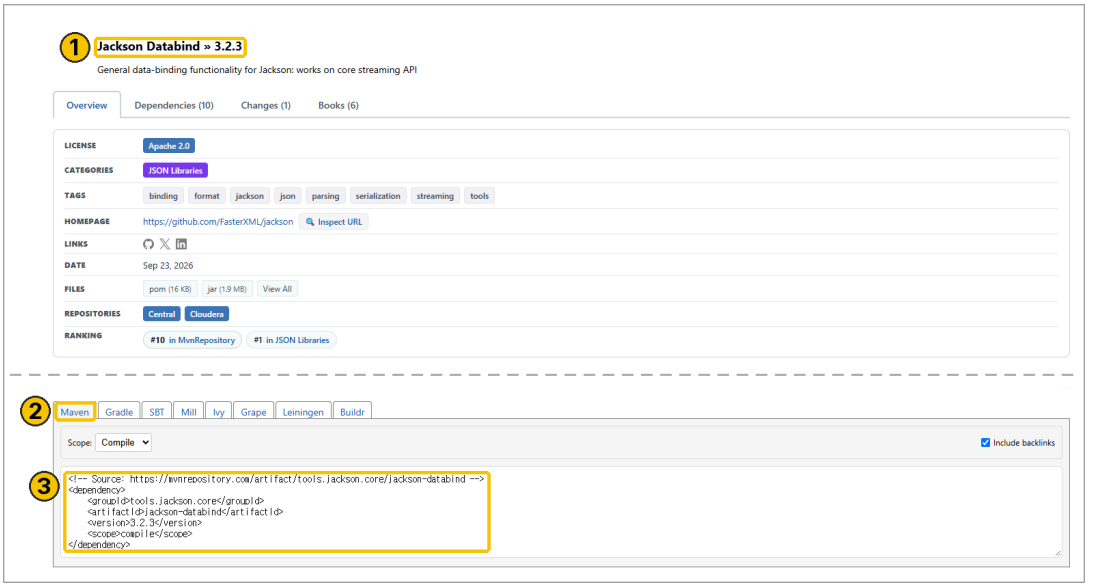



**그림 1-7** mvnrepository.com의 jackson-databind 3.2.3 페이지

1. 버전
2. Maven 탭
3. `pom.xml`에 붙여 넣을 `<dependency>` 코드

두원은 `tools.jackson.core` 쪽에서 글을 쓰는 시점의 최신 버전인 3.2.3을 눌렀다. 버전마다 배포된 날짜와 알려진 보안 취약점이 함께 보이고, 빌드 도구별로 붙여 넣을 코드가 나온다. 두원은 그림 1-7의 Maven 탭 코드를 복사해 `pom.xml`에 넣었다.

같은 페이지의 Dependencies 탭에는 jackson-databind 자신이 쓰는 라이브러리 목록이 나온다. 1.3절에서 마르코가 챙겨 준 나머지 두 JAR가 여기에 있다. 이 목록은 1차 패키징에서 Maven으로 직접 확인한다.

검색으로 찾은 글에 적힌 coordinates도 이곳에서 한번 확인하는 편이 좋다. 몇 년 전 글이면 버전도 몇 년 전 것이고, 버전 번호가 잘못 적혔을 수도 있다. JAR 파일 자체는 mvnrepository.com이 아니라 Maven 중앙 저장소(Maven Central)에 있다.

`TODO#1`과 `TODO#2`를 직접 채운 뒤 예제 1-4와 비교해 보자.

두원이 채운 두 의존성은 다음과 같다.

**예제 1-4** `pom.xml`: Jackson과 JUnit 의존성

```
<dependency>
  <groupId>tools.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>3.2.3</version>
</dependency>
<!-- 테스트에만 쓴다. 만든 JAR와 target/lib에는 들어가지 않는다 -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>6.1.3</version>
  <scope>test</scope>
</dependency>
```

Maven은 예제 1-4의 coordinates를 "pom.xml 읽기"에서 본 경로로 바꿔 Maven 중앙 저장소에서 JAR를 내려받는다. 내려받은 JAR는 사용자 폴더 아래의 `.m2/repository`(macOS와 Linux는 `~/.m2/repository`, Windows는 `C:\Users\사용자이름\.m2\repository`)에 저장된다. 이 폴더를 **로컬 저장소**(local repository)라고 한다.

로컬 저장소는 프로젝트마다 따로 있지 않고, 내 컴퓨터의 모든 Maven 프로젝트가 공유한다(그림 1-8). 다른 프로젝트에서 같은 Jackson 3.2.3을 쓰면 다시 내려받지 않고 로컬 저장소의 JAR를 그대로 쓰므로, 두 번째 빌드부터는 빨라진다.

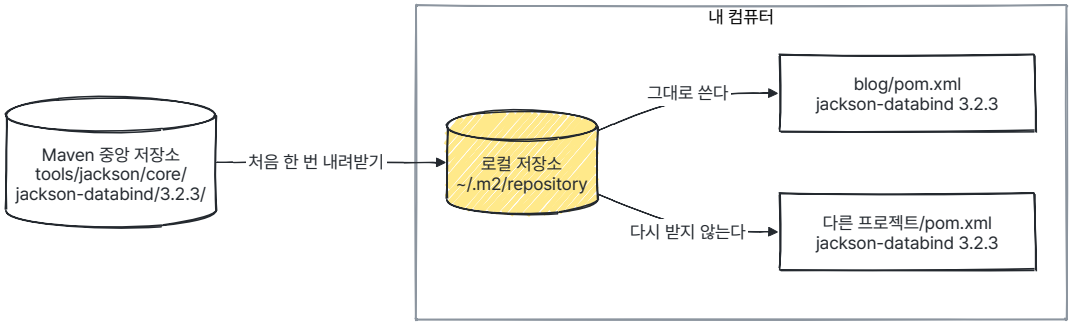



**그림 1-8** 중앙 저장소에서 한 번 내려받은 JAR를 로컬 저장소에 두고, 내 컴퓨터의 Maven 프로젝트가 함께 쓴다

### scope: 언제 필요한가

JUnit을 두 번째 의존성으로 추가한 것은 어제 매번 목록을 눈으로 확인하던 일을 코드로 남기기 위해서다. 두 의존성은 `<scope>`가 다르다. scope는 그 라이브러리가 언제 필요한지, 곧 컴파일할 때 쓰는지, 테스트할 때 쓰는지, 실행할 때도 있어야 하는지를 정한다. scope에 따라 Maven이 그 라이브러리를 언제 classpath에 넣을지, 결과물에 담을지가 달라진다.

Jackson처럼 scope를 적지 않으면 기본값인 `compile`이 된다. `compile`은 가장 넓은 scope로, 컴파일할 때도, 테스트할 때도, 실행할 때도 classpath에 들어간다. 1.3절에서 `javac`와 `java` 양쪽에 Jackson을 classpath로 지정해야 했던 것이 바로 `compile` scope의 뜻이다. 실행할 때도 필요하므로 uber jar에도 담긴다. 프로그램 코드가 직접 쓰는 라이브러리는 대부분 `compile`이다.

JUnit에는 `<scope>test</scope>`를 적었다. `src/test/java`를 컴파일하고 테스트를 실행할 때만 classpath에 들어가고, `src/main/java`에서는 쓸 수 없으며 결과물 JAR에도 담기지 않는다. 테스트 도구가 운영 결과물에 섞이지 않게 막는 scope다. scope는 이 밖에도 몇 가지가 더 있다. 다음 장에서 Lombok을 쓸 때 "JAR에 넣지 않는" `provided`가 나오고, 그 뒤로도 필요할 때마다 하나씩 다룬다.

테스트는 맛보기로 하나만 두었다. 이 테스트는 본문에 쉼표(1.2절에서 두원이 쉼표로 나누는 파일을 포기한 까닭)와 큰따옴표(JSON에서 escape해야 하는 문자)가 들어 있어도 JSON으로 바꿨다가 되돌리면 그대로인지 확인한다(예제 1-5). 테스트 파일은 실습 브랜치에 이미 들어 있으니 읽기만 해도 된다. 예제에는 메서드 하나만 실었다.

**예제 1-5** `src/test/java/net/java21/blog/BlogMainTest.java`: 쉼표와 큰따옴표가 든 글의 JSON 왕복 테스트

```
@Test
@DisplayName("쉼표와 따옴표가 든 글도 JSON을 거쳐 그대로 돌아온다")
void roundTripsCommasAndQuotesThroughJson() {
    ObjectMapper mapper = JsonMapper.builder().build();
    List<BlogMain.Post> posts =
            List.of(new BlogMain.Post(1, "첫 글", "안녕, \"블로그\""));

    // 글 목록 → JSON 문자열 → 글 목록으로 한 바퀴 돌린다
    String json = mapper.writeValueAsString(posts);
```

```
        assertEquals(posts, mapper.readValue(json,
                new TypeReference<List<BlogMain.Post>>() { }));
    }
```

`@Test`가 붙은 메서드 하나하나가 JUnit이 실행하는 테스트다. 테스트 클래스는 `BlogMain`과 같은 패키지에 있으므로 `public`이 없는 `BlogMain.Post`도 쓸 수 있다. `mvn package`의 `test` 단계가 이런 메서드를 찾아 차례로 실행한다. `assertEquals(기댓값, 실제값)`은 두 값이 같은지 확인하고, 다르면 그 테스트를 실패로 기록한다. record에는 필드 값이 같으면 같다고 보는 `equals`를 자바가 만들어 주므로, 두 리스트를 `assertEquals`로 비교할 수 있다. 이렇게 "이 값은 이래야 한다"고 적는 문장을 **단언**(assertion)이라고 하며, 3장에서 직접 써 본다.

### 참고 annotation

`@Test`처럼 `@`로 시작하는 표시를 **annotation**(애너테이션)이라고 한다. 클래스, 메서드, 필드 앞에 붙여 그 코드에 정보를 덧붙이는 표시다. annotation은 그 자체로는 아무 일도 하지 않는다. 읽는 쪽이 그 표시를 보고 동작한다. `@Test`는 JUnit이 읽어 테스트로 실행할 메서드를 고르고, 메서드 앞에 붙이는 `@Override`는 컴파일러가 읽어 부모 클래스나 인터페이스의 메서드를 정말 다시 정의했는지 확인한다. 프레임워크도 annotation을 읽어 할 일을 정한다. 5장에서는 annotation을 직접 만들고 읽어 본다.

눈으로 한 번 확인하는 편이 더 빨라 보여도, 그 확인은 한 번으로 끝나지 않는다. 코드를 고칠 때마다 전에 되던 것이 여전히 되는지 다시 봐야 하고, 고친 곳과 멀리 떨어진 기능이 망가지는 일도 흔하다. 이렇게 전에 동작하던 기능이 코드를 고친 뒤 동작하지 않게 되는 것을 **회귀**(regression)라고 한다. 테스트를 코드로 남겨 두면 `mvn package`를 실행할 때마다 지금까지 적어 둔 확인을 모두 다시 거친다. 테스트는 회귀를 막는 안전망이고, 테스트 이름과 단언은 "이 코드가 무엇을 해야 하는가"를 적은 문서이기도 하다. 이 책의 실습이 단계마다 테스트를 함께 건네는 것도 그래서다.

## 1.6 Maven으로 묶기

프로젝트가 테스트까지 갖추었으니 이제 묶을 차례다. 두원은 두 번에 나누어 묶었다. 1차는 1.4절처럼 JAR 옆에 라이브러리 폴더를 두는 방식이고, 2차는 라이브러리까지 JAR 하나에 담는 방식이다.

### 1차 패키징: JAR와 lib 폴더

먼저 1.4절에서 직접 한 일을 그대로 Maven에 맡겼다. 실습 브랜치의 `pom.xml`에서 ④의 플러그인 자리는 이렇게 되어 있다.

**실습 브랜치의 모습** `pom.xml`: ④ 플러그인 자리

```
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.6.0</version>
  </plugin>
  <!-- TODO#3 (1차) maven-jar-plugin 3.5.1에 manifest 설정
               (mainClass net.java21.blog.BlogMain, addClasspath true,
               classpathPrefix lib/)을 적고, maven-dependency-plugin
               3.11.0의 copy-dependencies를 package 단계에 붙여
               runtime 의존성을 target/lib에 모은다.
               (1.6절, 예제 1-6) 1.4절에서 직접 쓴 manifest.txt의
               Main-Class, Class-Path와 비교해 본다. -->
  <!-- TODO#4 (2차) TODO#3의 manifest 설정과
               maven-dependency-plugin을 지우고, maven-shade-plugin
               3.6.2를 package 단계에 붙여 uber jar를 만든다.
               Main-Class는 net.java21.blog.BlogMain.
               (1.6절, 예제 1-7) Main-Class는
               ManifestResourceTransformer로 적는다. -->
</plugins>
```

처음 나오는 maven-surefire-plugin은 테스트를 실행하는 플러그인으로, 버전만 정해 두었다. 1차 패키징은 `TODO#3`에서 두 플러그인으로 한다. maven-jar-plugin은 JAR를 만들고 manifest를 쓴다. maven-dependency-plugin은 의존성 JAR를 한 폴더에 모은다. 플러그인이 하는 일 하나하나에는 이름이 붙어 있고, 이 이름을 **goal**(골)이라고 한다. maven-

dependency-plugin의 `copy-dependencies` goal을 `package` 단계에 연결하면 실행에 필요한 JAR가 `target/lib`에 복사된다. 무엇을 적을지는 1.4절에서 직접 쓴 `manifest.txt`의 `Main-Class`, `Class-Path`와 비교해 보면 보인다. 태그 이름은 플러그인 문서의 예를 옮겨 써도 된다. 직접 정할 것은 `Main-Class`에 들어갈 클래스 이름과 `lib/` 접두어 둘이다.

`TODO#3`을 직접 채운 뒤 예제 1-6과 비교해 보자.

두원은 ④의 `TODO#3` 자리에 두 플러그인을 다음과 같이 적었다.

**예제 1-6** `pom.xml`: **1차 패키징 설정**

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.5.1</version>
  <!-- manifest에 Main-Class와 Class-Path를 적는다
       (1.4절의 manifest.txt) -->
  <configuration>
    <archive>
      <manifest>
        <mainClass>net.java21.blog.BlogMain</mainClass>
        <addClasspath>true</addClasspath>
        <classpathPrefix>lib/</classpathPrefix>
      </manifest>
    </archive>
  </configuration>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-dependency-plugin</artifactId>
  <version>3.11.0</version>
  <!-- package 단계에서 실행에 필요한 JAR를 target/lib에 복사한다 -->
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>copy-dependencies</goal>
      </goals>
      <configuration>
        <outputDirectory>${project.build.directory}/lib</outputDirectory>
        <includeScope>runtime</includeScope>
```

```
      </configuration>
    </execution>
  </executions>
</plugin>
```

`step4-solution`의 `pom.xml`은 2차 패키징까지 마친 설정이라, 예제 1-6의 1차 설정은 같은 폴더에 `pom-first.xml`이라는 이름으로 따로 넣어 두었다. 예제 1-6의 항목이 하는 일을 1.4절에서 직접 한 일과 짝지으면 표 1-2와 같다.

**표 1-2** 1차 패키징 설정의 항목과 1.4절에서 직접 한 일

| 항목 | 하는 일 | 1.4절에서 직접 한 일 |
|---|---|---|
| `mainClass` | manifest에 `Main-Class`를 적는다 | `manifest.txt`의 첫 줄 |
| `addClasspath` | 의존성 JAR 이름을 모아 manifest에 `Class-Path`를 적는다 | `manifest.txt`의 둘째 줄 |
| `classpathPrefix` | `Class-Path`의 JAR 이름 앞에 `lib/`를 붙인다 | `lib/jackson-databind-3.2.3.jar`의 `lib/` |
| `copy-dependencies` | maven-dependency-plugin의 goal. 의존성 JAR를 `target/lib`에 복사한다 | 마르코가 챙겨 준 JAR를 `lib/`에 둔 것 |
| `includeScope` | 실행에 필요한 것만 복사하고 테스트용은 뺀다. `runtime`은 컴파일에는 필요 없고 실행에만 필요한 scope인데, 이 설정은 `compile` scope도 함께 고른다 | 없음 |

`${project.build.directory}`는 `target/` 폴더를 가리키고, `<phase>package</phase>`는 이 goal을 `package` 단계에 연결하라는 뜻이다. 단계는 뒤의 lifecycle에서 다룬다. 처음 실행할 때는 플러그인과 라이브러리를 내려받아야 하므로 인터넷에 연결되어 있어야 한다.

터미널

```
$ mvn package
… (생략: 플러그인과 라이브러리 내려받기)
[INFO] --- compiler:3.16.0:compile (default-compile) @ blog ---
… (생략: 테스트 컴파일)
[INFO] --- surefire:3.6.0:test (default-test) @ blog ---
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.5.1:jar (default-jar) @ blog ---
[INFO] --- dependency:3.11.0:copy-dependencies (default) @ blog ---
[INFO] BUILD SUCCESS
```

`package` 한 단어로 컴파일, 테스트 컴파일, 테스트 실행, JAR 만들기, 의존성 모으기가 차례로 실행되었다. 두원은 `target/`에 무엇이 생겼는지 보았다.

터미널 · macOS/Linux Windows 734쪽

```
$ ls target/lib
$ unzip -p target/blog-1.0.jar META-INF/MANIFEST.MF
```

실행 결과

```
jackson-annotations-2.22.jar
jackson-core-3.2.3.jar
jackson-databind-3.2.3.jar
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.5.1
…
Class-Path: lib/jackson-databind-3.2.3.jar lib/jackson-annotations-2.22.
 jar lib/jackson-core-3.2.3.jar
Main-Class: net.java21.blog.BlogMain
```

1.4절에서 직접 만든 것과 같은 모양이다. 긴 `Class-Path` 줄이 72바이트에서 나뉜 것까지 같다. `blog-1.0.jar`에는 두원의 클래스와 manifest, 그리고 Maven이 넣는 `pom.xml` 사본과 `pom.properties`(coordinates를 적은 작은 파일)만 있다. 크기는 6KB 남짓으로, 이 사본 때문에 1.4절의 JAR보다 조금 크다.

`pom.xml`에는 `jackson-databind` 하나만 적었는데, `lib/`와 `Class-Path`에는 JAR 세 개가 모두 들어 있다. 왜 그런지는 Maven으로 직접 확인할 수 있다.

터미널

```
$ mvn dependency:tree
[INFO] net.java21:blog:jar:1.0
[INFO] +- tools.jackson.core:jackson-databind:jar:3.2.3:compile
[INFO] ¦  +- com.fasterxml.jackson.core:jackson-annotations:jar:2.22:compile
[INFO] ¦  \- tools.jackson.core:jackson-core:jar:3.2.3:compile
[INFO] \- org.junit.jupiter:junit-jupiter:jar:6.1.3:test
… (생략: JUnit이 쓰는 라이브러리 여덟 개)
```

jackson-databind는 자기 `pom.xml`에 `jackson-annotations`와 `jackson-core`를 실행에도 필요한 `compile` scope로 적어 두었다. 그래서 Maven은 jackson-databind를 내려받을 때 이 둘도 함께 내려받는다. 이렇게 내가 적은 라이브러리가 쓰는 라이브러리를 **전이 의존성**이라고 한다. 1.3절에서 찾지 못한 `TreeCodec`이 든 `jackson-core`가 바로 전이 의존성이다. 마르코가 물었던 "그 라이브러리들이 또 뭘 쓰는지는 어떻게 다 알아낼 거야?"의 답도 여기에 있다.

`jackson-annotations`만 groupId가 예전 이름(`com.fasterxml.jackson.core`)이고 버전도 다르다. Jackson 2.x와 3.x가 이 annotation 모음을 함께 쓰기 때문이다. JUnit 쪽에는 `test`가 붙어 있으니 `target/lib`에는 들어가지 않는다.

`target/` 안에서는 `lib/`가 JAR 바로 옆에 있으니 그대로 실행된다. 다른 컴퓨터로 보낼 때는 1.4절과 똑같이 `lib/` 폴더를 함께 보내야 한다. 그때 만든 `friend` 폴더가 남아 있으니 이번에는 `friend2` 폴더에 복사해 보았다.

터미널 · macOS/Linux — Windows 734쪽

```
$ mkdir friend2 && cp -r target/blog-1.0.jar target/lib friend2/
$ cd friend2
$ java -jar blog-1.0.jar write "첫 글" "안녕, 블로그"
$ java -jar blog-1.0.jar list
$ cd ..
```

실행 결과

```
1번 글을 저장했습니다.
1. 첫 글
```

JAR만 다른 폴더에 옮겨 실행하면 1.4절의 `friend` 폴더에서 본 것과 같은 `NoClassDefFoundError`가 발생한다(한국어 Windows에서는 1.3절에서 본 한국어 메시지로 나온다). manifest의 `Class-Path`가 가리키는 `lib/`가 옆에 없기 때문이다.

두원 manifest를 손으로 쓸 일은 없어졌네. 근데 `lib` 폴더를 같이 보내야 하는 건 그대로야.

마르코 그래서 2차로 하나 더 해 볼 거야. 어제 말한 uber jar.

### 2차 패키징: uber jar

**uber jar**는 라이브러리 JAR를 풀어 그 안의 클래스까지 JAR 하나에 모두 담는 방식이다. uber는 흔히 독일어 über("넘어서")에서 온 말로 풀이하며, 내 코드와 그 코드가 쓰는 라이브러리를 모두 아우른 JAR라는 뜻이다. 내용물이 많아 크다는 뜻으로 fat jar(팻 자)라고도 부른다. 라이브러리가 안에 들어 있으니 `Class-Path`도 `lib/` 폴더도 필요 없다.

직접 하려면 라이브러리 JAR를 하나하나 풀어 두원의 클래스와 한 폴더에 모으고 다시 묶어야 한다. 두원은 이 일을 maven-shade-plugin에 맡겼다. 채울 곳은 1차 패키징에서 본 실습 브랜치 ④의 `TODO#4`다. 1차의 manifest 설정(`<configuration>` 전체)과 maven-dependency-plugin을 지우고, maven-shade-plugin 3.6.2를 `package` 단계에 연결한다. `Main-Class`는 이제 shade 쪽에 `ManifestResourceTransformer`로 적는다.

`TODO#4`를 직접 채운 뒤 예제 1-7과 비교해 보자.

두원이 적은 shade 플러그인 설정은 다음과 같다.

**예제 1-7** `pom.xml`: 2차 패키징 설정

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.5.1</version>
  <!-- 설정 없이 내 클래스만 담은 JAR를 만든다.
       Main-Class는 shade가 적는다 -->
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.6.2</version>
  <!-- 내 JAR와 의존성 JAR를 풀어 하나의 JAR로 다시 묶는다(uber jar) -->
  <configuration>
    <createDependencyReducedPom>false</createDependencyReducedPom>
    <transformers>
```

```
        <transformer implementation=
          "org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
          <mainClass>net.java21.blog.BlogMain</mainClass>
        </transformer>
      </transformers>
    </configuration>
    <executions>
      <execution>
        <phase>package</phase>
        <goals>
          <goal>shade</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
```

---

maven-jar-plugin은 설정 없이 버전만 남겨, 평소처럼 두원의 클래스만 담은 JAR를 만들게 했다. `package` 단계에 연결한 `shade` goal이 그 JAR와 의존성 JAR를 풀어 하나로 다시 묶고, `ManifestResourceTransformer`가 다시 묶은 JAR의 manifest에 `Main-Class`를 적는다. `createDependencyReducedPom`을 `false`로 두면 의존성을 뺀 `dependency-reduced-pom.xml`을 따로 만들지 않는다. 이 JAR는 다른 프로젝트의 의존성으로 배포하지 않으므로 필요 없다.

이번에는 `clean`을 함께 적어, 1차에서 만든 `target/`을 지우고 처음부터 다시 빌드했다.

터미널

```
$ mvn clean package
… (생략: 컴파일, 테스트, JAR 만들기)
[INFO] --- shade:3.6.2:shade (default) @ blog ---
… (생략: module-info.class에 대한 경고 세 줄)
[WARNING] blog-1.0.jar, jackson-annotations-2.22.jar, jackson-core-3.2.3.jar,
          jackson-databind-3.2.3.jar define 1 overlapping resource:
[WARNING]   - META-INF/MANIFEST.MF
[WARNING] jackson-annotations-2.22.jar, jackson-core-3.2.3.jar,
          jackson-databind-3.2.3.jar define 2 overlapping resources:
[WARNING]   - META-INF/LICENSE
[WARNING]   - META-INF/NOTICE
… (생략: 겹치는 파일에 대한 일반 안내)
[INFO] BUILD SUCCESS
```

경고가 몇 줄 나와도 마지막 줄이 `BUILD SUCCESS`이면 JAR는 만들어졌다. 생략한 `module-info.class` 경고는 JAR를 하나로 합치면서 JAR마다 선언된 모듈(자바 9부터 생긴,

패키지를 묶는 단위)의 경계가 사라진다는 뜻이다. 이 책의 블로그는 모듈을 쓰지 않으므로 실행에는 지장이 없다. 겹치는 파일 경고가 무슨 뜻인지는 이 절 끝에서 본다. 결과물의 크기는 `target` 폴더의 목록에서 확인한다.

터미널

```
$ ls -l target
```

실행할 결과물은 2.6MB쯤 되는 `target/blog-1.0.jar`다. 함께 보이는 `original-blog-1.0.jar`는 shade가 다시 묶기 전의 JAR다. 1차의 JAR가 6KB 남짓이었으니, 늘어난 만큼이 Jackson JAR 세 개의 몫이다(그림 1-9). 두원은 이 JAR 하나만 빈 폴더 `fat`에 복사해 실행했다.

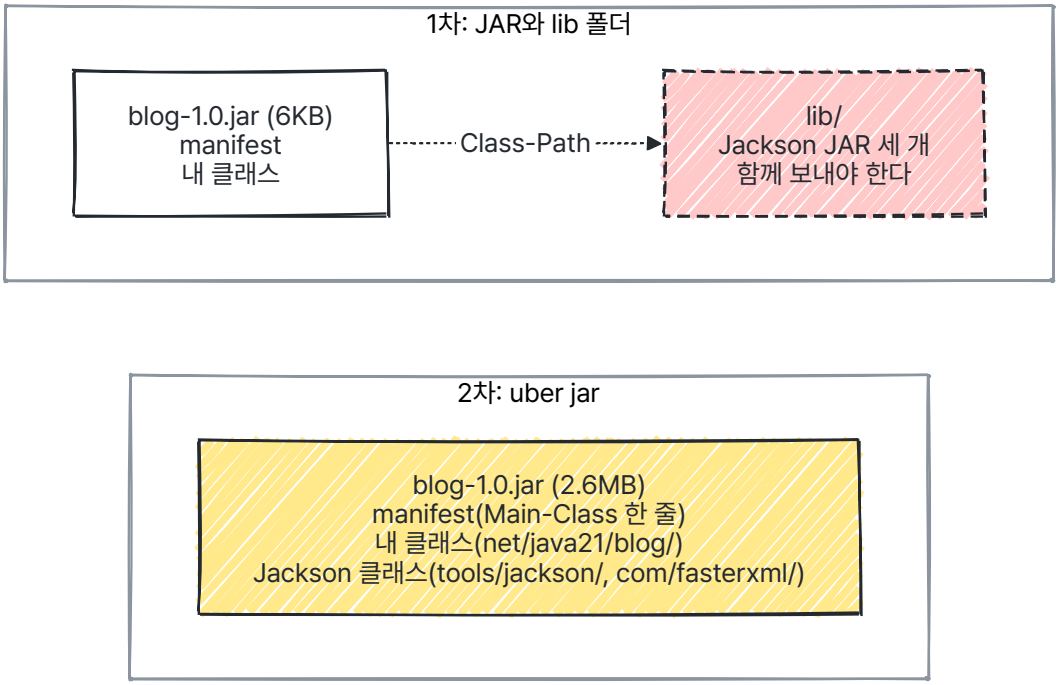



**그림 1-9** 1차 결과물은 JAR 옆에 `lib` 폴더가 있어야 실행되고, 2차 결과물(uber jar)은 Jackson까지 안에 담아 혼자 실행된다

```
터미널 · macOS/Linux                                    Windows 735쪽
$ mkdir fat && cp target/blog-1.0.jar fat/
$ cd fat
$ java -jar blog-1.0.jar write "첫 글" "안녕, 블로그"
$ java -jar blog-1.0.jar list
$ cd ..
```

1차의 `friend2`와 같은 결과가 출력된다. 자바만 설치되어 있으면 어디서든 실행된다. 다만 uber jar에는 따져 볼 점이 세 가지 있다.

- **크기.** 라이브러리가 늘수록 JAR가 커진다. 코드 한 줄을 고쳐도 라이브러리까지 통째로 다시 보낸다.
- **같은 이름의 파일.** JAR마다 `META-INF/MANIFEST.MF`가 있고, 라이브러리가 여럿이면 `META-INF/LICENSE`처럼 이름이 같은 파일끼리 충돌한다. JAR를 풀어 한 폴더에 합치면 나중에 푼 파일이 먼저 푼 파일을 아무 경고 없이 덮어쓰고, 어떤 기능은 제대로 동작하지 않게 된다. shade는 이 충돌을 경고로 알려 준다. `mvn clean package` 출력의 `overlapping resource` 경고가 바로 이 충돌이다.
- **라이선스.** uber jar는 다른 사람의 코드를 내 결과물에 담아 배포한다. 라이브러리마다 라이선스가 요구하는 고지를 함께 챙겨야 한다. 앞의 경고를 보면 Jackson JAR 세 개의 `META-INF/NOTICE`가 겹쳐 하나만 남았다. 세 파일의 내용은 서로 다르므로, 나머지 둘의 고지는 uber jar에서 빠졌다.

### lifecycle

Maven은 빌드를 정해진 순서의 단계(phase)로 나눠 두었다. 이 순서를 **lifecycle**이라고 한다. 알아 둘 단계는 표 1-3의 다섯 개다(실제로는 그 사이에 단계가 몇 개 더 있다).

**표 1-3** 알아 둘 lifecycle 단계

| 단계 | 하는 일 |
|---|---|
| `compile` | `src/main/java`를 컴파일해 `target/classes`에 둔다 |
| `test` | `src/test/java`를 컴파일하고 테스트를 실행한다 |
| `package` | 컴파일한 클래스를 JAR로 묶는다 |

| 단계 | 하는 일 |
|---|---|
| install | 만든 JAR를 로컬 저장소(~/.m2/repository)에 넣는다. 그러면 내 컴퓨터의 다른 프로젝트도 Jackson처럼 coordinates를 적어 이 JAR를 의존성으로 쓸 수 있다 |
| clean | target/ 폴더를 지운다. 엄밀히는 앞의 네 단계와 별도인 clean lifecycle에 속한다 |

어떤 단계를 실행하면 그 앞의 단계가 모두 먼저 실행된다. 그래서 `mvn package`는 테스트가 하나라도 실패하면 `package`까지 가지 않는다. 테스트를 통과하지 못한 코드로 결과물을 만들지 못하게 하려는 장치다. `clean`은 이 순서에 속하지 않으므로, 지난 결과를 지우고 처음부터 다시 만들 때는 `mvn clean package`처럼 함께 적는다.

단계는 순서만 정하고, 실제 일은 각 단계에 연결된 플러그인의 goal이 한다. 로그의 `compiler:3.16.0:compile`, `surefire:3.6.0:test` 같은 줄이 그 goal이다(`플러그인:버전:goal`). 1차의 `copy-dependencies`와 2차의 `shade`도 `package` 단계에 연결되어 JAR가 만들어진 뒤에 실행되었다. `compiler`처럼 `pom.xml`에 적지 않은 플러그인의 버전은 Maven이 정한다. `compiler:3.16.0`은 앞에서 설치한 Maven 3.10.0이 정한 버전이므로, 다른 버전의 Maven에서는 다르게 나올 수 있다. 단계의 이름과 순서는 어느 Maven 프로젝트에서든 같아서, 처음 보는 프로젝트도 `mvn package` 하나로 빌드할 수 있다.

### 직접 한 일과 Maven이 한 일

두원은 어제 직접 한 일과 오늘 Maven이 대신한 일을 표 1-4처럼 나란히 놓고 비교해 보았다.

**표 1-4** 직접 한 일과 Maven이 하는 일

| 직접 한 일 | Maven이 하는 일 |
|---|---|
| 라이브러리 JAR를 구해 lib/에 둔다 | coordinates 세 줄. 전이 의존성까지 내려받는다 |
| 컴파일과 실행 때 -cp를 지정한다 | Maven이 단계마다 classpath를 구성한다 |
| 결과를 눈으로 확인한다 | test 단계가 테스트를 실행하고, 실패하면 멈춘다 |
| manifest를 쓴다 | maven-jar-plugin의 mainClass, addClasspath |
| lib 폴더를 JAR 옆에 맞춰 둔다 | maven-dependency-plugin이 target/lib에 모은다 |
| (하지 못했다) JAR 하나로 묶기 | maven-shade-plugin. 겹치는 파일은 경고로 알린다 |

## 1.7 또 하나의 빌드 도구, Gradle

이 절은 Maven과 비교하며 Gradle을 살펴보는 곁가지다. 이 책의 실습은 모두 Maven으로 진행하므로, 처음 읽을 때는 건너뛰고 "이 장에서 배운 것"으로 가도 된다.

> **두원** Maven 말고 다른 것도 있어? 자바 예제를 찾아보면 Gradle이라는 것도 자주 보이던데.
>
> **마르코** 응, Gradle 쓰는 프로젝트도 많아. 빌드 설정을 XML 대신 스크립트로 쓰는 도구야. 하는 일은 어제 네가 적은 목록이랑 똑같아.

같은 블로그를 Gradle로 적으면 `build.gradle`이 된다(예제 1-8).

**예제 1-8** `build.gradle`: 같은 블로그의 Gradle 설정

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

java {
    toolchain { languageVersion = JavaLanguageVersion.of(21) }
}

dependencies {
    implementation 'tools.jackson.core:jackson-databind:3.2.3'
    testImplementation 'org.junit.jupiter:junit-jupiter:6.1.3'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}

application {
    mainClass = 'net.java21.blog.BlogMain'
}
```

coordinates 세 줄이 `그룹:이름:버전` 한 줄이 되었다. Maven의 scope와 같은 일을 Gradle에서는 configuration이 한다. Maven의 기본 scope(`compile`)는 `implementation`, `test`는 `testImplementation`에 해당한다. 버전 없이 적은 `junit-platform-launcher`는 Gradle이 JUnit 테스트를 실행할 때 쓰는 부품으로, 버전은 JUnit 쪽에 맞춰진다. 내려받는 곳도 같은 Maven 중앙 저장소(`mavenCentral()`)다. 전이 의존성도 똑같이 함께 내려받는다.

나머지 세 설정은 이렇다.

- `application` 플러그인은 `gradle run`으로 프로그램을 실행하게 해 주고, 시작할 클래스는 끝의 `application` 블록의 `mainClass`에 적는다. 이 값은 `gradle run`과 Gradle이 만드는 실행 스크립트가 쓰는 것이라 JAR의 manifest에는 들어가지 않는다.
- `toolchain`은 이 프로젝트를 자바 21로 컴파일하고 실행하라고 정하는 자리다. Maven의 `maven.compiler.release`와 비슷한 일을 한다.
- `useJUnitPlatform()`은 테스트를 JUnit으로 실행하라는 설정이다.

Gradle은 설치하지 않았으므로 여기서는 결과만 보자. 의존성 목록을 출력하는 명령과 그 결과는 다음과 같다.

터미널

```
$ gradle dependencies --configuration runtimeClasspath
runtimeClasspath - Runtime classpath of source set 'main'.
\--- tools.jackson.core:jackson-databind:3.2.3
     +--- com.fasterxml.jackson.core:jackson-annotations:2.22
     +--- tools.jackson.core:jackson-core:3.2.3
… (생략: 세 JAR의 버전을 맞추는 jackson-bom 줄과 기호 설명)
```

두 도구를 비교하면 표 1-5와 같다.

**표 1-5** Maven과 Gradle

| | Maven | Gradle |
|---|---|---|
| 설정 파일 | `pom.xml` (XML) | `build.gradle` (스크립트) |
| 결과물 만들기 | `mvn package` | `gradle build` |
| 결과물 자리 | `target/` | `build/libs/`(Jackson이 없는 JAR) |
| 성격 | 정해진 lifecycle과 구조를 따른다 | 짧고, 원하는 대로 바꾸기 쉽다 |

결과물에도 차이가 있다. `gradle build`가 `build/libs/`에 만드는 JAR에는 Jackson이 들어 있지 않고, manifest에 `Main-Class`도 적히지 않아 `java -jar`로 바로 실행되지 않는다. 이 장의 Maven 설정은 shade 플러그인으로 uber jar까지 만들었지만, Gradle에서 같은 결과를 얻으려면 uber jar를 만드는 플러그인을 따로 추가해야 한다. Gradle은 자유로운 만큼 프로젝트마다 모양이 달라지기도 한다. 어느 쪽이 낫다기보다, 팀이 이미 쓰는 쪽을 따르면 된다. 이 책은 Maven을 쓴다.

프런트엔드(브라우저에서 실행되는 쪽)도 다르지 않다. JavaScript에도 의존성을 내려받는 도구와 코드를 변환해 묶는 빌드 도구가 있어, 브라우저가 받을 파일 묶음을 만든다. 도구 이름은 몇 년마다 바뀌어도 "의존성을 내려받고, 컴파일하고, 검사하고, 결과물 하나로 묶는다"는 일은 그대로다. 그 일을 한 번 직접 해 본 사람은 새 도구가 나와도 무엇을 대신해 주는 도구인지 금방 알아본다.

### 개발 상식 바이트코드와 JVM, 그리고 classpath

**바이트코드.** `javac`가 만든 `.class`는 특정 CPU의 기계어가 아니라 JVM이 읽는 명령어, 곧 바이트코드다. 그래서 Linux에서 컴파일한 `.class`가 macOS에서도, Windows에서도 실행된다.

**매직 넘버.** `.class` 파일을 글자가 아니라 바이트 그대로 열어 보면, 맨 앞 4바이트는 언제나 16진수로 `ca fe ba be`다. 16진수 두 자리가 1바이트이므로 `ca`, `fe`, `ba`, `be` 네 개가 곧 4바이트다. 파일의 종류를 알리는 이 표식을 **매직 넘버**(magic number)라고 부른다. JVM은 `.class` 파일을 읽을 때 맨 앞 4바이트가 `ca fe ba be`인지부터 확인하고, 아니면 클래스 파일이

아니라고 보고 거부한다. 파일의 맨 앞 8바이트를 16진수로 출력해 보자. `out` 폴더가 남아 있지 않으면 `target/classes` 아래의 같은 파일을 쓴다.

터미널 · macOS/Linux

```
$ xxd -p -l 8 out/net/java21/blog/BlogMain.class
cafebabe00000041
```

Linux에 `xxd`가 없으면 다음 명령으로 같은 바이트를 볼 수 있다.

터미널 · macOS/Linux Windows 735쪽

```
$ od -A x -t x1 -N 8 out/net/java21/blog/BlogMain.class
```

앞의 4바이트 `cafebabe`가 매직 넘버다. 뒤의 4바이트는 버전으로, 앞 2바이트 `0000`이 부 버전(minor_version), 뒤 2바이트 `0041`이 주 버전(major_version)이다. 16진수 41은 10진수로 65이고, 65는 자바 21로 컴파일했다는 뜻이다. 자바 17로 컴파일하면 주 버전이 61이므로 `0000003d`가 출력된다. 그래서 더 높은 버전으로 컴파일한 `.class`를 낮은 버전의 JVM에서 실행하면, JVM이 이 숫자를 보고 `UnsupportedClassVersionError`를 던진다.

**javap.** `javap -c`로 열면 메서드마다 바이트코드 명령이 보인다. `out` 폴더가 남아 있지 않으면 `-cp target/classes`로 바꿔 실행한다.

터미널

```
  # 1.4절에서 다시 컴파일한 out(코드는 step2와 같다)
$ javap -c -cp out net.java21.blog.BlogMain
  public static void main(java.lang.String[]) throws java.lang.Exception;
    Code:
… (생략)
  10: invokestatic  #17  // Method tools/jackson/databind/json/JsonMapper.…
  13: iconst_1
  14: anewarray     #23  // class tools/jackson/databind/SerializationFea…
… (생략)
  29: invokevirtual #35  // Method tools/jackson/databind/json/JsonMapper$…
… (생략)
```

`tools/jackson/databind/json/JsonMapper` 같은 Jackson의 클래스 이름이 그대로 적혀 있다. 컴파일러는 다른 사람의 코드를 복사하지 않고 이름으로 가리킨다. 1.3절에서 실행할 때 Jackson을 다시 찾아야 했던 까닭이다.

**classpath.** JVM이 클래스를 찾는 곳의 목록이다. 폴더나 JAR를 적고, 앞에 적은 것부터 찾는다. 같은 이름의 클래스가 두 곳에 있으면 먼저 찾은 것을 쓴다. 버전이 다른 같은 라이브러리가 classpath에 둘 들어가면 예상하기 어려운 오류가 발생하는 까닭이다. `java -jar`로 실행하면 classpath는 그 JAR와 manifest의 `Class-Path`로 정해지고, 명령에 적은 `-cp`는 무시된다.

**ClassNotFoundException과 NoClassDefFoundError.** 둘 다 실행할 때 클래스를 못 찾았다는 뜻이다. `ClassNotFoundException`은 코드가 문자열로 된 클래스 이름으로 클래스를 찾다가 없을 때, `NoClassDefFoundError`는 컴파일할 때는 있던 클래스가 실행할 때 없을 때 주로 본다. 어느 쪽이든 첫 질문은 같다. "실행할 때의 classpath에 그 클래스가 든 JAR가 있는가?"

## 이 장에서 배운 것

**표 1-6** 1장 정리

| 항목 | 핵심 |
|---|---|
| **javac와 java** | javac는 소스를 바이트코드(.class)로 바꾸고, java는 그 .class를 실행한다. 실행되는 것은 소스가 아니다. |
| **영속성과 JSON** | 메모리에 둔 글은 JVM이 끝나면 함께 사라진다. 프로그램이 끝난 뒤에도 남기려면 파일에 저장한다. JSON은 값의 경계를 큰따옴표가 정하므로, 본문에 쉼표가 들어와도 구조가 어긋나지 않는다. |
| **classpath** | 라이브러리는 컴파일할 때도, 실행할 때도 classpath에 있어야 한다. 컴파일러는 라이브러리 코드를 복사하지 않고 이름만 적는다. 컴파일 성공은 실행 성공이 아니다. |
| **JAR와 manifest** | JAR는 .class를 묶은 zip 파일이고, manifest에 Main-Class를 적으면 java -jar 한 줄로 실행할 수 있다. Class-Path는 경로일 뿐, 라이브러리를 담지 않는다. |
| **uber jar** | 라이브러리까지 한 파일에 담아, 그 파일 하나만으로 실행된다. 대신 크기, 같은 이름 파일의 충돌, 라이선스 고지를 챙겨야 한다. |
| **패키징과 빌드 도구** | 실행에 필요한 것을 모아 하나로 묶는 일이 패키징이다. 빌드 도구는 필요한 라이브러리를 내려받고, classpath를 구성하고, 테스트를 실행하고, 결과물을 묶는 일을 대신한다. |

| 항목 | 핵심 |
|---|---|
| **Maven과 Gradle** | Maven은 coordinates로 의존성을 내려받고(전이 의존성까지), 정해진 lifecycle(`compile` → `test` → `package`)을 따른다. scope는 그 라이브러리가 언제 classpath에 들어가고 결과물에 담기는지를 정한다(`compile`은 실행까지, `test`는 테스트에만). Gradle은 같은 일을 하되 빌드 설정을 스크립트로 작성한다. |

이 장의 코드는 실습 저장소 `build-and-packaging`의 `step1-exercise`부터 `step4-solution`까지의 브랜치에 있다(부록 B).

## 연습 문제

풀이는 실습 저장소 `build-and-packaging`의 `exercises` 브랜치에 있다. 먼저 직접 풀어 본 뒤 비교해 보자.

1. [기본] `step1-exercise`에서 본문을 빠뜨리고 `write`를 실행하면 예외가 발생한다. `args.length`를 확인해, 인자가 모자라면 예외 대신 사용법이 출력되게 고쳐 보자. `view`에서 번호를 빠뜨리거나 `view abc`처럼 숫자가 아닌 값을 주면 어떻게 되는지도 확인하자.
2. [응용] `step2-exercise`에서 1.3절의 `javac` 명령을 `-cp lib/*`처럼 큰따옴표 없이 실행해 보자(macOS와 Linux). 무엇이 실패하는가? 1.3절에서 큰따옴표로 감싼 까닭과 이어서 설명해 보자.
3. [기본] 1.4절에서 만든 `manifest.txt`에서 `Class-Path` 세 줄을 모두 지우고 JAR를 다시 만들어 보자. `lib/`가 바로 옆에 있는 같은 폴더에서도 실패하는가? 실패한다면 오류 메시지는 1.3절에서 본 어느 오류와 같은가?
4. [응용] `step4-solution`의 1차 패키징 설정(예제 1-6, `pom-first.xml`)에서 `includeScope` 줄을 지우고, 다음 명령으로 처음부터 다시 빌드해 보자.

   터미널

   ```
   $ mvn -f pom-first.xml clean package
   ```

   `target/lib`에 어떤 JAR가 더 생기는가? `mvn dependency:tree`의 출력에서 그 JAR의 scope를 찾아, 표 1-2의 `includeScope` 설명과 비교해 보자. 그다음 2차 설정(예제 1-7)으로 만든 uber

jar의 `META-INF/MANIFEST.MF`를 열어, 어느 JAR의 manifest가 남았는지 보자(1차의 manifest에서 본 `Created-By` 줄이 단서다).

5. [기본] `step4-exercise`의 `pom.xml` TODO를 채우고 빌드하자. 그다음 테스트에서 `assertEquals`의 첫 인자를 `List.of()`로 바꾸고, `mvn clean package`로 처음부터 다시 빌드한다. `target/`에 JAR가 생기는가?
6. [응용] 2차 설정(예제 1-7)에서 JUnit 의존성의 `<scope>test</scope>` 줄을 지우고 다시 빌드하면 uber jar의 크기가 어떻게 달라지는가? 다음 명령으로 JUnit이 함께 가져오는 전이 의존성을 보고, 1.6절에서 본 uber jar의 단점 가운데 어느 것과 이어지는지 설명해 보자.

터미널

```
$ mvn dependency:tree
```

---

## 마르코의 질문

**상황:** 내 컴퓨터에서 `mvn package`로 만든 JAR는 잘 실행된다. 그런데 같은 JAR를 서버에 배포하자 시작하자마자 `NoClassDefFoundError`가 발생한다. 동료가 말한다.

**"서버 라이브러리 폴더에 JAR를 하나 넣어 주면 되잖아요."**

**짚어야 할 것**

- 먼저 결과물이 무엇을 담았는지 본다. `jar tf`로 없다는 클래스가 JAR 안에 있는지 확인한다.
- `dependency:tree`에서 그 라이브러리의 scope가 `provided`(2장)나 `test`로 빠지지 않았는지, manifest의 `Main-Class`와 `Class-Path`가 무엇인지 확인한다.
- 고칠 곳은 대개 서버가 아니라 `pom.xml`이다. "이 결과물 하나만 있으면 어디서든 실행되는가?"가 기준이다.

**빠지기 쉬운 답:** "서버에 JAR를 따로 넣으면 됩니다." 그러면 그 서버에서만 실행되는 결과물이 된다. 서버를 늘리거나 다시 배포할 때마다 누군가 그 JAR를 직접 복사해 넣어야 하고, 서버마다 넣은 버전이 달라져도 알아차리기 어렵다.

· · ·

일요일 밤, 두원은 Maven으로 만든 JAR를 마르코에게 보냈다. 몇 분 뒤 답이 왔다. "잘 돌아가. 근데 글 목록이 비어 있는데?"

`posts.json`은 두원의 노트북에만 있었다. 글을 보려면 마르코가 두원의 컴퓨터에 와야 했다. 두원은 그날 밤 글에 작성 시각을 넣었다. 그리고 국숫집에서 마르코가 물었던 목록 순서를 최신 글이 먼저 나오게 바꾸다가, 저장이 가끔 되지 않는 문제를 겪었다. 원인을 찾느라 `System.out.println("여기까지 옴")`, `System.out.println("posts 크기: " + posts.size())` 같은 줄을 코드 곳곳에 넣었다. 문제를 찾고 나자 이번에는 그 줄이 어디 있는지 찾아 지우는 일이 남았다. 지우고 나면 다음에 또 넣어야 할 것 같았다.

> **마르코** 그 `println`들, 다음에 또 문제가 생기면 다시 넣을 거야? 그리고 언젠가 내가 이 블로그를 브라우저로 보게 되면, 그때도 터미널 앞에 앉아서 지켜보고 있을 거야?

2
장






# 로깅

프로그램이 하는 일을 기록으로 남기기

2.1 같은 번호의 글 두 개

2.2 System.out으로는 할 수 없는 것

2.3 API와 구현체: SLF4J와 Logback

2.4 로그 레벨을 고르고, 값을 넘기는 법

2.5 매번 쓰는 한 줄: Lombok의 @Slf4j

2.6 logback.xml: pattern, appender, logger

2.7 내 로그를 리뷰하기

2.8 스레드 두 개가 같은 파일에 쓸 때

2.9 자기 환경에 대입하기

### 이 장에서 배울 것

- 프로그램의 출력과 로그를 구분할 수 있다.
- 로그 레벨을 고르는 기준을 설명할 수 있다.
- 예외를 원인까지 로그에 남길 수 있다.
- 로그에서 어느 스레드가 언제 무엇을 했는지 읽어 낼 수 있다.

### 이 장의 실습

- **저장소:** `logging`
- **브랜치:** `step1-exercise`에서 시작해 `step4-solution`까지. 단계마다 `stepN-exercise`에서 시작, `stepN-solution`과 비교
- **만들 것:** SLF4J와 Logback으로 로그를 남기는 터미널 블로그, Lombok `@Slf4j`로 줄인 로거 선언, 형식·내보낼 곳·레벨을 정한 `logback.xml`, 두 스레드가 함께 남기는 로그

월요일 저녁, 두원이 마르코에게 메시지를 보냈다.

> **두원** 어젯밤에 찾던 거 찾았어. 저장이 안 된 게 아니었어. 번호가 겹쳐서, 하나를 지우면 같은 번호 글이 같이 지워진 거야.
>
> **마르코** 오, 찾았구나. 어제 말한 그 `println`들로?
>
> **두원** 응, 여기저기 넣어 봤지. 근데 이제 `list`만 입력해도 `DEBUG`로 시작하는 출력이 글 목록 사이에 섞여 나와. 지우자니 다음에 또 필요할 것 같고, 두자니 지저분하고.
>
> **마르코** 그 디버깅 코드를 그대로 두고 JAR를 만들면, 다음에 나한테 보낼 JAR에도 그 출력이 다 들어가겠네?
>
> **두원** 어… 그렇겠네.
>
> **마르코** 나도 처음엔 그렇게 넣었다가 지우는 걸 깜빡하고 그대로 내보낸 적 있어. 이번엔 그걸 지우지 말고, 필요할 때만 남길 수 있게 바꿔 볼래? 예제는 새 저장소에 올려 둘게.

잠시 뒤 마르코가 저장소 주소를 보냈다. 1장에서처럼 단계마다 실습 브랜치(exercise)와 완성 브랜치(solution)가 있었고, 첫 실습 브랜치에는 디버깅 코드가 그대로 남은 지금의 블로그

코드가 들어 있었다. 1장의 `build-and-packaging` 폴더 안에서 내려받으면 저장소 안에 저장소가 생긴다. 그래서 두원은 내려받기 전에, 앞 장에서 정한 대로 실습 저장소를 모아 두는 `~/dev`(Windows는 `C:\dev`)로 먼저 이동했다.

터미널 · macOS/Linux Windows 735쪽

```
$ cd ~/dev
```

그리고 저장소를 내려받아 첫 실습 브랜치로 전환한다.

터미널

```
$ git clone https://github.com/backend-blog/logging.git
$ cd logging
$ git switch step1-exercise
```

이 장의 명령은 모두 `logging` 폴더 안에서 실행한다. 단계를 마치면 완성 브랜치와 비교한다. 1단계라면 다음과 같다.

터미널

```
$ git diff origin/step1-solution
```

1~3단계에는 TODO를 채웠는지 확인하는 테스트(`LoggerSetupTest`, `LombokSetupTest`, `LogbackConfigTest`)가 들어 있다. TODO가 남아 있으면 `mvn test`와 `mvn package`가 실패하고, 실패 메시지에 남은 TODO 번호가 나온다. 다만 이 테스트는 로거를 두었는지, 표준 출력에 디버깅 줄이 남았는지, 설정 값이 들어 있는지만 확인한다. 로그의 레벨과 메시지가 알맞은지는 완성 브랜치와 비교하고, 직접 해 보기의 마지막 확인 항목을 따라 해 보며 판단한다.

다음 단계로 넘어갈 때는 고친 것을 먼저 커밋하고, 브랜치 이름의 숫자만 바꿔 `git switch`를 실행한다. `git add -A`는 고친 파일과 새 파일을 모두 담는다. 1단계에서 2단계로 넘어간다면 이렇다.

터미널

```
$ git add -A
$ git commit -m "1단계 채우기"
$ git switch step2-exercise
```

## 2.1 같은 번호의 글 두 개

앞 장에서 Maven으로 묶은 터미널 블로그는 그사이 조금 바뀌었다. `BlogMain.java` 하나였던 코드는 명령을 받는 `BlogMain`, `posts.json`을 읽고 쓰는 `PostStore`, 글을 나타내는 record `Post`로 나뉘었고, 글에는 작성 시각이 추가되었다. 그리고 국숫집에서 마르코가 물었던 목록 순서도 정했다. 최신 글이 먼저 나오도록 새 글을 목록 맨 앞에 넣는다(`posts.addFirst(…)`, 1장의 `getLast()`처럼 자바 21에서 `List`에 추가된 메서드다).

코드 모양도 달라졌다. 명령을 고르는 곳은 1장의 `if`/`else` 대신 화살표 모양의 `switch`(`case "write" -> { … }`)를 쓴다. 값이 일치하는 `case`의 블록 하나만 실행되고, 다음 `case`로 넘어가지 않는다.

글을 지우는 `posts.removeIf(p -> p.id() == id)`의 `p -> p.id() == id`는 람다(lambda)다. "글 `p`를 받아 `p.id() == id`의 결과를 돌려준다"는 이름 없는 메서드를 값처럼 넘기는 문법이다. `removeIf`는 글마다 이 람다를 실행해 참이 나온 글을 지운다.

그런데 글을 몇 편 써 보니 목록이 이상하게 나왔다. 이 절은 읽기만 해도 된다. 실습 브랜치는 이 버그를 고친 상태(예제 2-1)에서 시작한다.

터미널

```
$ java -jar target/blog-1.0.jar write "첫 글" "안녕"
1번 글을 저장했습니다.
$ java -jar target/blog-1.0.jar write "둘째 글" "반가워"
2번 글을 저장했습니다.
$ java -jar target/blog-1.0.jar write "셋째 글" "또"
2번 글을 저장했습니다.
$ java -jar target/blog-1.0.jar list
2. 셋째 글 (2026-10-18T22:40:52)
2. 둘째 글 (2026-10-18T22:40:51)
1. 첫 글 (2026-10-18T22:40:49)
$ java -jar target/blog-1.0.jar delete 2
2번 글을 삭제했습니다.
$ java -jar target/blog-1.0.jar list
1. 첫 글 (2026-10-18T22:40:49)
```

괄호 안은 글을 쓴 시각이다. 가운데의 `T`는 날짜와 시각을 가르는 글자로, `LocalDateTime`의 `toString()`이 쓰는 ISO 8601 형식이다.

셋째 글도 2번을 받아 2번 글이 두 개가 되었다. 그래서 `delete 2`가 둘을 함께 지웠고, 셋째 글은 저장되지 않은 것처럼 보였다.

두원은 새 번호를 정하는 줄 앞뒤에 `System.out.println`을 넣어 목록의 번호와 새 번호를 출력해 보았다. 금방 원인이 보였다. 새 번호는 1장에서처럼 목록의 마지막 글 번호에 1을 더해(`posts.getLast().id() + 1`) 정하고 있었다. 새 글을 맨 뒤에 넣던 때는 마지막 글이 가장 큰 번호였다. 맨 앞에 넣도록 바꾸자 마지막 글은 가장 오래된 1번 글이 되었고, 셋째 글은 1 + 1 = 2번을 받았다. 번호를 정하는 코드가 목록의 순서에 의존하고 있었다.

두원은 번호를 "가장 큰 번호 + 1"로 고쳤다(예제 2-1).

**예제 2-1** `BlogMain.java`: 새 번호 정하기(`nextId`, 디버깅 코드가 남은 채)

```
static long nextId(List<Post> posts) {
    long max = 0;
    for (Post p : posts) {
        System.out.println("DEBUG 비교 id = " + p.id() + ", max = " + max);
        max = Math.max(max, p.id());
    }
    return max + 1;
}
```

목록의 글을 처음부터 끝까지 살피며 지금까지 본 가장 큰 번호를 `max`에 남기므로, 글이 어떤 순서로 놓여 있든 새 번호는 어느 글과도 겹치지 않는다.

버그는 고쳤다. 문제는 그다음이다. 예제 2-1의 `DEBUG 비교` 줄처럼 원인을 찾는 동안 넣은 `println`이 `BlogMain`과 `PostStore`에 여덟 개 남았다. 글을 하나 쓰면 `DEBUG 저장: [Post[id=1, title=첫 글, body=안녕, …]]` 같은 출력이 네다섯 개씩 먼저 나온다. 목록을 파일로 저장해(`>`는 화면 대신 파일로 출력을 보낸다, 2.2절에서 자세히 본다) 마르코에게 보내려 하자, 그 파일에도 디버깅 출력이 그대로 들어갔다.

터미널

```
$ java -jar target/blog-1.0.jar list > out.txt
$ cat out.txt
DEBUG args = list
DEBUG 읽은 글 수 = 3
3. 셋째 글 (2026-10-18T23:05:13)
2. 둘째 글 (2026-10-18T23:05:12)
1. 첫 글 (2026-10-18T23:05:11)
```

**마르코** 그 파일 한번 볼래? `3. 셋째 글`은 프로그램이 사용자에게 보여 주는 결과잖아. `DEBUG 읽은 글 수`는 누구한테 하는 말이야?

**두원** 나한테지. 디버깅하고 있는 나.

**마르코** 그렇지. 근데 둘이 같은 통로로 섞여 나가고 있잖아. 사용자에게 보여 줄 것과 너만 볼 것을 어떻게 따로 내보낼 수 있을까?

두원은 디버거부터 생각했다. 디버거는 코드의 원하는 줄(브레이크포인트)에서 프로그램을 멈춰 세우고 그 순간의 변수 값을 보여 주는 도구다.

**두원** 아예 `println`을 다 지우고, 다음부터는 디버거로 브레이크포인트를 걸어서 보면 안 돼?

**마르코** 네 노트북에서 멈춰 볼 수 있을 땐 그게 제일 편하지. 그런데 이 블로그는 곧 서버가 돼. 서버를 멈춰 세우면 그동안 다른 요청도 다 같이 기다려야 하고, 이미 지나간 일은 남겨 둔 기록으로만 볼 수 있어. 그래서 지우는 대신 따로 남기는 법이 필요해.

## 2.2 System.out으로는 할 수 없는 것

두원이 넣은 디버깅 코드는 모두 `System.out.println`이다. 표준 출력(System.out)은 프로그램이 사용자에게 결과를 내보내는 통로다. 디버깅 출력까지 같은 통로로 내보내니 결과와 섞인다. 그런데 프로그램이 글자를 내보내는 통로는 하나가 아니다. 그림 2-1은 두 통로가 어디로 가는지 보여 준다.

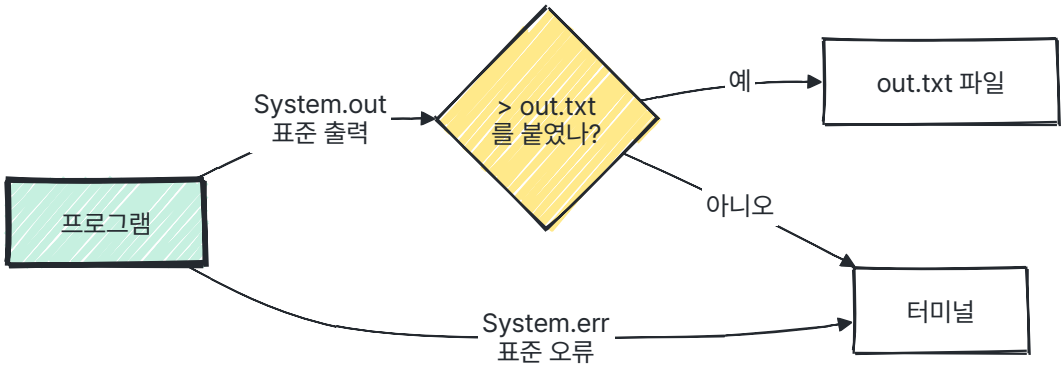



**그림 2-1** > `out.txt`를 붙이면 표준 출력만 파일로 가고, 표준 오류는 그대로 터미널에 남는다

### 참고 표준 출력과 표준 오류

프로그램은 시작할 때 운영체제에서 통로 세 개를 받는다. 입력을 받는 표준 입력(stdin), 결과를 내보내는 표준 출력(stdout), 오류와 안내를 내보내는 표준 오류(stderr)다. 자바에서는 각각 `System.in`, `System.out`, `System.err`로 쓴다. 터미널에서는 표준 출력과 표준 오류가 모두 화면에 나와 하나처럼 보이지만, 실제로는 다른 통로다. 2.1절에서 쓴 `>`는 표준 출력만 파일로 보내고, 표준 오류는 그대로 화면에 남긴다. 표준 오류를 따로 파일로 보내려면 `2>`를 쓴다. 이 구분은 macOS와 Linux의 셸에서도, Windows PowerShell에서도 같다.

출력이 섞이는 것 말고도, `System.out`으로 하기 어려운 일은 표 2-1처럼 여럿이다.

**표 2-1** `System.out`으로는 하기 어려운 일

| 하고 싶은 일 | `System.out`으로는 |
|---|---|
| 평소에는 남기지 않고, 필요할 때만 남기기 | 지우거나 주석 처리하고 다시 빌드한다 |
| 중요한 로그와 사소한 로그 나누기 | `"DEBUG"`라는 문자열을 붙일 뿐이다 |
| 언제, 어느 스레드, 어느 클래스인지 알기 | 매번 직접 써넣어야 한다 |
| 파일로 남기거나 형식 바꾸기 | 모든 `println`을 고쳐야 한다 |
| 운영에서 비용 줄이기 | 멈출 수 없으니 언제나 문자열을 만든다 |

마지막 항목은 지금은 사소해 보인다. 그런데 이 프로그램은 다음 장에서 서버가 된다. 요청마다 글 목록 전체를 문자열로 만들어 출력한다면, 아무도 읽지 않는 출력에 매번 시간을 버린다.

`DEBUG 저장:` 출력에는 문제가 하나 더 있다. record의 `toString()`이 본문 `안녕`까지 내보냈다. 비밀번호가 담긴 객체였다면 어땠을까(2.4절).

그래서 이런 도구가 필요하다. 로그마다 레벨(얼마나 중요한가)과 시각, 스레드, 클래스 이름을 붙이고, 어디로 어느 레벨부터 남길지는 설정으로 정할 수 있어야 한다. 이런 기록이 로그이고, 그 도구가 로깅 라이브러리다.

## 2.3 API와 구현체: SLF4J와 Logback

### 직접 해 보기 1 디버깅 출력을 로그로

**브랜치:** `step1-exercise`에서 시작, `step1-solution`과 비교

**목표:** `System.out` 디버깅 출력을 SLF4J 로그로 바꾸고, Logback이 출력하게 한다.

**왜 필요한가:** 디버깅 출력과 프로그램의 출력이 같은 통로로 나가 섞인다. 멈추거나 나눌 방법이 없다.

1. 1단계 실습 브랜치로 전환한다.

   터미널

   ```
   $ git switch step1-exercise
   ```

2. `pom.xml`에 로깅 의존성 두 개를 넣는다(`TODO#1`).
3. `PostStore`와 `BlogMain`에 로거를 하나씩 선언한다(`TODO#2`, `#3`). `DEBUG` 출력 여덟 개 가운데 남길 것은 레벨에 맞는 로그로 바꾸고, 필요 없는 것은 지운다. 레벨은 2.4절의 기준을 읽고 고른다. 완성 브랜치가 여덟 개를 어떻게 정리했는지도 그 절에 있다. `main`에서 예외를 잡아 로그로 남기는 일(`TODO#4`, 예제 2-4)도 이 단계에서 한다.
4. 빌드한 뒤 `list`를 실행해, 로그에 시각·스레드·레벨·클래스 이름이 붙는지 확인한다. 로그가 출력과 섞이는 문제는 직접 해 보기 3에서 푼다.

터미널

```
$ mvn package
$ java -jar target/blog-1.0.jar list
```

**생각해 보기:** 내 코드는 SLF4J만 호출하고 Logback은 전혀 모른다. 로깅을 API와 구현체로 굳이 나눠 두면 어떤 점이 좋을까?

자바의 로깅 라이브러리는 보통 API와 구현체로 나뉜다. 코드가 호출하는 쪽이 API이고, 로그를 실제로 콘솔이나 파일에 출력하는 쪽이 구현체(구현을 담은 라이브러리)다. API는 다른 코드가 호출할 수 있게 공개한 메서드의 목록과 그 사용 규칙이다. 1.2절에서 본 웹 API도 넓게 보면 그 한 종류다. 이 장에서는 API로 SLF4J를, 구현체로 Logback을 쓴다.

그림 2-2처럼 내 코드는 SLF4J의 `Logger`만 호출한다. SLF4J가 받은 로그를 Logback에 넘기면, Logback이 시각과 레벨을 붙여 출력한다. 내 코드는 Logback을 전혀 모른다.

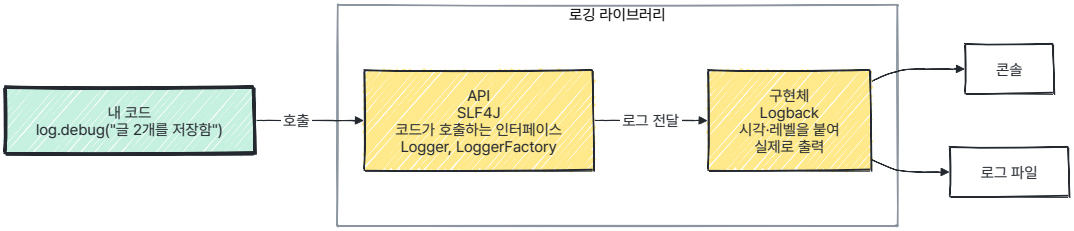



**그림 2-2** 코드는 API(SLF4J)를 호출하고, 실제 출력은 구현체(Logback)가 맡는다

실습 브랜치의 `pom.xml`에는 1장에서 쓴 Jackson과 JUnit 의존성 다음에 로깅 의존성을 넣을 자리가 비어 있다.

**실습 브랜치의 모습** `pom.xml`: 로깅 의존성을 넣을 자리

```
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>6.1.3</version>
      <scope>test</scope>
    </dependency>
    <!-- TODO#1 SLF4J(API)와 Logback(구현체)을 더한다.
              (2.3절, 예제 2-2) 코드가 부르는 쪽은 org.slf4j:slf4j-api 2.0.19,
              실제로 출력하는 쪽은 ch.qos.logback:logback-classic 1.6.5다.
              둘 다 scope 없이(compile) 적는다.
              채우면 LoggerSetupTest의 TODO#1 테스트가 통과한다. -->
</dependencies>
```

`TODO#1`에는 그림 2-2의 두 라이브러리, 곧 코드가 부르는 API인 `slf4j-api`와 실제로 출력하는 구현체인 `logback-classic`을 `<dependency>` 두 개로 적는다. 둘 다 `<scope>`를 적지 않는다(기본값 `compile`). 채우면 실습 브랜치의 `LoggerSetupTest`에서 `TODO#1`을 확인하는 테스트가 통과한다.

`TODO#1`을 직접 채운 뒤 예제 2-2와 비교해 보자.

두원이 추가한 의존성은 다음과 같다. 버전은 Maven 중앙 저장소에서 확인해 고정했다.

**예제 2-2** `pom.xml`: SLF4J와 Logback 의존성

```
<!-- 코드가 부르는 로깅 API -->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.19</version>
</dependency>
<!-- 실제로 로그를 출력하는 구현체 -->
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.6.5</version>
</dependency>
```

> **두원** 근데 왜 두 개야? 하나로 되면 좋잖아.
>
> **마르코** 그럼 처음엔 API 하나만 넣어 볼래? 무슨 일이 생기는지 보고 얘기하자.

로거(logger)는 로그를 남기는 객체다. 클래스마다 하나씩 두고, `log.info(…)`처럼 레벨 이름의 메서드를 호출해 로그를 남긴다. 두원은 Logback 의존성을 빼고 SLF4J만 넣은 채 로거를 두고, 디버깅 출력을 로그로 바꿨다. 궁금하면 따라 해 보고, 끝나면 Logback을 다시 넣는다. 이때는 `LoggerSetupTest`가 `TODO#1`에서 실패하므로 `mvn package -DskipTests`로 빌드한다. 실습 브랜치의 `PostStore`에는 로거를 둘 자리에 `TODO#2`가 있고, 그 뒤의 메서드에는 `System.out` 디버깅 줄이 남아 있다.

**실습 브랜치의 모습** `PostStore.java`: 로거를 둘 자리와 남은 디버깅 줄

```
// 글 목록을 JSON 파일 하나에 읽고 쓴다
public class PostStore {

    // TODO#2 로거를 두고 System.out 디버깅 줄을 로그로 바꾼다.
    //        (2.3절, 예제 2-3) 로거는 클래스마다 하나,
    //        private static final 필드로 둔다. 값은 문자열을 이어 붙이지 말고
    //        {} 자리에 넘긴다(2.4절 "문자열을 이어 붙이지 않는다").
    //        저장할 때 목록 전체를 남기지 말고 글 수와 파일 이름만
    //        남긴다(2.4절 "남기면 안 되는 값"). 레벨은 2.4절의 기준으로 고른다.
    //        빈 파일은 사용자가 알아야 할 일이다.
    //        힌트: LoggerFactory.getLogger(PostStore.class),
    //        log.debug("…{}", 값), log.warn

    // … (생략: 필드와 생성자)

    public List<Post> load() throws IOException {
        if (Files.notExists(file)) {
            System.out.println("DEBUG 파일 없음: " + file);
            return new ArrayList<>();
        }
        // … (생략: 파일을 읽는다. 빈 파일, 읽은 글 수에도 DEBUG 줄이 있다)
```

`TODO#2`에서 할 일은 두 가지다. 클래스에 로거를 `private static final` 필드로 하나 두고(`LoggerFactory.getLogger(PostStore.class)`), `System.out` 디버깅 줄을 레벨에 맞는 로그로 바꾼다. 값은 `+`로 이어 붙이지 않고 `{}` 자리에 넘기며, 저장할 때는 목록 전체 대신 글 수와 파일 이름만 남긴다. 레벨을 고르는 기준과 `{}`가 왜 좋은지는 2.4절에서 자세히 본다.

`TODO#2`를 직접 채운 뒤 예제 2-3과 비교해 보자.

두원이 고친 `PostStore`의 주요 부분은 다음과 같다.

**예제 2-3** PostStore.java: 로거와 로그

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
// … (생략: 나머지 import)

// 글 목록을 JSON 파일 하나에 읽고 쓴다
public class PostStore {

    // 로거는 클래스마다 하나 둔다. 로그 줄에 이 클래스 이름이 남는다(2.3절)
    private static final Logger log = LoggerFactory.getLogger(PostStore.class);

    // … (생략: 필드와 생성자)

    public List<Post> load() throws IOException {
        if (Files.notExists(file)) {
            log.debug("글 파일이 없어 빈 목록으로 시작: {}", file);
            return new ArrayList<>();
        }
        // … (생략: 파일을 읽어 JSON을 글 목록으로 바꾼다)
        log.debug("글 {}개를 읽음: {}", posts.size(), file);
        return new ArrayList<>(posts);
    }
```

`PostStore.class`는 `PostStore` 클래스를 가리키는 값이고(5.6절에서 자세히 본다), `LoggerFactory.getLogger(PostStore.class)`는 클래스 이름 `net.java21.blog.PostStore`를 그대로 로거 이름으로 쓰는 로거를 돌려준다. 로그 레벨을 정할 때 이 로거 이름을 쓴다(2.6절).

`log.debug("…: {}", file)`의 `{}` 자리에는 뒤의 값이 들어간다. 빌드하고 실행하자 이런 경고가 먼저 출력되었다.

터미널

```
$ java -jar target/blog-1.0.jar write "첫 글" "안녕"
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further det…
1번 글을 저장했습니다.
```

컴파일도 되고 실행도 된다. 그런데 SLF4J가 "구현체(provider)를 찾지 못해 아무것도 하지 않는 NOP 로거를 쓴다"고 경고한다. 로그는 하나도 남지 않았다. SLF4J는 받은 로그를 구현체에 넘길 뿐이고, 넘길 구현체가 없으면 버린다.

Logback 의존성을 다시 넣고 빌드하자 경고가 사라지고 로그가 출력되었다.

터미널

```
$ java -jar target/blog-1.0.jar write "둘째 글" "반가워"
21:41:19.570 [main] DEBUG net.java21.blog.BlogMain -- 명령 시작: write
21:41:19.876 [main] DEBUG ….PostStore -- 글 1개를 읽음: posts.json
21:41:19.906 [main] DEBUG ….PostStore -- 글 2개를 저장함: posts.json
21:41:19.906 [main] INFO net.java21.blog.BlogMain -- 글 저장: id=2
2번 글을 저장했습니다.
```

설정 파일이 없을 때 Logback은 기본 형식으로 `DEBUG` 이상을 콘솔에 출력한다. `….PostStore`는 지면이 좁아 패키지 이름을 줄여 실었다. 아직 출력과 섞이지만, 로그마다 시각, 스레드 이름(`main`), 레벨, 로거 이름이 붙었다. 스레드는 프로그램 안에서 코드를 차례로 실행하는 흐름이며, 2.8절에서 자세히 본다.

**마르코** 이제 잘 나오네. 그럼 아까 질문으로 돌아가 볼까? 왜 두 개로 나눠 놨을까?

**두원** 그러게. Logback 하나만 있으면 될 것 같은데.

**마르코** Jackson 같은 라이브러리가 로그를 남긴다고 해 보자. Jackson이 Logback을 직접 쓰면 어떻게 될까?

**두원** 그럼… Jackson을 쓰는 사람은 다 Logback을 써야겠네. 나는 다른 걸 쓰고 싶어도.

**마르코** 맞아. 그래서 라이브러리는 API에만 기대고, 구현체는 프로그램을 만드는 쪽이 고르는 거야.

그림 2-3처럼 내 코드와 라이브러리의 로그가 모두 한 구현체로 모여 한 형식으로 쌓인다. 구현체를 바꿀 때도 코드는 그대로 두고, `pom.xml`의 의존성과 그 구현체의 설정 파일(Logback이라면 2.6절에서 만들 `logback.xml`)만 바꾸면 된다.

### 참고 facade와 퍼사드 패턴

facade는 건물의 정면을 뜻하는 말이다. 길에서 보이는 것은 정면 하나뿐이고, 그 뒤에 방이 몇 개인지, 배관이 어떻게 지나가는지는 보이지 않는다. 소프트웨어에서도 복잡한 안쪽을 감추고 단순한 앞면 하나만 내놓는 구조를 이렇게 부른다. 1994년 에릭 감마(Erich Gamma) 등 네

저자가 쓴 『Design Patterns』는 이것을 **퍼사드 패턴**(facade pattern)이라는 이름으로 정리했다. 여러 부품으로 이루어진 시스템 앞에 창구 하나를 두고, 쓰는 쪽은 그 창구만 알게 하는 구조다.

SLF4J라는 이름도 Simple Logging Facade for Java의 줄임말이다. 공식 설명서의 첫 문장은 이렇다.

**"The Simple Logging Facade for Java (SLF4J) serves as a simple facade or abstraction for various logging frameworks ..."**

여러 로깅 프레임워크 앞에 놓인 단순한 앞면, 곧 추상화(abstraction)라는 뜻이다. 이 문장은 이어서, 사용자가 원하는 로깅 프레임워크를 배포할 때 끼워 넣을 수 있다고 설명한다. 방금 두원이 겪은 일이 그 설명 그대로다. 코드는 SLF4J의 `Logger`만 호출하고, 로그를 실제로 출력하는 일은 실행할 때 classpath에서 찾은 구현체가 맡는다. `pom.xml`에서 Logback 의존성을 빼자 로그가 사라졌고, 되돌려 넣자 다시 출력되었다. 그동안 `BlogMain`과 `PostStore`의 로그 코드는 한 줄도 바뀌지 않았다.

『Design Patterns』의 퍼사드는 주로 "부품이 많아 쓰기 복잡한 것"을 단순하게 만드는 데 쓴다. SLF4J는 조금 다른 것을 감춘다. 뒤에 있는 구현체가 Logback인지, Log4j 2인지, JDK의 `java.util.logging`인지를 쓰는 쪽이 모르게 한다. 그래서 설명서도 facade와 abstraction을 나란히 적었다. 앞면 하나로 뒤쪽을 감춘다는 점은 같다.

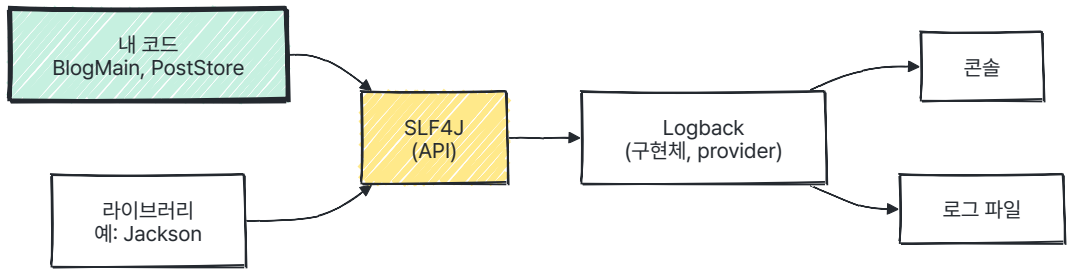



**그림 2-3** 라이브러리도 SLF4J만 호출하니, 내 코드와 라이브러리의 로그가 프로그램이 고른 Logback 하나로 모인다

## 2.4 로그 레벨을 고르고, 값을 넘기는 법

로그 레벨(log level)은 로그의 중요도다. SLF4J의 로그 레벨은 다섯 단계다. 터미널 블로그에서는 표 2-2처럼 골랐다.

**표 2-2** 로그 레벨과 터미널 블로그에서 고른 예

| 레벨 | 언제 쓰나 | 터미널 블로그에서는 |
|---|---|---|
| `ERROR` | 오류가 발생해 작업이 실패했다. 바로 확인해야 한다 | 글 파일이 망가져 명령이 실패함 |
| `WARN` | 당장 장애는 아니지만 잠재적인 문제가 있다 | 빈 파일이라 빈 목록으로 시작 (파일이 아예 없으면 `DEBUG`) |
| `INFO` | 정상 흐름에서 기록해 둘 주요 이벤트 | 글 저장, 글 삭제 |
| `DEBUG` | 개발하거나 디버깅할 때 보는 상세 정보 | 글 몇 개를 읽고 썼나 |
| `TRACE` | DEBUG보다 더 상세한 정보. 보통은 남기지 않는다 | 다음 번호를 계산함 |

고르는 기준은 하나다. **이 로그를 보는 사람이 무엇을 해야 하나.** 할 일이 있으면 `ERROR`, 살펴볼 일이면 `WARN`, 나중에 "그때 무슨 일이 있었지?"를 물을 때 필요하면 `INFO`다. 나머지는 `DEBUG`나 `TRACE`로 내린다.

표 2-2의 레벨은 `ERROR`에서 `TRACE`로 갈수록 덜 중요하고 더 상세하다. 레벨을 하나 정하면 그 레벨과 그보다 중요한 로그만 남고, 덜 중요한 로그는 버려진다. 그림 2-4는 레벨을 `INFO`로 정했을 때의 모습이다. 이 레벨은 2.6절에서 설정 파일로 정한다.

두원은 처음에 "지울 글이 없음"을 `WARN`으로 남겼다. `delete 7`처럼 없는 번호를 넣었을 때 남는 로그다.

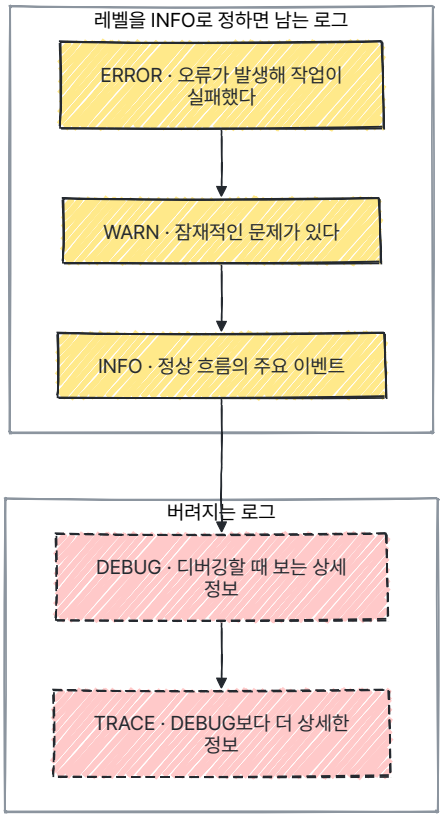



**그림 2-4** 레벨을 INFO로 정하면 ERROR, WARN, INFO는 남고 DEBUG, TRACE는 버려진다

> **마르코** 나쁘지 않은데, 그 `WARN`을 보면 누가 뭘 해야 해?
>
> **두원** 음… 아무도? 사용자한테는 이미 "7번 글이 없습니다"라고 알려 주니까.

사용자가 잘못 넣은 값은 사용자에게 알리는 것으로 충분하다. 두원은 그 로그를 `INFO`로 내렸다. 대신 `posts.json`이 비어 있을 때를 `WARN`으로 남기기로 했다. 프로그램은 빈 목록으로 계속 동작하지만, 누군가 파일을 비운 이유는 살펴볼 만하다.

`BlogMain`에도 로거를 둘 자리가 있다. 실습 브랜치의 `BlogMain` 앞부분에는 이 클래스의 디버깅 출력을 정리하는 `TODO#3`과, 예외를 로그로 남기는 `TODO#4`가 있다.

**실습 브랜치의 모습** `BlogMain.java`: 로거와 예외 처리를 넣을 자리

```
public class BlogMain {

    // TODO#3 로거를 두고, 이 클래스의 DEBUG 출력
    //        네 개(args, 새 id, removed, 비교 id)를 정리한다.
    //        (2.4절, 표 2-2와 이어지는 문단) 남길 것은 레벨에 맞는 로그로
    //        바꾸고, 결과 로그와 겹치는 것은 지운다. 반복문 안에서
    //        글마다 남기던 줄은 반복문이 끝난 뒤 TRACE 한 줄로 줄인다.
    //        write, edit, delete에는 명령의 결과를 남기는 INFO
    //        로그를 새로 넣는다.
    //        힌트: log.debug, log.info, log.trace
    //        (TODO#2와 같은 모양으로 로거를 선언한다)

    // TODO#4 main에서 예외를 잡아 로그로 남기고, 사용자에게는 짧게 알린다.
    //        (2.4절 "예외는 마지막 인자로", 예제 2-4) main의 throws 절을 지우고
    //        run 호출을 try로 감싼다. 예외 객체는 log.error의 마지막 인자로
    //        넘겨야 스택 트레이스가 남는다.
    //        힌트: catch (IOException | RuntimeException e),
    //        log.error("…{}", command, e)
    public static void main(String[] args) throws IOException {
        System.out.println("DEBUG args = " + String.join(",", args));
        if (args.length == 0) {
            // … (생략: 사용법을 보여 주고 끝낸다)
            return;
        }
        run(args[0], args);
    }
```

`TODO#3`은 `TODO#2`와 같은 모양으로 로거를 선언하고, `main`의 `args`, `write`의 `새 id`, `delete`의 `removed`, `nextId`의 `비교 id` 네 출력을 표 2-2의 기준으로 정리하는 자리다. 결과 로그와 겹치는 것은 지우고, `write`, `edit`, `delete`에는 명령의 결과를 남기는 `INFO` 로그를 새로 넣는다. 메시지 글자는 달라도 된다. 레벨과 로그를 남기는 자리만 맞으면 된다. `TODO#4`는 조금 뒤 "예외는 마지막 인자로"에서 채운다.

`TODO#3`을 직접 채운 뒤 이어지는 정리와 비교해 보자.

직접 해 보기 1의 완성 브랜치는 이 기준으로 `DEBUG` 출력 여덟 개를 다음과 같이 정리했다.

- 로그로 바꾼 것(여섯 개): `args`는 `DEBUG` "명령 시작"이 되었고, 파일이 없을 때와 글을 읽었을 때는 `DEBUG`, 빈 파일은 `WARN`이 되었다. 저장할 때 목록 전체를 내보내던 출력은 글

수와 파일 이름만 남기는 `DEBUG`로 바꿨다(이 절 끝의 "남기면 안 되는 값"). 반복문 안에서 글마다 출력하던 `비교 id`는 반복문이 끝난 뒤 남기는 `TRACE` 한 줄("다음 번호 계산: 글 N개, 가장 큰 번호 M")로 줄였다.

- 지운 것(두 개): `새 id`와 `removed`. 바로 뒤에 남기는 결과 로그에 같은 정보가 들어가므로 따로 남길 필요가 없다.
- 새로 넣은 것: 명령의 결과를 남기는 `INFO` 로그다. `write`의 "글 저장: id=…", `edit`의 "고칠 글이 없음: id=…"과 "글 수정: id=…", `delete`의 "지울 글이 없음: id=…"과 "글 삭제: id=…, 남은 글 …개"다.

내 코드가 이것과 조금 달라도 기준에 맞게 골랐다면 괜찮다. 다만 2.6절에서 보여 주는 로그 파일 내용은 완성 브랜치의 메시지 그대로다. 메시지가 다르면 그만큼 내 로그 파일의 줄도 달라진다.

### 문자열을 이어 붙이지 않는다

로그 메시지는 `{}` 자리에 값을 넘겨 만든다.

```
log.debug("글 {}개를 읽음: {}", posts.size(), file);     // 좋은 예
log.debug("글 " + posts.size() + "개를 읽음: " + file);  // 나쁜 예
```

두 코드의 차이는 `DEBUG` 로그를 남기지 않도록 레벨을 정했을 때 드러난다. 첫 번째 코드는 `DEBUG`를 남기지 않을 때 메시지 문자열을 아예 만들지 않는다. 두 번째 코드는 `+`로 문자열을 먼저 만들고, 로거는 그 문자열을 출력하지 않고 버린다. 출력하지도 않을 로그에 비용만 치른다. 글 목록 전체를 이어 붙였다면 목록의 `toString()` 비용까지 든다. 다만 `{}`가 미루는 것은 문자열을 만드는 일뿐이다. 인자로 넘긴 `posts.size()` 같은 식은 레벨과 상관없이 먼저 계산된다. 값을 구하는 일 자체가 무겁다면 `if (log.isDebugEnabled())`로 감싼다.

### 예외는 마지막 인자로

두원은 일부러 `posts.json`을 반쯤 지워 망가뜨려 보았다. 글 두 개가 들어 있는 파일에서 첫 글이 끝나는 `}, {` 줄은 남기고, 그 뒤를 끝까지 지웠다.

직접 해 보려면 이렇게 한다.

1. `list`로 글이 두 개 이상인지 보고, 모자라면 `write`로 채운다. 글이 하나뿐이면 `}, {` 줄이 없다.
2. 편집기로 `posts.json`을 열어 같은 곳을 지운다.
3. 실험이 끝나면 `posts.json`을 지워 둔다. 다음 실행에서 빈 목록으로 새로 시작한다.

터미널 · macOS/Linux Windows 735쪽

```
$ rm -f posts.json
```

로그를 넣기 전에는 `Exception in thread "main"`과 함께 프로그램이 종료되었다. 앞에서 본 실습 브랜치의 `main`은 `throws IOException`으로 예외를 밖으로 던지기만 하기 때문이다. 예외를 로그로 남기는 `TODO#4`는 `main`의 `throws` 절을 지우고 `run` 호출을 `try`로 감싼 뒤, `catch`에서 예외 객체를 `log.error`의 마지막 인자로 넘기는 자리다. 사용자에게는 짧게 알린다.

`TODO#4`를 직접 채운 뒤 예제 2-4와 비교해 보자.

두원이 채운 `main`은 다음과 같다.

**예제 2-4** BlogMain.java: 예외를 로그로 남기는 main

```
public static void main(String[] args) {
    if (args.length == 0) {
        // … (생략: 사용법을 보여 주고 끝낸다)
        return;
    }
    String command = args[0];
    // 값은 문자열로 이어 붙이지 않고 {} 자리에 넘긴다(2.4절)
    log.debug("명령 시작: {}", command);
    try {
        run(command, args);
    } catch (IOException | RuntimeException e) {
        // 예외는 마지막 인자로 넘긴다.
        // 그래야 스택 트레이스까지 남는다(2.4절)
        log.error("명령 실패: {}", command, e);
        // 사용자에게 보여 줄 말은 로그가 아니라 프로그램의 출력이다
        System.out.println("명령을 실행하지 못했습니다. "
                + "자세한 내용은 로그를 확인해 주세요.");
    }
}
```

`catch (IOException ¦ RuntimeException e)`는 둘 중 어느 예외든 같은 자리에서 잡는다. `log.error("명령 실패: {}", command, e)`에서 `{}`는 하나인데 인자는 둘이다. SLF4J는 마지막 인자가 예외이면 메시지에 넣을 값이 아니라 예외로 보고, 스택 트레이스를 함께 남긴다.

```
… (앞줄 생략: DEBUG 명령 시작 로그 한 줄)
21:41:30.817 [main] ERROR net.java21.blog.BlogMain -- 명령 실패: list
java.io.IOException: 글 파일이 JSON 형식이 아니다: posts.json
    at net.java21.blog.PostStore.load(PostStore.java:49)
    … (2줄)
Caused by: tools.jackson.core.exc.UnexpectedEndOfInputException:
        Unexpected end-of-input: expected close marker for Object (…)
 at […] (through reference chain: java.util.ArrayList[1])
    … (Jackson 줄 열네 개와 PostStore 줄 하나)
명령을 실행하지 못했습니다. 자세한 내용은 로그를 확인해 주세요.
```

이 단계에는 아직 `logback.xml`이 없어 `DEBUG`도 출력되므로, `ERROR` 줄 앞에 `DEBUG … 명령 시작: list` 한 줄이 먼저 나온다. 앞의 출력에서는 그 줄을 생략했다.

`ERROR` 줄 다음 줄은 내 코드가 감싼 예외다. `PostStore`는 Jackson이 던진 예외 `e`를 잡아 `throw new IOException("글 파일이 JSON 형식이 아니다: " + file, e)`로 다시 던진다. 생성자의 둘째 인자로 넘긴 `e`가 새 예외의 원인(cause)이 되고, 스택 트레이스에는 `Caused by:` 줄로 출력된다. 이렇게 감싸면 "무엇을 하다가"(글 파일 읽기)와 "왜"(JSON이 중간에 끝남)가 한 로그에 함께 남는다.

진짜 원인은 `Caused by:` 아래에 있다(지면이 좁아 `Caused by:` 줄을 나눠 실었다). 객체를 닫는 `}`가 나오기 전에 JSON이 끝나 버렸고(`expected close marker for Object`), 그곳이 목록의 둘째 글(`ArrayList[1]`, 번호는 0부터 센다)이라는 뜻이다. 원인이 여러 겹이면 `Caused by:`가 여러 번 나오고, 맨 아래에 나온 원인이 근본 원인이다.

`System.out`도 하나 남았다. 사용자에게 보여 줄 안내다. 로그는 개발자에게, 출력은 사용자에게 간다.

### 남기면 안 되는 값

로그에는 비밀번호, 토큰, 개인정보를 남기지 않는다. 객체를 통째로 넘기는 것도 조심한다. 2.1절의 `DEBUG 저장:` 출력은 record의 `toString()`을 거쳐 본문까지 내보냈다. 로그는 생각보다

멀리 간다. 파일로 남아 다른 사람이 읽고, 누군가 문제를 물을 때 그대로 붙여 넣기도 한다. 그래서 번호(`id=2`)처럼 꼭 필요한 것만 남긴다.



## 2.5 매번 쓰는 한 줄: Lombok의 @Slf4j

### 직접 해 보기 2 로거 선언을 annotation으로

**브랜치:** `step2-exercise`에서 시작, `step2-solution`과 비교

**목표:** 클래스마다 쓰는 로거 선언을 Lombok `@Slf4j`로 바꾼다.

**왜 필요한가:** 클래스를 만들 때마다 로거 선언을 복사해 붙여 넣어야 한다. 복사한 뒤 클래스 이름을 고치지 않으면 로그에 엉뚱한 클래스 이름이 남는다.

1. 2단계 실습 브랜치로 전환한다.

```
터미널
$ git switch step2-exercise
```

2. `pom.xml`에 Lombok 의존성(`provided`)과 annotation processor(컴파일할 때 코드를 만들어 넣는 프로그램, 이 절에서 이어서 본다) 설정을 넣는다(`TODO#1`).
3. `BlogMain`과 `PostStore`에 `@Slf4j`를 붙인다(`TODO#2`).
4. 빌드한다.

```
터미널
$ mvn package
```

5. 다음 명령으로 컴파일된 `PostStore` 클래스의 필드를 본다. 출력에 `private static final org.slf4j.Logger log;`가 있으면 Lombok이 로거 선언을 만들었다는 뜻이다. 다만 실습 브랜치에는 직접 쓴 같은 필드가 이미 있어서, TODO를 채우기 전에도 출력은 똑같다. 소스에서 직접 쓴 로거 선언을 지운 뒤에도 이 필드가 있는지 확인한다.

```
터미널
$ javap -p -cp target/classes net.java21.blog.PostStore
```

**생각해 보기:** Lombok 의존성은 `provided` scope로 넣는다. 왜 실행할 때는 Lombok이 없어도 될까?

두원은 클래스를 만들 때마다 로거 선언을 복사해 붙여 넣었다.

```
private static final Logger log = LoggerFactory.getLogger(PostStore.class);
```

한 번은 복사한 뒤 클래스 이름을 고치지 않아, `BlogMain`의 로그가 `PostStore` 이름으로 남았다.

> **마르코** 흔한 일이야. 이 선언을 누가 대신 써 주면 좋겠지?

Lombok은 컴파일할 때 코드를 만들어 주는 라이브러리다. 클래스에 `@Slf4j`를 붙이면 앞에서 본 로거 선언을 Lombok이 만들어 넣는다. 실습 브랜치의 `PostStore`에는 직접 쓴 로거 선언 바로 앞에 `TODO#2`가 있다. `BlogMain`도 같다.

**실습 브랜치의 모습** `PostStore.java`: 로거 선언을 `@Slf4j`로 바꿀 자리

```
// 글 목록을 JSON 파일 하나에 읽고 쓴다
public class PostStore {

    // TODO#2 아래 로거 선언을 지우고 클래스에 @Slf4j를 붙인다.
    //        (PostStore와 BlogMain 두 곳)
    //        (2.5절) Lombok이 컴파일할 때 아래 줄과 같은 log 필드를 만들어
    //        넣는다. 쓰지 않게 된 import 두 줄(Logger, LoggerFactory)도 지운다.
    //        힌트: import lombok.extern.slf4j.Slf4j;
    private static final Logger log = LoggerFactory.getLogger(PostStore.class);
```

`TODO#2`는 직접 쓴 로거 선언을 지우고 클래스 선언 앞에 `@Slf4j`를 붙이는 자리다. 이 annotation이 동작하려면 `pom.xml`의 `TODO#1`(조금 뒤에 본다)도 채워야 한다.

`TODO#2`를 직접 채운 뒤 이어지는 코드와 비교해 보자.

두원이 고친 `PostStore`에서 바뀐 부분은 이렇다.

```
-import org.slf4j.Logger;
-import org.slf4j.LoggerFactory;
+import lombok.extern.slf4j.Slf4j;

 // 글 목록을 JSON 파일 하나에 읽고 쓴다
+// Lombok이 컴파일할 때 로거 필드(log)를 만들어 넣는다(2.5절)
+@Slf4j
 public class PostStore {
-
-    private static final Logger log = LoggerFactory.getLogger(PostStore.class);
```

`Logger`와 `LoggerFactory`를 가져오던 `import` 두 줄도 지운다. 남겨 둬도 컴파일은 되지만, 쓰지 않는 `import`는 읽는 사람을 헷갈리게 한다. `BlogMain`도 같은 방법으로 고친다.

로거 선언을 만들어 넣는 일은 annotation processor가 맡는다. 컴파일러가 소스를 읽는 도중에 함께 실행되어 코드를 추가하는 프로그램이다. 실습 브랜치의 `pom.xml`에는 로깅 의존성 다음에 그 설정을 넣을 자리가 `TODO#1`로 비어 있다.

**실습 브랜치의 모습** `pom.xml`: Lombok을 넣을 자리

```
    <!-- 실제로 로그를 출력하는 구현체 -->
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.6.5</version>
    </dependency>
    <!-- TODO#1 Lombok을 provided scope로 더하고,
               maven-compiler-plugin에 annotation processor로 등록한다.
               (2.5절, 예제 2-5) 의존성은 org.projectlombok:lombok 1.18.48,
               scope는 provided.
               (2.5절, 예제 2-6) annotation processor 설정은
               아래 <build><plugins> 안에 maven-compiler-plugin 3.16.0을
               새로 적고 annotationProcessorPaths에 lombok을 넣는다. -->
</dependencies>
```

`TODO#1`에서 할 일은 두 곳으로 나뉜다. `<dependencies>`에는 Lombok을 `provided` scope로 적고(예제 2-5), `<build><plugins>`에는 maven-compiler-plugin을 새로 적어 Lombok을 annotation processor로 알려 준다(예제 2-6).

`TODO#1`을 직접 채운 뒤 예제 2-5, 예제 2-6과 비교해 보자.

두원이 적은 Lombok 의존성은 다음과 같다.

**예제 2-5** `pom.xml`: Lombok 의존성

```
<!-- 컴파일할 때만 쓴다. JAR에는 들어가지 않는다 -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.48</version>
  <scope>provided</scope>
</dependency>
```

`provided` scope는 원래 "실행할 때는 실행 환경이 준다고 보고 JAR에 넣지 않는다"는 뜻이다. Lombok은 실행 환경이 주는 라이브러리가 아니다. 하지만 컴파일이 끝나면 더 할 일이 없다. "컴파일할 때만 쓰고 JAR에 넣지 않는다"는 점이 `provided`와 같아서 이 scope를 빌려 쓴다.

예제 2-6은 컴파일러 플러그인에 annotation processor를 알려 주는 설정으로, `pom.xml`의 `<build><plugins>` 안에 넣는다.

`maven-compiler-plugin`은 1.6절의 출력에서 `compiler:…:compile`로 본 바로 그 플러그인이다. 지금까지는 Maven이 정한 버전을 썼지만, 설정을 추가하려면 `pom.xml`에 이름과 버전을 적어야 한다. 실습 저장소는 1.6절에서 본 3.16.0(1장에서 설치한 Maven 3.10.0이 정한 버전)으로 고정했다.

**예제 2-6** `pom.xml`: annotation processor 설정

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.16.0</version>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.48</version>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

두원은 처음에 이 설정을 빠뜨렸다. JDK 21에서는 그래도 빌드가 된다. 다만 `mvn package` 출력에 `Annotation processing is enabled because…`로 시작하는 안내가 섞여 나온다. 지금 JDK는 classpath에 있는 processor를 알아서 실행해 주지만, 앞으로 나올 JDK는 processor를 따로 지정하지 않으면 실행하지 않을 수 있다는 안내다. 실제로 JDK 23부터는 기본 동작이 그렇게 바뀌었다. JDK 버전을 올린 날 갑자기 `log`를 찾지 못해 빌드가 실패하지 않도록, processor가 들어 있는 JAR를 이렇게 따로 적어 둔다.

정말로 코드가 생겼는지는 컴파일된 클래스를 열어 보면 알 수 있다. `javap -p`는 클래스 파일의 필드와 메서드를 보여 준다.

터미널

```
$ javap -p -cp target/classes net.java21.blog.PostStore
Compiled from "PostStore.java"
public class net.java21.blog.PostStore {
  private static final org.slf4j.Logger log;
  private final java.nio.file.Path file;
  private final tools.jackson.databind.ObjectMapper mapper;
  … (생성자와 메서드)
}
```

소스에는 없는 `log` 필드가 클래스 파일에는 있다. 이 출력은 실습 브랜치에서 TODO를 채우기 전에도 같다. 그때는 직접 쓴 선언이 있었기 때문이다. 그래서 확인할 것은 두 가지다. 소스에서 `LoggerFactory`를 쓰는 줄이 사라졌는지, 그리고 그 상태로 빌드한 클래스 파일에 이 필드가 있는지다. 앞의 것은 다음 명령으로 확인한다. 아무것도 출력되지 않으면 된다.

터미널 · macOS/Linux Windows 736쪽

```
$ grep -rn LoggerFactory src/main/java
```

### TIP 편집기의 Lombok 지원

편집기가 `log`에 빨간 밑줄을 표시한다면 편집기의 Lombok 지원을 확인한다. IntelliJ IDEA와 VS Code의 Java 확장 묶음에는 기본으로 들어 있고, Eclipse는 lombok.jar로 따로 설치한다.

### Lombok이 대신 써 주는 코드

Lombok에는 `@Slf4j` 말고도 자주 쓰는 annotation이 여럿 있다. 모두 자바에서 직접 반복해 쓰던 코드를 대신 만들어 준다. 예제 2-7은 회원 정보를 담는 클래스를 직접 쓴 코드다.

**예제 2-7** Member.java: 직접 쓴 클래스

```
public class Member {

    private final String name;
    private String email;

    public Member(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return "Member(name=" + name + ", email=" + email + ")";
    }
}
```

필드는 둘인데 코드는 스무 줄 가까이 된다. 필드가 하나 늘면 생성자와 getter(게터, 필드 값을 돌려주는 `getX()` 메서드), setter(세터, 필드 값을 바꾸는 `setX()` 메서드), `toString()`도 함께 고쳐야 한다. 예제 2-8은 같은 클래스를 Lombok으로 쓴 코드다.

**예제 2-8** Member.java: Lombok으로 쓴 클래스

```
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.Setter;
import lombok.ToString;

@Getter
@ToString
@AllArgsConstructor
public class Member {

    private final String name;
    @Setter private String email;
}
```

컴파일하면 예제 2-7과 같은 생성자와 메서드가 클래스 파일에 생긴다. annotation이 하는 일은 표 2-3처럼 대개 이름 그대로다.

**표 2-3** 자주 쓰는 Lombok annotation

| annotation | 만들어 주는 것 |
|---|---|
| `@Getter` | 필드마다 getter(`getName()`, `getEmail()`) |
| `@Setter` | setter. 필드에 붙이면 그 필드만(`setEmail()`) |
| `@ToString` | 필드 값을 보여 주는 `toString()` |
| `@AllArgsConstructor` | 모든 필드를 받는 생성자 |
| `@RequiredArgsConstructor` | `final` 필드만 받는 생성자 |
| `@EqualsAndHashCode` | 필드 값으로 비교하는 `equals()`와 `hashCode()` |
| `@Data` | `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode`, `@RequiredArgsConstructor`를 한 번에 |
| `@Builder` | `Member.builder().name("두원").build()`처럼 쓰는 builder |
| `@Slf4j` | SLF4J 로거 선언(`log` 필드) |

### 참고 record와 Lombok

자바 16부터는 값을 담는 클래스를 record로 쓸 수 있다. 1장에서 만든 `Post`가 record다. record는 생성자와 값을 꺼내는 메서드, `equals()`, `hashCode()`, `toString()`을 자바가 만들어 준다. 만든 뒤 바뀌지 않는 값이라면 Lombok 없이 record로 충분하다. 이 책에서도 값은 record로 쓰고, Lombok은 `@Slf4j`만 쓴다.

> **마르코** 편하지? 그럼 반대로, 이게 불편해지는 때는 언제일까?
>
> **두원** 음… 소스에 없는 코드가 생기니까, 읽는 사람이 Lombok을 모르면 `log`가 어디서 왔는지 모르겠네.

바로 그 점이 판단 기준이 된다. `@Slf4j`처럼 하는 일이 한 줄로 분명한 annotation은 얻는 것이 크다. `@Data`처럼 한 번에 많은 코드를 만드는 annotation은 조심해서 쓴다. `@Data`는 getter,

setter에 `equals()`, `hashCode()`, `toString()`까지 만든다. 그래서 비밀번호 필드가 있는 클래스라면 `log.debug("{}", user)` 한 줄로 비밀번호가 로그에 남는다. `equals()`가 무엇을 비교하는지도 코드에 보이지 않는다.

**! 주의 모르게 들어오는 Lombok**

예제 코드를 복사해 오거나 다른 사람의 변경을 합치다 보면, 나도 모르게 Lombok이 딸려 들어오는 일이 흔하다. 클래스 선언에 `@Data`, `@Builder`가 붙고 `pom.xml`에 의존성이 생긴다. 그래서 다른 사람의 코드를 가져온 뒤에는 `git diff pom.xml`부터 보고, 그 annotation이 무엇을 만드는지 설명할 수 있는지 스스로 물어본다.

## 2.6 logback.xml: pattern, appender, logger

### 직접 해 보기 3 logback.xml로 정하기

**브랜치:** `step3-exercise`에서 시작, `step3-solution`과 비교

**목표:** `src/main/resources/logback.xml`로 형식, 내보낼 곳, 레벨을 코드 밖에서 정한다.

**왜 필요한가:** 기본 설정은 `DEBUG` 이상을 콘솔에 출력한다. 로그가 여전히 글 목록과 섞이고, 남기지 않게 할 수도 없다.

1. 3단계 실습 브랜치로 전환한다.

```
터미널
$ git switch step3-exercise
```

2. `src/main/resources/logback.xml` 틀의 `TODO#1`~`#4`를 채워 콘솔과 파일 두 곳(appender), 스레드 이름이 들어간 형식, 내 코드와 그 밖의 로거 레벨을 정한다.
3. `list > out.txt`로 저장한 파일에는 글 목록만 남고, `logs/blog.log`에는 `INFO` 로그가 쌓이는지 확인한다.
4. 레벨을 `DEBUG`로 바꿔 다시 빌드하고, 같은 명령에서 로그가 늘어나는지 확인한다.

**생각해 보기:** `logback.xml`은 코드 밖에 있는데도 레벨을 바꾸려면 다시 빌드해야 했다. 다시 빌드하지 않고 레벨을 바꾸려면 이 파일이 어디에 있어야 할까?

2.3절의 로그는 아직 표준 출력으로 나가 목록과 섞이고, `DEBUG`까지 모두 나온다. 로그를 어디로, 어느 레벨부터 내보낼지 정하는 곳이 `logback.xml`이다(예제 2-9). `src/main/resources`에 두면 빌드할 때 JAR 안으로 들어가고, Logback이 시작하면서 찾아 읽는다. 실습 브랜치의 이 파일에는 appender(어펜더, 로그를 내보낼 곳 하나하나) 두 개의 뼈대와 `TODO#1`~`#4`만 적혀 있다.

**실습 브랜치의 모습** `src/main/resources/logback.xml`: appender 두 개의 뼈대와 TODO

```
<configuration>
  <!-- 이 파일은 빈 틀이다. TODO#1~#4를 채워 2.6절의 예제 2-9를 완성한다.
       채우기 전에도 컴파일은 되지만, Logback이 설정 오류를 표준 출력에 찍고
       LogbackConfigTest가 실패하며 남은 TODO 번호를 알려 준다. -->

  <!-- 콘솔: 터미널로 내보내는 appender -->
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <!-- TODO#1 표준 출력이 아니라 표준 오류로 내보내고,
              WARN 이상만 터미널에 보이게 거른다.
              (2.6절 "appender: 어디로 보낼까")
              그래야 list > out.txt의 파일에 글 목록만 남는다.
              힌트: <target>,
              ch.qos.logback.classic.filter.ThresholdFilter의 <level> -->
    <encoder>
      <!-- TODO#3 한 줄의 모양(pattern)을 정한다.
                시각, 스레드 이름, 레벨, 클래스 이름, 메시지 순서.
                (2.6절 "pattern: 한 줄의 모양", 그림 2-5)
                터미널에는 짧게, 패키지 없이 클래스 이름만.
                힌트: %d{시각 형식}, %thread, %-5level, %logger{길이},
                %m, %n을 이어 붙인다 -->
      <!-- 터미널이 쓰는 문자 인코딩을 따른다(Windows 기본 창은 MS949) -->
      <charset>${stderr.encoding:-UTF-8}</charset>
    </encoder>
  </appender>

  <!-- 파일: 로그를 파일에 쌓는 appender -->
  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <!-- TODO#2 logs/blog.log 파일에 남긴다. 필터는 달지 않는다.
              (2.6절 "appender: 어디로 보낼까")
              경로는 명령을 실행한 폴더 기준이다.
              힌트: <file> -->
    <encoder>
      <!-- TODO#3 파일 쪽 pattern. 밀리초와 로거 이름(20자까지)도 남긴다.
                힌트: %d{HH:mm:ss.SSS}, %logger{20} -->
    </encoder>
  </appender>
```

```
    <!-- TODO#4 내 코드(net.java21 패키지)는 INFO부터,
              그 밖의 로거는 WARN부터 남긴다.
              appender 두 개는 root에만 연결한다.
              (2.6절 "logger와 root: 어느 레벨부터 남길까",
              "같은 로그가 두 번 남을 때")
              힌트: <logger name="…" level="…"/>, <root level="…">,
              <appender-ref ref="…"/> -->
</configuration>
```

채울 곳은 네 가지다. `TODO#1`은 콘솔 appender가 표준 오류로 내보내고 `WARN` 이상만 통과시키게 하는 자리다(`<target>`과 `ThresholdFilter`). `TODO#2`는 파일 appender가 쓸 파일을 정한다(`<file>`). `TODO#3`은 두 appender의 한 줄 모양을 정한다(`<pattern>`, 터미널은 짧게 클래스 이름만, 파일은 밀리초와 로거 이름까지). `TODO#4`는 내 코드와 그 밖의 로거 레벨을 정하고, appender를 `root`에 연결한다(`<logger>`, `<root>`, `<appender-ref>`). 각 설정이 무슨 뜻인지는 예제 2-9 뒤에서 차례로 설명한다. 막히면 그 설명을 먼저 읽어도 된다.

`TODO#1`부터 `TODO#4`까지 직접 채운 뒤 예제 2-9와 비교해 보자.

두원이 완성한 `logback.xml`은 다음과 같다.

**예제 2-9** src/main/resources/logback.xml

```
<configuration>
  <!-- 콘솔: 표준 오류로 내보낸다. 프로그램의 출력과 섞이지 않는다 -->
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <target>System.err</target>
    <!-- 터미널에는 WARN 이상만 보인다 -->
    <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
      <level>WARN</level>
    </filter>
    <encoder>
      <pattern>%d{HH:mm:ss} [%thread] %-5level %logger{0} - %m%n</pattern>
      <!-- 터미널이 쓰는 문자 인코딩을 따른다(Windows 기본 창은 MS949) -->
      <charset>${stderr.encoding:-UTF-8}</charset>
    </encoder>
  </appender>

  <!-- 파일: 밀리초와 로거 이름까지 남긴다. 다시 실행하면 이어서 쓴다 -->
  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/blog.log</file>
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{20} - %m%n</pattern>
```

```
    </encoder>
  </appender>

  <!-- 내 코드(net.java21 패키지)는 INFO부터 -->
  <logger name="net.java21" level="INFO"/>

  <!-- 그 밖의 모든 로거(라이브러리 포함)는 WARN부터 -->
  <root level="WARN">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="FILE"/>
  </root>
</configuration>
```

예제 2-9는 차례대로 세 부분으로 나뉜다.

**appender: 어디로 보낼까.** 콘솔 appender는 표준 출력이 아니라 2.2절에서 본 표준 오류(System.err)로 보낸다. 그래서 `>`로 출력을 파일에 저장해도 로그는 터미널에 남는다. 필터를 달아 `WARN` 이상만 보이게 했다. 파일 appender는 필터 없이 `logs/blog.log`에 모두 쌓는다. 이 경로는 명령을 실행한 폴더 기준이라, 저장소 폴더에서 실행하면 그 아래 `logs/`에 생긴다.

encoder는 로그 한 건을 정해진 형식의 문자열로 만들고, 그 문자열을 바이트로 바꿔 appender에 넘기는 부품이다. 형식은 그 안의 `<pattern>`이 정한다.

콘솔 쪽 encoder의 `charset`은 터미널로 글자를 내보낼 때 쓰는 인코딩이다. JDK 18부터 기본 인코딩이 UTF-8이라, 이 줄이 없으면 Logback은 로그를 UTF-8로 내보낸다. 한국어 Windows의 기본 터미널 창은 MS949(CP949)로 읽으므로 로그의 한글이 `濮 ?뚯뵆…`처럼 잘못 표시된다. `${stderr.encoding:-UTF-8}`은 자바가 알아낸 표준 오류 쪽 터미널의 인코딩을 쓰고, 알 수 없으면 UTF-8을 쓰라는 뜻이다. macOS와 Linux 터미널은 UTF-8이라 달라지는 것이 없다. 자바는 표준 출력에는 처음부터 터미널의 인코딩을 쓴다. 그래서 같은 창에서도 `System.out.println`의 한글은 제대로 나왔다. 이 책의 실습 저장소에 있는 `logback.xml`에는 모두 이 줄이 들어 있다.

**pattern: 한 줄의 모양.** 그림 2-5는 로그 한 줄의 각 부분이 pattern의 어느 조각에서 나왔는지 보여 준다.

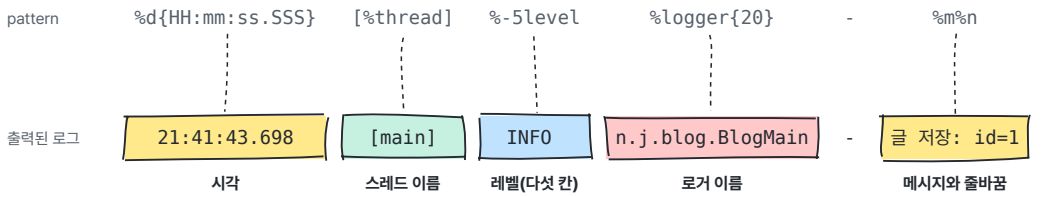



**그림 2-5** pattern의 조각마다 로그 한 줄의 한 부분이 채워진다

`%d{HH:mm:ss.SSS}`는 시각, `%thread`는 스레드 이름, `%-5level`은 다섯 칸에 맞춘 레벨, `%logger{20}`은 로거 이름, `%m`은 메시지, `%n`은 줄바꿈이다. 이름이 20자를 넘으면 앞쪽 패키지를 줄여 `n.j.blog.PostStore`처럼 출력한다. 콘솔 쪽의 `%logger{0}`은 패키지를 모두 빼고 클래스 이름만 남긴다. 날짜까지 남기려면 `%d`만 쓴다.

**logger와 root: 어느 레벨부터 남길까.** 로거 이름은 점으로 나뉜 계층이다. `net.java21`에 정한 레벨은 그 아래의 `net.java21.blog.BlogMain`과 `net.java21.blog.PostStore`에 모두 적용된다. 나머지 로거는 맨 위의 `root`를 따른다. 그러니 이 설정은 "내 코드는 `INFO`부터, 라이브러리는 `WARN`부터"라는 뜻이다. 라이브러리의 `DEBUG` 로그에 내 로그가 묻히지 않는다.

빌드하고 같은 명령을 다시 실행했다. 앞 단계에서 쓴 글이나 2.4절에서 망가뜨린 파일이 남아 있으면 결과가 달라지므로, 먼저 `posts.json`을 지운다.

터미널 · macOS/Linux Windows 736쪽

```
$ rm -f posts.json
```

실행 결과

```
$ java -jar target/blog-1.0.jar write "첫 글" "안녕"
1번 글을 저장했습니다.
$ java -jar target/blog-1.0.jar write "둘째 글" "반가워"
2번 글을 저장했습니다.

$ java -jar target/blog-1.0.jar delete 1
1번 글을 삭제했습니다.
$ java -jar target/blog-1.0.jar delete 7
7번 글이 없습니다.
$ java -jar target/blog-1.0.jar list > out.txt
$ cat out.txt
2. 둘째 글 (2026-10-19T21:41:44)
$ cat logs/blog.log
21:41:43.698 [main] INFO  n.j.blog.BlogMain - 글 저장: id=1
21:41:44.912 [main] INFO  n.j.blog.BlogMain - 글 저장: id=2
21:41:45.704 [main] INFO  n.j.blog.BlogMain - 글 삭제: id=1, 남은 글 1개
21:41:46.482 [main] INFO  n.j.blog.BlogMain - 지울 글이 없음: id=7
```

터미널과 목록 파일에는 프로그램이 보여 주는 결과만 남고, 로그는 파일에 쌓였다.

`WARN` 이상인 로그는 터미널에도 보인다. `posts.json`을 비운 뒤 목록을 보며 확인해 보자. 파일은 다음 명령으로 비운다. macOS와 Linux의 `: >`는 아무것도 쓰지 않고 파일을 비우는 셸 문법이다.

터미널 · macOS/Linux Windows 736쪽

```
$ : > posts.json
```

실행 결과

```
$ java -jar target/blog-1.0.jar list
21:41:47 [main] WARN  PostStore - 글 파일이 비어 있어 빈 목록으로 시작: …
```

`list`는 파일에 쓰지 않으므로 `posts.json`은 빈 채로 남는다. 다음 실험의 출력과 맞추려면, 레벨을 바꾸기 전에 `posts.json`을 지우고 `write`를 한 번 실행해 글 하나를 만들어 둔다. `posts.json`은 Git이 관리하지 않는 파일이라 `git restore`로는 되돌릴 수 없다.

터미널 · macOS/Linux Windows 736쪽

```
$ rm -f posts.json
$ java -jar target/blog-1.0.jar write "첫 글" "안녕"
```

### 레벨을 바꾸면 남는 로그가 바뀐다

다시 번호 버그 같은 일이 생겼다고 해 보자. 이번에는 `println`을 넣지 않는다. 내 코드의 레벨을 정한 한 줄만 바꾼다.

```
-  <logger name="net.java21" level="INFO"/>
+  <logger name="net.java21" level="DEBUG"/>
```

`logback.xml`은 JAR 안에 들어가므로 다시 빌드한다. JAR를 다시 만들지 않고 레벨을 바꾸려면 `logback.xml`을 JAR 밖에 두고, 실행할 때 `-Dlogback.configurationFile=파일경로`로 그 위치를 알려 준다. `INFO`일 때 `write`와 `delete` 두 명령은 "글 저장", "글 삭제" 로그 두 개를 남겼다. `DEBUG`로 바꾸고 같은 두 명령(`write "둘째 글" "반가워"`와 `delete 1`)을 실행하자 여덟 개가 되었다. 앞의 네 개가 `write`의 로그다.

```
21:42:04.680 [main] DEBUG n.j.blog.BlogMain - 명령 시작: write
21:42:04.949 [main] DEBUG n.j.blog.PostStore - 글 1개를 읽음: posts.json
21:42:04.971 [main] DEBUG n.j.blog.PostStore - 글 2개를 저장함: posts.json
21:42:04.971 [main] INFO  n.j.blog.BlogMain - 글 저장: id=2
… (delete도 같은 모양으로 네 개)
```

레벨을 `DEBUG`로 바꾼 뒤 `mvn package`를 실행하면, 테스트가 남긴 `DEBUG` 로그도 같은 `logs/blog.log`에 기록된다. `src/main/resources`의 `logback.xml`이 테스트를 실행할 때도 쓰이기 때문이다. 그래서 파일에는 이 로그들보다 먼저 `글 1개를 저장함: /tmp/junit-…/posts.json` 같은 줄이 섞여 있을 수 있다. 테스트가 만든 임시 폴더의 파일이라 이 실험과는 상관없다.

코드는 한 글자도 바뀌지 않았다. 의심 가는 클래스 하나만 보고 싶다면 그 로거의 레벨만 내린다.

```
<logger name="net.java21" level="INFO"/>
<logger name="net.java21.blog.PostStore" level="DEBUG"/>
```

이렇게 하면 `BlogMain`의 "명령 시작" 로그는 빠지고 `PostStore`의 `DEBUG` 로그만 나온다. 그림 2-6처럼, 로거는 자기 이름에 정한 레벨이 없으면 가장 가까운 상위 로거의 레벨을 물려받는다. 그리고 더 구체적인 이름에 정한 레벨이 우선한다. 원인을 찾고 나면 그 설정을 지운다.

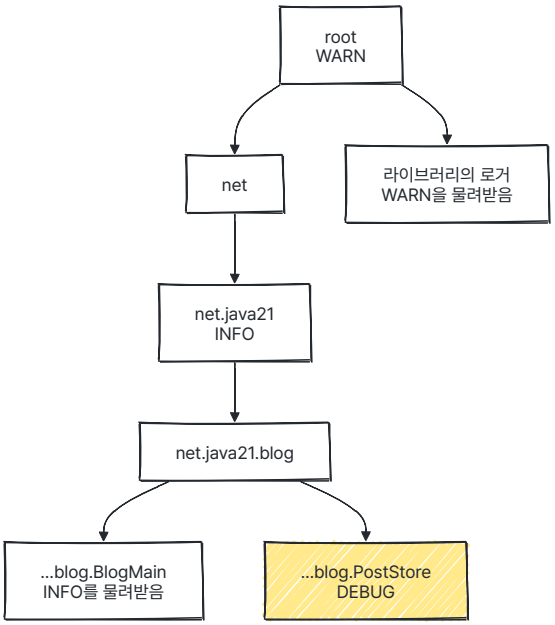



**그림 2-6** BlogMain은 net.java21의 INFO를 물려받고, 레벨을 따로 정한 PostStore만 DEBUG가 된다

로그를 거르는 곳이 두 군데라는 것도 눈여겨보자. 로거의 레벨은 "이 로그를 만들까"를 정한다. 콘솔 appender의 필터는 "만든 로그 가운데 무엇을 터미널로 보낼까"를 정한다. 그래서 `INFO` 로그는 만들어지지만 파일에만 간다. 그림 2-7은 프로그램의 출력과 로그 하나가 각각 어디로 전달되는지 보여 준다.

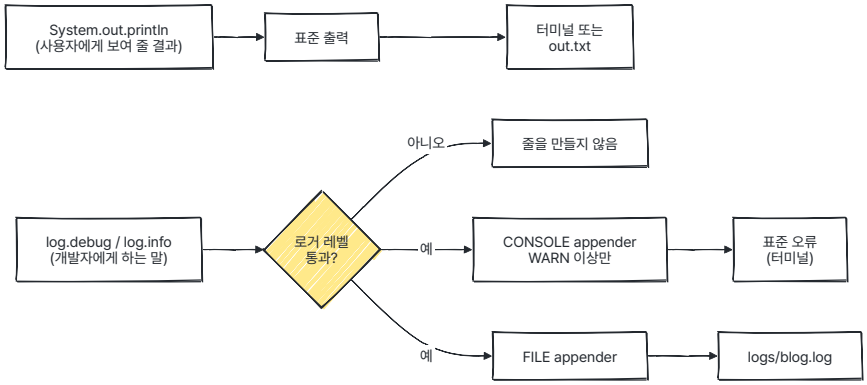



**그림 2-7** 프로그램이 보여 주는 결과는 표준 출력으로, 로그는 로거 레벨과 appender를 거쳐 표준 오류와 로그 파일로 간다

### 같은 로그가 두 번 남을 때: additivity

두원은 자기 코드의 로그도 파일에 남기고 싶어서, `net.java21` 로거에도 파일 appender를 연결해 보았다. 읽기만 해도 된다. 직접 해 보려면 끝난 뒤 이 줄을 원래대로 되돌린다.

```
<logger name="net.java21" level="INFO">
  <appender-ref ref="FILE"/>
</logger>
```

그러자 로그 파일에 같은 로그가 두 번씩 기록되었다.

```
21:42:04.971 [main] INFO  n.j.blog.BlogMain - 글 저장: id=2
21:42:04.971 [main] INFO  n.j.blog.BlogMain - 글 저장: id=2
```

이유는 로그가 전달되는 방식에 있다. 그 경로는 그림 2-8과 같다.

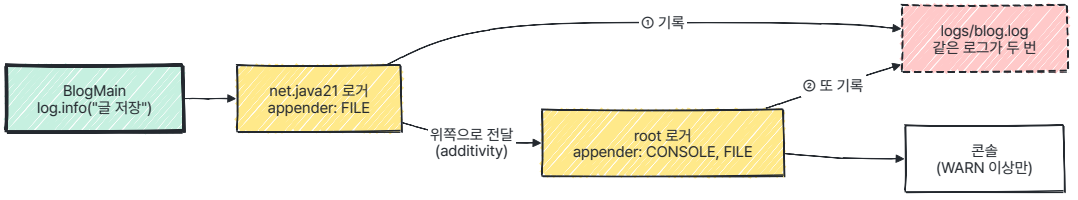



**그림 2-8** 로그는 net.java21 로거의 FILE에 한 번, root로 올라가 root의 FILE에 또 한 번 기록된다

`BlogMain`에서 남긴 로그는 먼저 `net.java21` 로거로 가서, 거기 연결된 `FILE` appender에 한 번 기록된다. 그런 다음 위쪽의 `root` 로거로도 전달되고, `root`에도 `FILE` appender가 연결되어 있으니 한 번 더 기록된다. 로그가 위쪽 로거로 계속 전달되는 이 동작을 **additivity**라고 한다.

고치는 방법은 두 가지다.

- **appender는 `root`에만 연결한다.** 로거에는 레벨만 적는다(예제 2-9). 로그는 어차피 `root`까지 올라가므로 파일에 한 번만 기록된다. 이쪽이 더 단순하다.
- **로거에 `additivity="false"`를 설정한다.** `<logger name="net.java21" level="INFO" additivity="false">`처럼 쓰면 그 로거의 로그는 `root`로 올라가지 않는다. 이때 콘솔에도 남기려면 그 로거에 `CONSOLE` appender도 따로 연결해야 한다.

## 2.7 내 로그를 리뷰하기

여기까지는 코드를 두원이 직접 작성했다. `BlogMain`과 `PostStore`를 보여 주자, 마르코는 고칠 곳을 짚어 주는 대신 질문 세 개를 보냈다.

```
코드는 아직 고치지 말고 리뷰만 해 봐.
1. 로그마다 고른 레벨이 맞는지, 아니라면 왜인지.
2. 나중에 버그를 쫓을 때 빠진 정보가 무엇인지.
3. 남기면 안 되는 값이 로그에 들어가는지.
근거가 되는 로그를 함께 적어 봐.
```

두원은 이 세 질문을 코드와 함께 AI에게 붙여 넣고 리뷰를 부탁했다. 답은 금방 왔다. 레벨은 대부분 알맞고, "지울 글이 없음"을 `INFO`로 둔 것도 사용자가 잘못 넣은 값이니 괜찮다고 했다. 제목과 본문을 로그에 남기지 않은 점도 잘했다고 했다. 그리고 마지막에 제안을 하나 덧붙였다. "`ERROR` 로그에 스택 트레이스가 수십 줄씩 붙어 읽기 어렵습니다. 원인 메시지만 한 줄로 남기면 로그가 깔끔해집니다."

```
log.error("명령 실패: " + e.getMessage());
```

그럴듯했다. 두원도 스택 트레이스가 길어 거슬리던 참이었다. 그래도 바로 붙여 넣지 않고 먼저 실행해 보았다. 망가진 `posts.json`으로 `list`를 실행하자 로그 파일에는 한 줄만 남았다.

```
21:42:14.611 [main] ERROR n.j.blog.BlogMain - 명령 실패: 글 파일이 JSON 형식이…
```

`Caused by:`가 모두 사라져, JSON이 어디서 끊겼는지 알 수 없다. 제안 한 줄에 이 장에서 피하자고 한 두 가지가 함께 들어 있었다. 문자열 이어 붙이기와 예외 메시지만 남기기다. 급히 쓴 로그에서 자주 보는 실수가 하나 더 있다. 모든 로그를 `INFO`로 두는 것이다. 레벨을 고르는 기준 없이 "일단 다 남겨 두자"고 생각할 때 흔히 생긴다.

> **마르코** 붙여 넣기 전에 잡았네. 그 제안이 틀린 걸 어떻게 알았어?
>
> **두원** 실행해 봤지. 그리고 `Caused by:`가 왜 필요한지 내가 알고 있었으니까.
>
> **마르코** 그럼 `Caused by:`를 몰랐으면?
>
> **두원** …더 깔끔해졌다고 좋아했을 거야. 틀린 줄도 모르고.
>
> **마르코** AI 리뷰가 쓸모없다는 얘기는 아니야. 다만 제안을 받아들일지는 네가 정해야 하고, 그러려면 기준을 적어 둬야 해. 적어 두지 않으면 석 달 뒤의 너도, AI도 맞는지 틀린지 가릴 수 없거든.

기준을 알아야 내 코드도, 다른 사람의 코드도, AI가 제안한 코드도 판단할 수 있다. 이 책에서 프레임워크나 도구가 해 주는 일을 직접 먼저 해 보는 이유다. 오늘의 세 질문은 앞으로 로그가 들어간 코드를 리뷰할 때도, AI에게 리뷰를 부탁할 때도 그대로 쓸 수 있다.

## 2.8 스레드 두 개가 같은 파일에 쓸 때

### 직접 해 보기 4 스레드 둘의 로그 읽기

**브랜치:** `step4-exercise`에서 시작, `step4-solution`과 비교

**목표:** 옛 글 파일 여러 개를 파일마다 스레드를 하나씩 두어 동시에 읽고, 로그에서 스레드를 구분한다.

**왜 필요한가:** 일이 동시에 실행되면 로그가 뒤섞인다. 누가 남긴 로그인지 모르면 로그를 읽을 수 없다.

① 4단계 실습 브랜치로 전환한다.

터미널

```
$ git switch step4-exercise
```

② 파일마다 스레드를 시작해 이름을 붙이고, 모두 끝날 때까지 기다리는 `readAll`을 만든다(`TODO#1`). 그리고 `import` 명령에서 이 메서드를 부른다(`TODO#2`).

③ `import old-a.json old-b.json`을 실행한 뒤, 로그 파일의 스레드 칸에 `import-1`, `import-2`가 보이는지 확인한다.

**생각해 보기:** 두 스레드가 같은 파일에 로그를 쓸 때, 파일에 적힌 순서가 곧 일이 일어난 순서일까?

두원은 예전에 다른 곳에 쓰던 글을 옮겨 오고 싶었다. 내보낸 파일이 두 개, 글이 3,000개와 2,000개다. 실습 브랜치에는 두 파일 `old-a.json`, `old-b.json`과 중간을 잘라 망가뜨린 `old-c.json`이 들어 있다. 실습 브랜치의 `import` 명령은 파일을 하나씩 차례로 읽으므로, 로그의 스레드 칸이 모두 `[main]`이다.

2.3절에서 짧게 말한 대로, 스레드는 프로그램 안에서 코드를 차례로 실행하는 흐름이다. 한 프로그램에 스레드를 여럿 두면 여러 일을 동시에 진행할 수 있다. 두원이 만들려는 구조는 그림 2-9와 같다.

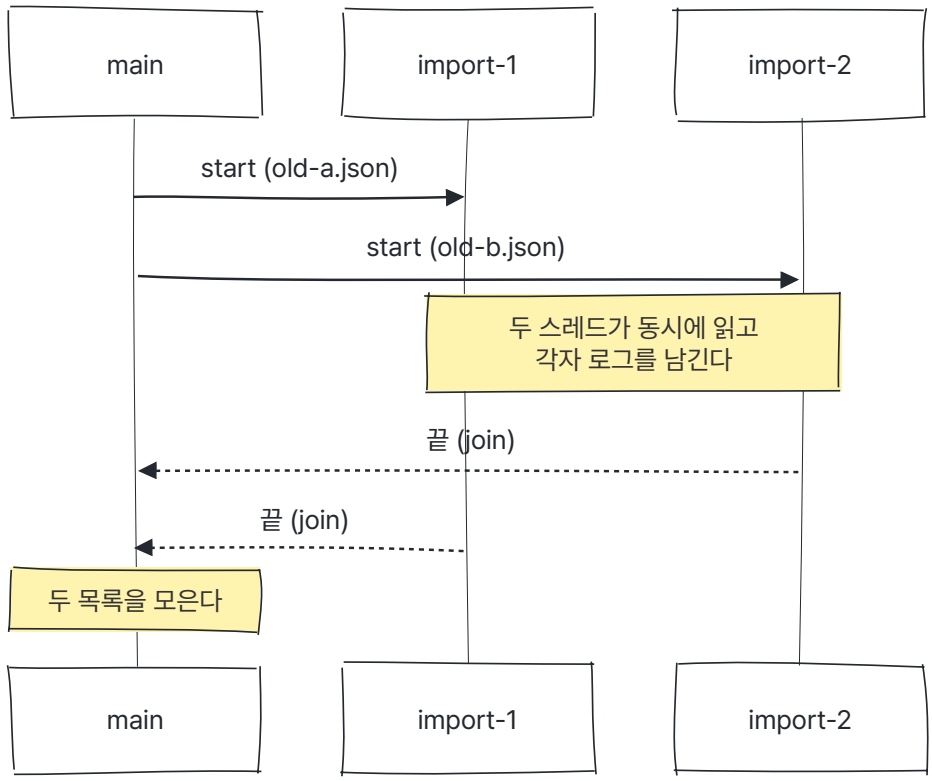



**그림 2-9** main은 파일마다 스레드를 시작하고, 두 스레드가 모두 끝나기를 기다린 뒤 결과를 모은다

실습 브랜치의 `BlogMain`에는 이 일을 하는 메서드를 만들 자리가 `TODO#1`로 비어 있다. 바로 다음에는 파일 하나를 읽는 `readFile`이 이미 있다.

**실습 브랜치의 모습** `BlogMain.java`: `readAll`을 만들 자리

```
// TODO#1 파일마다 스레드를 하나씩 시작해 동시에 읽고, 모두 끝나면 모으는
//        static List<Post> readAll(String[] args) 메서드를 여기에 만든다.
//        args[0]은 명령 이름이다.
//        (2.8절, 예제 2-10, 그림 2-9) 스레드마다 자기 목록에만 넣고,
//        main은 모두 끝난 뒤에 모은다. 스레드에는 "import-1",
//        "import-2"처럼 이름을 붙여 로그에서 구분한다.
//        힌트: new Thread(Runnable, 이름), start(), join(),
//        InterruptedException

// 예전 블로그에서 내보낸 파일 하나를 읽는다
```

```
static List<Post> readFile(Path file) {
    log.info("읽기 시작: {}", file);
    try {
        List<Post> posts = new PostStore(file).load();
        log.info("읽기 끝: {}, 글 {}개", file, posts.size());
        return posts;
    } catch (IOException e) {
        log.warn("옮기지 못한 파일: {}", file, e);
        return List.of();
    }
}
```

`TODO#1`은 `readAll`을 만드는 자리다. `args[1]`부터 들어 있는 파일마다 스레드를 하나씩 시작해 `readFile`을 실행하게 하고, 모든 스레드가 끝날 때까지 기다렸다가 결과를 모은다(`args[0]`은 명령 이름이다). 스레드마다 읽은 글을 자기 목록에만 넣게 하고, 스레드에는 `import-1`, `import-2`처럼 이름을 붙인다. 여러 스레드가 한 목록에 함께 넣으면 결과가 어긋날 수 있다. 왜 그런지는 5.7절에서 본다. 쓸 API는 `new Thread(할 일, 이름)`, `start()`, `join()`이다.

`TODO#1`을 직접 채운 뒤 예제 2-10과 비교해 보자.

두원은 파일마다 스레드를 하나씩 만들어 동시에 읽게 했다.

**예제 2-10** `BlogMain.java`: 파일마다 스레드 하나로 읽기(`readAll`)

```
// 파일마다 스레드를 하나씩 시작해 동시에 읽고, 모두 끝나면 모은다
static List<Post> readAll(String[] args) {
    List<List<Post>> results = new ArrayList<>();
    List<Thread> threads = new ArrayList<>();
    for (int i = 1; i < args.length; i++) {
        Path file = Path.of(args[i]);
        List<Post> result = new ArrayList<>();
        results.add(result);
        Thread t = new Thread(() -> result.addAll(readFile(file)),
                "import-" + i);
        threads.add(t);
        t.start();
    }
    for (Thread t : threads) {
        try {
            t.join();   // 그 스레드가 끝날 때까지 기다린다
        } catch (InterruptedException e) {
            // … (생략: 기다리다 끊기면 멈춘다)
        }
```

```
        }
        List<Post> all = new ArrayList<>();
        results.forEach(all::addAll);
        return all;
    }
```

---

`new Thread(할 일, 이름)`의 첫 인자는 새 스레드가 실행할 코드이고, 둘째는 스레드 이름이다. 첫 인자의 `() -> result.addAll(readFile(file))`은 2.1절에서 본 람다로, 인자를 받지 않고 화살표 뒤의 코드를 실행한다. `start()`는 새 스레드에서 그 일을 시작하고 곧바로 돌아오며, `join()`은 그 스레드가 끝날 때까지 기다린다.

기다리는 동안 다른 곳에서 이 스레드에 중단 요청(interrupt)을 보내면 `join()`이 `InterruptedException`을 던지므로 `try`로 감쌌다. 생략한 `catch` 블록의 내용은 완성 브랜치에서 `Thread.currentThread().interrupt();`와 `throw new IllegalStateException("옮기기를 멈췄다", e);` 두 줄이다. 첫 줄은 예외가 던져지면서 지워진 중단 요청 표시를 지금 스레드(`main`)에 다시 남겨, 뒤에 실행되는 코드도 중단 요청이 있었음을 알 수 있게 한다. 둘째 줄은 옮기기를 그 자리에서 멈춘다. 중단 요청을 다루는 법은 4장에서 다시 본다.

그림 2-9처럼 스레드마다 자기 목록(`result`)에만 넣고, `main`은 모두 끝난 뒤(`join`)에 모은다. `results.forEach(all::addAll)`의 `all::addAll`은 `list -> all.addAll(list)`를 줄여 쓴 메서드 참조다. 스레드마다 모은 목록을 `all`에 차례로 이어 붙인다.

`readAll`을 만들었으면 `import` 명령에서 부른다. 실습 브랜치의 `case "import"` 블록은 이렇게 되어 있다.

**실습 브랜치의 모습** `BlogMain.java`: `readAll`을 부를 자리

```
case "import" -> {
    List<Post> imported = new ArrayList<>();
    // TODO#2 아래 for 반복문(과 위의 빈 목록 선언)을
    //        TODO#1의 readAll 호출 한 줄로 바꾼다.
    //        (2.8절, 예제 2-10 다음 문단) 번호를 새로 매겨 저장하는
    //        아래 코드는 그대로 둔다.
    for (int i = 1; i < args.length; i++) {
        imported.addAll(readFile(Path.of(args[i])));
    }
    long id = nextId(posts);
    // … (생략: 옮긴 글에 새 번호를 매겨 저장하고, 결과를 알린다)
}
```

`TODO#2`는 파일을 하나씩 차례로 읽던 `for` 반복문(과 그 앞의 빈 목록 선언)을 `readAll` 호출 한 줄로 바꾸는 자리다. 그다음에 번호를 새로 매겨 저장하는 코드는 그대로 둔다.

`TODO#2`를 직접 채운 뒤 이어지는 코드와 비교해 보자.

두원이 바꾼 것은 `List<Post> imported = readAll(args);` 한 줄이다.

다시 빌드하고 옛 글 파일 두 개를 옮겨 본다.

터미널

```
$ mvn package
$ java -jar target/blog-1.0.jar import old-a.json old-b.json
```

터미널에는 `글 5000개를 옮겼습니다.` 한 줄만 나온다. `INFO` 로그는 2.6절의 설정대로 `logs/blog.log`에만 쌓이기 때문이다. 이 파일에는 앞 단계의 로그도 남아 있으므로 마지막 몇 줄만 본다.

터미널 · macOS/Linux Windows 737쪽

```
$ tail -n 5 logs/blog.log
```

이어지는 로그 두 묶음(스레드에 이름이 없을 때와 이름을 붙인 뒤)은 이 명령의 출력이다. 망가진 파일을 섞는 마지막 예는 터미널에 나온 그대로다.

처음에는 스레드에 이름을 붙이지 않았다(예제 2-10에서 `new Thread`의 둘째 인자를 뺀 모양). 그러면 자바가 `Thread-0`, `Thread-1` 같은 이름을 붙인다.

```
21:42:59.922 [Thread-1] INFO  n.j.blog.BlogMain - 읽기 시작: old-b.json
21:42:59.922 [Thread-0] INFO  n.j.blog.BlogMain - 읽기 시작: old-a.json
21:43:00.093 [Thread-1] INFO  n.j.blog.BlogMain - 읽기 끝: old-b.json, 글 20…
21:43:00.103 [Thread-0] INFO  n.j.blog.BlogMain - 읽기 끝: old-a.json, 글 30…
21:43:00.188 [main] INFO  n.j.blog.BlogMain - 옮기기 끝: 파일 2개, 글 5000개
```

먼저 시작한 쪽은 첫째 파일을 맡은 스레드였을 텐데, 둘째 파일을 맡은 스레드의 로그가 먼저 기록되었다. 실행할 때마다 순서가 바뀌고, 이 출력에는 없지만 앞에 적힌 로그의 시각이 뒤에 적힌 로그보다 늦을 때도 있다. 시각을 잰 순간과 로그를 쓴 순간 사이에 다른 스레드가 먼저 로그를 쓸 수 있기 때문이다. 동시에 실행되는 일의 로그는 적힌 순서가 일어난 순서와 다를 수 있다(그림 2-10). 그래서 로그마다 시각과 스레드 이름이 있어야 한다.

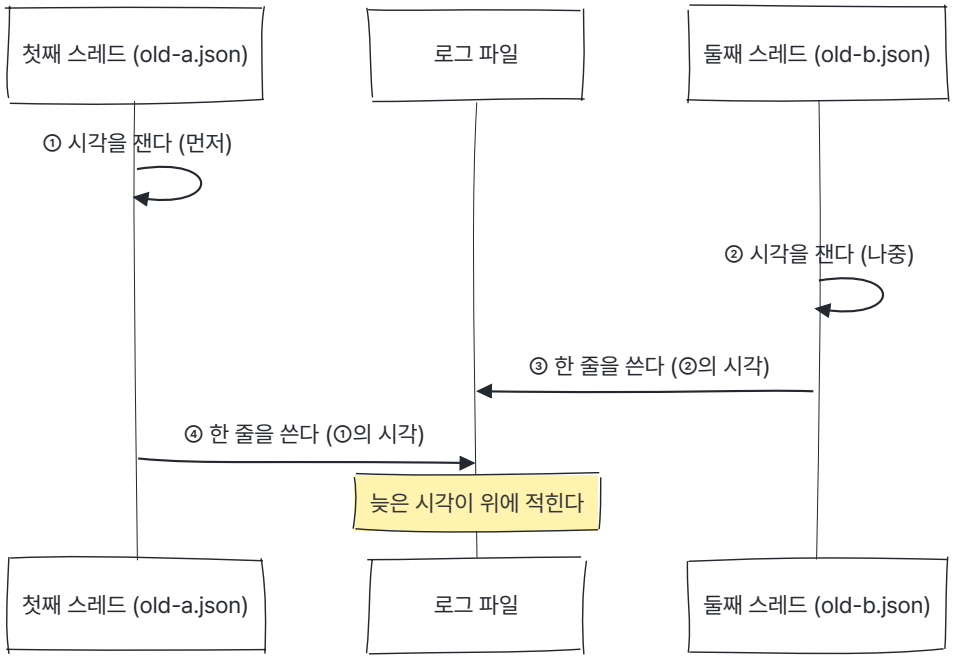



**그림 2-10** 먼저 시각을 잰 스레드가 나중에 쓰면, 늦은 시각의 로그가 파일에서 위에 온다

`Thread-0`이라는 이름은 무슨 일을 하는 스레드인지 알려 주지 않는다. 이름을 붙인 뒤에는 이렇게 기록되었다.

```
21:43:24.887 [import-2] INFO  n.j.blog.BlogMain - 읽기 시작: old-b.json
21:43:24.887 [import-1] INFO  n.j.blog.BlogMain - 읽기 시작: old-a.json
21:43:25.047 [import-2] INFO  n.j.blog.BlogMain - 읽기 끝: old-b.json, 글 20…
21:43:25.066 [import-1] INFO  n.j.blog.BlogMain - 읽기 끝: old-a.json, 글 30…
21:43:25.149 [main] INFO  n.j.blog.BlogMain - 옮기기 끝: 파일 2개, 글 5000개
```

이번에는 망가진 파일 `old-c.json`을 섞어 보았다. 터미널에 `WARN`이 나타났다.

터미널

```
$ java -jar target/blog-1.0.jar import old-a.json old-c.json
21:43:26 [import-2] WARN  BlogMain - 옮기지 못한 파일: old-c.json
java.io.IOException: 글 파일이 JSON 형식이 아니다: old-c.json
    … (생략: stack trace와 Caused by)
글 3000개를 옮겼습니다.
```

`[import-2]`만 보고도 둘째 파일을 맡은 스레드가 실패했고, 첫째 파일은 무사히 끝났음을 알 수 있다. 지금은 메시지에 파일 이름이 있어 스레드 이름이 없어도 짐작할 수 있다. 그런데 메시지만으로는 누가 남겼는지 알 수 없는 로그도 곧 생긴다. 여러 스레드가 요청을 나눠 처리하는 서버(4장)가 그런 경우다.

## 2.9 자기 환경에 대입하기

Gradle이라면 의존성은 `implementation`으로, Lombok은 `compileOnly`와 `annotationProcessor` 두 줄로 적는다. Maven의 `provided`와 `annotationProcessorPaths`가 하는 일을 나눠 적는 셈이다. Python의 `logging`, Go의 `log/slog`처럼 다른 언어의 로깅 도구도 이름만 다르다. 어느 환경에서든 다음 네 가지를 물어본다.

- 출력과 로그가 다른 통로로 나가는가?
- 코드를 고치지 않고 패키지마다 레벨을 바꿀 수 있는가?
- 로그마다 시각, 스레드, 남긴 곳이 붙는가?
- 예외를 남길 때 원인의 사슬까지 남는가?

**개발 상식 자바 로깅 라이브러리의 계보**

**여러 개인 이유.** 자바에는 로깅 라이브러리가 여럿이다. JDK 1.4에 `java.util.logging`(JUL)이 들어오기 전부터 Log4j가 널리 쓰였다. 둘을 함께 감싸려고 Commons Logging이 나왔고, Commons Logging의 문제를 고치려고 SLF4J가 나왔다. Logback은 SLF4J API를 직접 구현한 라이브러리다. Log4j도 Log4j 2로 새로 쓰였다. 그래서 오래된 라이브러리 하나를 가져와 보면, 그 안에서 JUL이나 Log4j를 쓰고 있는 경우가 흔하다.

**브리지.** 이럴 때 쓰는 것이 브리지다. `jul-to-slf4j`는 JUL로 남긴 로그를 SLF4J로 넘기고, `log4j-over-slf4j`는 Log4j 1과 같은 API를 제공하면서 받은 로그를 SLF4J로 전달한다. 이렇게 하면 라이브러리가 무엇을 쓰든 모든 로그가 Logback 하나로 모인다. 2.3절에서 본 "라이브러리는 API에, 구현체는 프로그램이"를 옛 라이브러리에까지 넓히는 셈이다.

**구현체가 둘이면.** SLF4J는 여러 provider를 찾았다고 경고하고 그중 하나를 고른다. 내가 고른 것이 아닐 수도 있다. 의존성을 추가한 뒤 로그 형식이 바뀌었다면 `mvn dependency:tree`로 구현체가 둘 들어오지 않았는지 확인한다.

**로그 라이브러리도 의존성이다.** 2021년 말 Log4j 2에서, 로그에 남긴 문자열 하나로 원격 코드를 실행할 수 있는 취약점이 발견되었다. 로깅 라이브러리도 버전을 고정하고, 알려진 취약점이 있는지 검사해야 한다.

**그다음.** 이 장의 로그는 사람이 읽는 한 줄 글이다. 운영에서는 로그를 한곳에 모아 검색한다. 그래서 로그마다 필드가 정해진 JSON으로 남기고(구조화된 로그), 한 요청이 남긴 로그를 묶는 요청 ID를 붙인다.

## 이 장에서 배운 것

**표 2-4** 2장 정리

| 항목 | 핵심 |
|---|---|
| **출력과 로그** | `System.out`은 사용자에게 주는 출력이다. 디버깅 출력을 같은 통로에 실으면 결과와 섞이고, 멈출 수도 나눌 수도 없다. 콘솔 로그는 표준 오류로 내보내 결과와 통로를 나눈다. |

| 항목 | 핵심 |
|---|---|
| **SLF4J와 Logback** | SLF4J는 코드가 호출하는 API, Logback은 로그를 출력하는 구현체다. 여러 구현체 앞에 앞면 하나를 두는 이 구조가 퍼사드 패턴이다. 라이브러리는 API에만 의존하고 구현체는 프로그램이 고른다. 구현체가 없으면 SLF4J는 경고하고 로그를 버린다. |
| **레벨과 값** | 레벨은 "이 로그를 보는 사람이 무엇을 해야 하나"로 고른다. 값은 `{}`로, 예외는 마지막 인자로 통째로 넘긴다. 비밀번호, 토큰, 개인정보는 남기지 않고, 객체도 통째로 넘기지 않는다. |
| **Lombok `@Slf4j`** | 로거 선언을 컴파일할 때 만들어 준다. `provided` scope와 annotation processor를 함께 적고, `@Data`처럼 한 번에 많은 코드를 만드는 annotation은 무엇을 만드는지 알고 쓴다. |
| **`logback.xml`** | appender는 어디로, pattern은 어떤 모양으로, logger와 root는 어느 레벨부터를 정한다. 더 구체적인 로거 이름의 레벨이 우선한다. 로그는 위쪽 로거로도 전달되므로(additivity) appender는 `root`에만 연결한다. |
| **스레드와 로그** | 동시에 실행되는 일의 로그는 적힌 순서와 일어난 순서가 다를 수 있다. 로그마다 시각과 스레드 이름을 남기고, 스레드에는 하는 일을 나타내는 이름을 붙인다. |

이 장의 코드는 실습 저장소 `logging`의 `step1-exercise`부터 `step4-solution`까지의 브랜치에 있다(부록 B).

## 연습 문제

풀이는 실습 저장소 `logging`의 `exercises` 브랜치에 있다. 먼저 직접 풀어 본 뒤 비교해 보자.

1. [응용] `step1-solution`의 `pom.xml`에서 `slf4j-api` 의존성을 지우고 `logback-classic`만 남겨 보자. 빌드와 실행은 어떻게 되는가? 다음 명령으로 이유를 찾아보자.

```
터미널
$ mvn dependency:tree
```

2. [기본] `main`의 `log.error("명령 실패: {}", command, e)`를 `log.error("명령 실패: {}", e)`로 바꾸고, 망가진 `posts.json`으로 `list`를 실행해 보자. 메시지와 스택 트레이스는 어떻게 남는가? 2.4절의 규칙으로 설명해 보자.

3. [기본] 2.6절에서 `net.java21` 로거를 `DEBUG`로 바꾸자 `write` 한 번이 로그 네 개를 남겼다. `step3-solution`의 `logback.xml`에서 이 로거를 `TRACE`로 바꾸고(다시 빌드한다) 같은 `write` 명령을 실행하면 `write`가 `logs/blog.log`에 남기는 로그는 몇 개인가(빌드 중 테스트가 남긴 줄은 빼고 센다)? `TRACE`에서만 나오는 로그는 무엇이고, 2.4절의 표 2-2에서 어느 예에 해당하는가?
4. [응용] 2.8절에서 망가진 파일을 섞었을 때, 터미널에는 나오지 않았지만 `logs/blog.log`의 마지막 로그는 "옮기기 끝: 파일 2개, 글 3000개"였다. 이 로그를 나중에 읽는 사람은 무엇을 오해할 수 있을까? 어떻게 고치겠는가?
5. [도전] 내 코드를 2.7절의 세 질문으로 스스로 리뷰해 보자. 고치고 싶은 로그가 생기면 바꾸기 전에 실제로 실행해, 이 장의 기준에 맞는지 확인해 보자.

---

## 마르코의 질문

**상황:** 코드 리뷰에서 동료가 묻는다.

**"디버깅할 때 쓰던 `System.out.println` 몇 줄 남은 건 그냥 둬도 되지 않아요? 어차피 콘솔에만 나오는데."**

**짚어야 할 것**

- 프로그램의 출력과 섞인다. 출력을 파일로 받거나 다른 프로그램에 넘기는 순간 그 출력이 결과물에 들어간다.
- 레벨이 없어 일부만 골라 남기거나 막을 수 없다. 운영에서 문제가 생겨도 그 문제와 관련된 출력만 따로 남길 방법이 없고, 시각과 스레드가 없어 다른 로그와 이어 읽기 어렵다.
- 무엇을 출력하는지 확인한다. 객체를 통째로 출력했다면 `toString()`이 개인정보를 내보낼 수 있다.

**빠지기 쉬운 답:** "몇 줄인데요, 뭐." 줄 수가 아니라 통로가 문제다. 필요한 출력이면 레벨을 정해 로그로 바꾸고, 필요 없으면 지운다.

· · ·

> **두원** 이제 로그 파일만 열면 누가 언제 뭘 했는지 다 보여. 스레드 이름까지.

**마르코** 좋아. 그럼 이 블로그, 친구가 브라우저로 보게 하려면 어떻게 해야 할까?

**두원** 웹에 올려야지. 서버로 만들어서.

**마르코** 웹에 올리면 여러 사람이 동시에 들어올 텐데, 그때 어느 요청이 뭘 했는지는 어떻게 알지?

**두원** 요청마다 스레드가 하나씩 붙는다면… 스레드 이름을 보면 되겠네. 오늘 본 `import-1`, `import-2`처럼.

**마르코** 그럼 그 스레드가 어디서 생기는지, 한번 직접 만들어 볼래? 소켓 하나에서 시작해서.